Planeación de acciones para robots de servicio mediante modelos de lenguaje y sistemas basados en reglas

Planeación de acciones para robots de servicio a partir de instrucciones en lenguaje natural mediante Qwen, CLIPS y modelos GPT

Luis Sergio Cano Olguin

Tutor: Dr. Jesús Savage Carmona

Agradecimientos

Resumen

Índice general

1. Introducción **1**

1.1. Contexto y Motivación 2
1.2. Planteamiento del problema 4
1.3. Hipótesis 5
1.4. Objetivos 5
1.5. Estructura del documento 6

2. Antecedentes y Estado del Arte **8**

2.1. Sistemas Basados en Reglas 8
2.2. El Lenguaje CLIPS: Características, Ventajas y Limitaciones 10
2.2.1. Características Fundamentales de CLIPS 10
2.3. Modelos de lenguaje para interpretación de instrucciones robóticas . . . 11
2.3.1. Familia de modelos Qwen 12
2.3.2. Modelos GPT 13
2.3.3. Interpretación semántica y generación de planes robóticos 14
2.3.4. Limitaciones de los modelos de lenguaje en robótica 15
2.3.5. Salidas estructuradas y validación 15
2.3.6. Adaptación eficiente de modelos: LoRA y QLoRA 16
2.4. Arquitecturas neuro-simbólicas 16
2.5. Síntesis y conexión con la propuesta 18

3. Formalización del problema y arquitectura conceptual **21**

3.1. Contexto arquitectónico y alcance del sistema 22
3.1.1. Integración dentro de la arquitectura ViRBot 22
3.2. GPSR como dominio de referencia 23
3.2.1. La prueba General Purpose Service Robot 23

3.2.2. Generación de comandos y conocimiento del dominio 25
3.2.3. Dominio de referencia y alcance lingüístico 26
3.3. Formalización de órdenes, objetivos y planes 27
3.3.1. Orden en lenguaje natural 28
3.3.2. Equivalencia semántica entre formulaciones 29
3.3.3. Entidades y conocimiento del dominio 29
3.3.4. Representación canónica de objetivos 31
3.3.5. Objetivos semánticos y acciones ejecutables 33
3.3.6. Dependencias y contexto entre objetivos 34
3.4. Arquitecturas conceptuales evaluadas 37
3.4.1. Base común de las arquitecturas evaluadas 37
3.4.2. Arquitectura Qwen–CLIPS 38
3.4.3. Arquitectura de planificación directa con GPT 41
3.4.4. Comparación conceptual de las arquitecturas 44
3.5. Criterios conceptuales de validez 46
3.5.1. Niveles de validez 46
3.5.2. Validez de la interpretación semántica 47
3.5.3. Validez y consistencia del plan 48
3.5.4. Validez de la respuesta del sistema 49
3.5.5. Condición de control para la etapa simbólica 49
3.5.6. Planificación y ejecución física 50

4. Metodología y diseño de las arquitecturas evaluadas 52
4.1. Diseño metodológico de la investigación 53
4.1.1. Problema experimental 53
4.1.2. Variables y condiciones de comparación 53
4.1.3. Estrategia general de evaluación 54
4.2. Construcción del dominio experimental 55
4.2.1. Fuentes de conocimiento GPSR 55
4.2.2. Catálogos de personas, objetos y ubicaciones 56
4.2.3. Familias de tareas 56
4.2.4. Representación de los objetivos de referencia 57
4.3. Construcción del conjunto de datos 58
4.3.1. Adaptación del CommandGenerator 58
4.3.2. Generación de órdenes 59

4.3.3. Generación de objetivos canónicos 60
4.3.4. Variación lingüística 60
4.3.5. Balance y deduplicación 61
4.3.6. División de entrenamiento y evaluación 62
4.3.7. Validación previa del conjunto de datos 62
4.4. Diseño de la arquitectura Qwen–CLIPS 64
4.4.1. Función asignada a Qwen 64
4.4.2. Especialización para interpretación semántica 64
4.4.3. Interfaz entre objetivos y planificación simbólica 65
4.4.4. Función asignada a CLIPS 66
4.5. Diseño de la arquitectura GPT directo 66
4.5.1. Funciones asignadas al modelo 66
4.5.2. Conocimiento y restricciones proporcionadas 67
4.5.3. Contrato de salida 67
4.5.4. Plan ejecutable, aclaración y rechazo 68
4.6. Diseño de la comparación 68
4.6.1. Igualdad del dominio de evaluación 69
4.6.2. Igualdad del repertorio de acciones 69
4.6.3. Separación entre interpretación y planificación 70

5. Implementación 71
5.1. Plataforma experimental 72
5.1.1. Robot Justina 72
5.1.2. Plataforma de cómputo 72
5.1.3. Entorno de software y ROS 2 73
5.2. Implementación del conjunto de datos 73
5.2.1. Generador de órdenes 73
5.2.2. Catálogos y CompetitionTemplate 74
5.2.3. Validadores 74
5.2.4. Dataset final 75
5.3. Entrenamiento e inferencia de Qwen2.5-7B 76
5.3.1. Preparación del modelo 76
5.3.2. Configuración QLoRA 77
5.3.3. Hiperparámetros de entrenamiento 77
5.3.4. Selección y conservación del checkpoint 78

5.3.5. Inferencia 81
5.3.6. Despliegue del modelo 82
5.4. Implementación del planificador CLIPS 83
5.4.1. Conversión de objetivos a hechos gpsr-goal 83
5.4.2. Representación del estado simbólico 86
5.4.3. Reglas de expansión 86
5.4.4. Generación de acciones 88
5.4.5. Condición de finalización 89
5.5. Integración Qwen–CLIPS con Justina 90
5.5.1. Componentes ROS 2 90
5.5.2. Flujo de comunicación 91
5.5.3. Recepción del plan y alcance de las pruebas 92
5.6. Ejemplo integrado de funcionamiento de Qwen–CLIPS 93
5.6.1. De la instrucción a los hechos simbólicos 93
5.6.2. Aplicación de reglas y evolución del estado 95
5.6.3. Finalización y construcción de la salida 96
5.6.4. Plan resultante 96
5.6.5. Programa experimental Qwen–CLIPS 97
5.7. Implementación del planificador GPT 98
5.7.1. Interfaz con la API 98
5.7.2. System prompt 99
5.7.3. JSON Schema y Structured Outputs 99
5.7.4. Generación de objetivos y planes 100
5.7.5. Tratamiento de aclaraciones y rechazos 101
5.7.6. Validación determinista 101
5.8. Reproducibilidad 102
5.8.1. Versiones de modelos 102
5.8.2. Configuraciones experimentales 103
5.8.3. Registro de ejecuciones 105

6. Validación y experimentación 108
6.1. Diseño experimental 108
6.1.1. Conjuntos de desarrollo y prueba 109
6.1.2. Procedimiento experimental 110
6.1.3. Repeticiones y control de variabilidad 112

6.2. Condiciones experimentales . . . 113
6.2.1. Arquitectura Qwen–CLIPS . . . 113
6.2.2. Arquitectura GPT directo . . . 113
6.2.3. Control con objetivos de referencia y CLIPS . . . 114
6.3. Evaluación de la interpretación semántica . . . 115
6.3.1. Exactitud de objetivos . . . 115
6.3.2. Validez estructural . . . 115
6.3.3. Entidades y parámetros . . . 115
6.3.4. Dependencias entre objetivos . . . 116
6.4. Evaluación de la planificación . . . 116
6.4.1. Acciones válidas . . . 117
6.4.2. Consistencia del plan . . . 117
6.4.3. Cumplimiento de subtareas . . . 117
6.4.4. Precondiciones y dependencias . . . 118
6.4.5. Planificabilidad y terminación . . . 118
6.5. Clasificación de errores . . . 119
6.5.1. Errores de interpretación . . . 119
6.5.2. Errores de adaptación y planificación . . . 119
6.5.3. Errores combinados y alcance de la atribución . . . 119
6.6. Robustez, límites del dominio y alcance físico . . . 120
6.6.1. Variación lingüística y composición . . . 120
6.6.2. Ambigüedad, restricciones y capacidades no verificables . . . 120
6.6.3. Delimitación de la evaluación física . . . 121
6.7. Rendimiento computacional . . . 121
6.7.1. Latencia . . . 121
6.7.2. Uso de recursos . . . 122
6.7.3. Tokens y coste del modelo GPT . . . 122

7. Resultados **123**
7.1. Resultados de la especialización de Qwen . . . 124
7.1.1. Comportamiento durante entrenamiento . . . 124
7.1.2. Evaluación del modelo seleccionado . . . 124
7.2. Resultados de interpretación semántica . . . 124
7.2.1. Qwen . . . 124
7.2.2. GPT . . . 124

7.2.3. Comparación 124
7.3. Resultados de planificación 124
7.3.1. Qwen–CLIPS 124
7.3.2. GPT directo 124
7.3.3. Condición de control CLIPS 124
7.3.4. Comparación 124
7.4. Análisis de errores 124
7.4.1. Errores de interpretación 124
7.4.2. Errores de planificación 124
7.4.3. Casos representativos 124
7.5. Resultados de ejecución física 124
7.6. Rendimiento computacional 124
7.6.1. Latencia 124
7.6.2. Recursos computacionales 124
7.6.3. Tokens y coste 124

8. Discusión **125**
8.1. Interpretación de los resultados respecto a la hipótesis 126
8.2. Interpretación semántica 126
8.2.1. Especialización de Qwen 126
8.2.2. Capacidad de GPT 126
8.2.3. Efecto de la variabilidad lingüística 126
8.3. Planificación de acciones 126
8.3.1. Expansión determinista mediante CLIPS 126
8.3.2. Planificación generativa mediante GPT 126
8.3.3. Separación entre errores de interpretación y planificación 126
8.4. Comparación Qwen–CLIPS frente a GPT directo 126
8.4.1. Exactitud 126
8.4.2. Consistencia 126
8.4.3. Latencia 126
8.4.4. Recursos y coste 126
8.4.5. Dependencia de infraestructura externa 126
8.5. Generalización fuera de las plantillas GPSR 126
8.6. Limitaciones y amenazas a la validez 126
8.6.1. Dominio de tareas 126

8.6.2. Dataset 126
8.6.3. Modelo Qwen 126
8.6.4. Modelo GPT 126
8.6.5. Plataforma física 126
8.7. Implicaciones para la planificación en robots de servicio 126

9. Conclusiones y trabajo futuro **127**
9.1. Conclusiones principales 127
9.2. Cumplimiento de los objetivos 127
9.3. Evaluación de la hipótesis 127
9.4. Contribuciones de la investigación 127
9.4.1. Representación canónica de órdenes 127
9.4.2. Arquitectura Qwen–CLIPS 127
9.4.3. Comparación con planificación directa mediante GPT 127
9.4.4. Metodología para separar errores de interpretación y planificación 127
9.5. Trabajo futuro 127

Índice de figuras

3.1. Arquitectura general ViRBot del robot Justina. 23
3.2. Arquitectura conceptual Qwen–CLIPS. 40
3.3. Arquitectura conceptual del planificador GPT directo. 42

5.1. Evolución de las pérdidas de entrenamiento y validación durante el ajuste de Qwen2.5-7B mediante QLoRA. La pérdida de entrenamiento se registra cada 50 actualizaciones y la de validación cada 250. Se muestra el historial completo, hasta el paso 9 000. Elaboración propia. 80
5.2. Transformaciones de la instrucción del ejemplo: orden, objetivos predichos y campos de los hechos `gpsr-goal`. La predicción de Qwen es ilustrativa. Cada fila de la tabla representa un hecho construido por el adaptador. Elaboración propia. 94
5.3. Expansión de los cuatro objetivos y evolución del estado simbólico. Se muestran las acciones operativas principales; los anuncios y las frases asociadas se conservan en el plan completo. Los cambios de estado representan efectos previstos durante la planificación. Elaboración propia. 95

Capítulo 1

Introducción

Los robots de servicio representan un sector de gran crecimiento para la inteligencia artificial y la automatización, con aplicaciones que van desde la asistencia a personas mayores hasta la gestión autónoma de entornos residenciales (1), donde deben interpretar instrucciones expresadas por personas y convertirlas en acciones que puedan ejecutarse mediante módulos de navegación, percepción, manipulación e interacción (2).

La transformación de una instrucción en lenguaje natural a un plan robótico presenta dos problemas relacionados. El primero es semántico: identificar correctamente los objetivos, entidades, destinos y relaciones expresados por el usuario. El segundo es simbólico: convertir dicha interpretación en una secuencia de acciones pertenecientes al repertorio real del robot y con parámetros compatibles con el ejecutor.

Los sistemas basados en reglas han demostrado ser robustos y predecibles en escenarios estructurados, como líneas de producción o laboratorios controlados (3). En particular, CLIPS utiliza hechos y reglas de producción para determinar qué transformaciones deben aplicarse a partir de un estado simbólico. Sin embargo, su rigidez los hace poco adaptables ante variaciones no previstas en el entorno o ante comandos expresados en lenguaje natural con alto grado de ambigüedad o complejidad (4).

Recientemente, los modelos de lenguaje grande (LLMs), como ChatGPT, han emergido como herramientas capaces de interpretar y generar lenguaje natural, e incluso de actuar como agentes de planificación autónomos (5). Su flexibilidad contextual permite traducir instrucciones verbales en secuencias de acciones, complementando así la solidez de los sistemas basados en reglas. No obstante, su naturaleza probabilística puede producir formatos inválidos, entidades inexistentes o acciones que no pertenecen a las

capacidades del robot. Por lo tanto, la falta de garantías formales de seguridad limita su uso directo en aplicaciones robóticas donde la integridad física y la predictibilidad son prioritarias (6).

Esta tesis estudia dos aproximaciones para resolver el problema. La primera es una arquitectura neuro-simbólica que utiliza Qwen2.5 como intérprete semántico, con dos configuraciones basadas en las variantes Qwen2.5-3B y Qwen2.5-7B, ambas ajustadas al dominio mediante QLoRA. Las denominaciones 3B y 7B indican tamaños nominales de aproximadamente tres mil millones y siete mil millones de parámetros, respectivamente. En ambas configuraciones, el modelo transforma una orden en objetivos canónicos representados en JSON. Posteriormente, un módulo determinista normaliza y convierte esos objetivos en hechos, que son procesados por reglas de producción en CLIPS para generar el plan compatible con el ejecutor del robot. La segunda aproximación emplea un modelo GPT como planificador generativo directo. Este recibe la orden, el contexto y el catálogo de acciones, y produce una respuesta estructurada mediante JSON Schema que la somete a validación determinista.

Las dos aproximaciones no se combinan mediante un selector de planes durante la ejecución. Se implementan como arquitecturas independientes y se comparan experimentalmente bajo el mismo conjunto de órdenes, contexto y criterios de evaluación. Esta comparación permite analizar el compromiso entre interpretación semántica, consistencia, trazabilidad, flexibilidad, latencia y coste de uso.

1.1. Contexto y Motivación

El área de la robótica orientada al servicio doméstico ha evolucionado significativamente en la última década, impulsada por avances en percepción, planificación y interacción humano-robot (1).

El robot de servicio Justina, desarrollado en el Laboratorio de Bio-Robótica de la UNAM, integra módulos de percepción, navegación, manipulación e interacción mediante ROS 2. Para ejecutar una orden, el robot necesita una representación intermedia que conecte el lenguaje del usuario con las acciones ofrecidas por dichos módulos.

Estos sistemas se han diseñado para operar en entornos no estructurados como hogares, hospitales o centros de cuidado, donde deben realizar tareas que van desde la entrega de objetos hasta la asistencia en actividades de la vida diaria. En este contexto, la capacidad de planificar acciones de manera autónoma, segura y adaptativa se

convierte en un requisito fundamental.

La planificación de acciones en robótica ha sido tradicionalmente abordada mediante sistemas basados en reglas *(Rule-Based Systems, RBS)*, que ofrecen un marco predecible y verificable para la toma de decisiones (2). Sin embargo, una solución exclusivamente simbólica puede generar planes consistentes cuando recibe objetivos correctamente estructurados, pero no resuelve directamente la interpretación del lenguaje natural. En contraste, un modelo de lenguaje puede interpretar distintas formulaciones de una orden, aunque su salida no es necesariamente compatible con las restricciones del robot. La motivación de este trabajo es evaluar si la separación entre interpretación neuronal y expansión simbólica permite obtener planes ejecutables y trazables, y contrastar sus resultados con los de un planificador basado exclusivamente en un modelo GPT.

El interés de la comparación no consiste en demostrar que un paradigma es universalmente superior. Se busca identificar qué arquitectura presenta ventajas dependiendo del comando, así como qué errores introduce cada etapa y cuáles son los costes asociados con su despliegue local o mediante una API externa.

Para la evaluación comparativa se implementan dos programas independientes: uno ejecuta la cadena Qwen–CLIPS con el intérprete seleccionado y otro consulta al planificador GPT directo y valida su salida. Se consideran tres configuraciones experimentales: Qwen2.5-3B–CLIPS, Qwen2.5-7B–CLIPS y GPT directo. Todas reciben las mismas órdenes y contexto equivalente, y entregan objetivos y planes en un formato común para aplicar los mismos criterios de evaluación. En las dos configuraciones Qwen–CLIPS se mantienen el mismo normalizador, adaptador, reglas y estado simbólico inicial. La comparación examina las salidas de interpretación y planificación sin ejecutar físicamente los planes. La experiencia de Qwen–CLIPS integrado en Justina durante la RoboCup 2026 se documenta por separado.

Las dos variantes de Qwen se ajustaron mediante QLoRA utilizando el mismo conjunto de datos, la misma partición de entrenamiento y validación, los mismos hiperparámetros y el mismo criterio de selección del checkpoint. En el código de entrenamiento se modificó únicamente el identificador del modelo base, de `Qwen/Qwen2.5-7B` a `Qwen/Qwen2.5-3B`. Esta configuración permite comparar ambas variantes bajo un procedimiento de ajuste común.

1.2. Planteamiento del problema

Una orden puede contener variaciones lingüísticas, referencias espaciales, objetos, personas, destinos y varias subtareas relacionadas. Para que el robot pueda ejecutarla, primero deben identificarse sus objetivos y después transformarse en una secuencia válida de acciones primitivas. En la arquitectura propuesta, Qwen realiza la interpretación lingüística y CLIPS utiliza hechos y reglas para expandir los objetivos en un plan.

Una solución basada exclusivamente en reglas enfrenta las siguientes limitaciones:

1. **Rigidez interpretativa:** No se pueden procesar comandos en lenguaje natural complejo o ambiguo.
2. **Falta de adaptabilidad:** Las reglas deben ser definidas a priori; cualquier situación no prevista en la base de conocimiento puede llevar al fracaso o a un comportamiento no deseado.
3. **Dificultad para gestionar la incertidumbre:** No manejan bien la información incompleta o cambiante del entorno en tiempo real.
4. **Escalabilidad limitada:** Mantener y extender un conjunto grande de reglas para cubrir todos los escenarios posibles resulta complejo y laborioso.

Por otro lado, los resultados deben satisfacer al menos las siguientes condiciones (6):

- Las entidades finales deben pertenecer al catálogo o ser tratadas explícitamente como desconocidas.
- Los objetivos deben conservar el significado y el orden causal de la instrucción.
- Las acciones generadas deben pertenecer al repertorio del ejecutor.
- Cada acción debe contener los parámetros requeridos.
- La secuencia debe respetar precondiciones simbólicas básicas, como la ubicación del robot o la posesión de un objeto.

Los modelos de lenguaje pueden ampliar la cobertura lingüística, aunque también pueden generar resultados inexistentes o inconsistentes. El problema de investigación se formula, por tanto, de la siguiente manera:

¿Qué diferencias de exactitud, consistencia, robustez y coste existen entre una arquitectura neuro-simbólica Qwen–CLIPS y un planificador generativo basado en GPT al convertir órdenes en planes para un robot de servicio?

Dentro de la arquitectura neuro-simbólica, se examina además cómo cambian la exactitud de la interpretación, la consistencia de los planes, la robustez y la latencia al utilizar Qwen2.5-3B o Qwen2.5-7B como intérprete semántico.

1.3. Hipótesis

Para la interpretación y planeación de órdenes, una arquitectura neuro-simbólica compuesta por Qwen y CLIPS producirá una mayor proporción de planes estructuralmente válidos, trazables y consistentes que un planificador generativo basado directamente en GPT. Por otra parte, se espera que el planificador GPT directo presente mayor flexibilidad ante formulaciones lingüísticas novedosas; también se compararán la variabilidad, la latencia y el coste de inferencia de ambas arquitecturas.

La hipótesis se contrasta mediante la evaluación de los objetivos interpretados y los planes generados por Qwen2.5-3B–CLIPS, Qwen2.5-7B–CLIPS y GPT directo ante un mismo conjunto de órdenes y contextos. Los resultados de cada configuración se presentan por separado, sin presuponer que el modelo Qwen de mayor tamaño obtendrá un mejor desempeño. Las conclusiones se circunscriben a los modelos y condiciones evaluados. La participación de Justina en la RoboCup 2026 utilizando Qwen–CLIPS se presenta como evidencia de aplicación del sistema integrado y no forma parte de esta comparación controlada.

1.4. Objetivos

Diseñar, implementar y evaluar una arquitectura neuro-simbólica Qwen–CLIPS para interpretar órdenes y generar planes compatibles con un robot de servicio, comparándola con un planificador generativo basado en GPT con salida estructurada.

- **Objetivos específicos**
  - Definir el dominio operacional evaluado, una representación canónica de objetivos y su correspondencia con las acciones y efectos documentados del ejecutor.
  - Ajustar Qwen2.5-3B y Qwen2.5-7B mediante QLoRA para convertir órdenes en objetivos canónicos representados en JSON.
  - Implementar la normalización, validación y conversión de los objetivos generados por Qwen en hechos.
  - Implementar reglas de producción en CLIPS que expandan objetivos de alto nivel en secuencias de acciones compatibles con el ejecutor.
  - Diseñar un planificador GPT directo mediante un prompt, un JSON Schema y un validador determinista.

  - Construir un conjunto experimental que distinga nuevas instancias de las estructuras admitidas por el generador, variaciones lingüísticas que preserven su intención, composiciones no contempladas por dichas estructuras y entradas ambiguas, contradictorias o que excedan las capacidades disponibles, documentando su relación con los datos de entrenamiento.
  - Comparar Qwen2.5-3B–CLIPS, Qwen2.5-7B–CLIPS y GPT directo en fidelidad semántica, conformidad y cumplimiento simbólico del plan, robustez lingüística, latencia, variabilidad y coste computacional observable, analizando por separado el desempeño dentro del dominio operacional y el comportamiento fuera de sus estructuras o capacidades.
  - Documentar la integración y el desempeño de Qwen–CLIPS en Justina durante la RoboCup 2026, distinguiendo esta experiencia de la comparación experimental de planes.
- **Contribuciones esperadas**
  - Una representación intermedia de objetivos que conecta la interpretación lingüística con la planeación simbólica.
  - Dos variantes de Qwen2.5, de 3B y 7B, ajustadas mediante QLoRA para traducir órdenes a objetivos canónicos.
  - Una integración ROS 2–CLIPS que convierte dichos objetivos en planes compatibles con el ejecutor.
  - Un planificador GPT directo restringido mediante prompt, JSON Schema y validación determinista.
  - Un protocolo experimental reproducible y un conjunto documentado de evaluación que distingan cobertura del dominio, generalización lingüística y composicional, y comportamiento ante límites de capacidad.
  - Un análisis de errores que diferencia fallos de interpretación, adaptación y planificación en el experimento, y documenta por separado los fallos de ejecución observados en Justina durante la RoboCup 2026.

1.5. Estructura del documento

Este documento está organizado en los siguientes capítulos que describen el sistema propuesto para el desarrollo de esta tesis, abarcando el planteamiento, desarrollo, implementación y la evaluación de las arquitecturas estudiadas. La estructura es la

siguiente:

1. **Capítulo 1: Introducción.** Presenta el contexto, motivación, problemática, hipótesis y objetivos de la investigación, así como la justificación del enfoque híbrido en robótica de servicio doméstico.
2. **Capítulo 2: Antecedentes y estado del arte.** Revisa la evolución de los sistemas basados en reglas en robótica, las características de CLIPS, los enfoques modernos de aprendizaje automático y los sistemas híbridos existentes.
3. **Capítulo 3: Marco teórico y conceptual.** Describe los fundamentos de la planificación jerárquica, los sistemas de producción, los modelos de lenguaje grande (LLMs) y la arquitectura de integración propuesta.
4. **Capítulo 4: Metodología y diseño de las arquitecturas evaluadas.** Describe las dos arquitecturas evaluadas, la representación de objetivos y planes, y el protocolo experimental utilizado para comparar Qwen–CLIPS con GPT directo.
5. **Capítulo 5: Implementación.** Explica los entornos de desarrollo, la configuración técnica, la interfaz con APIs y las pruebas utilizados para validar la integración de los módulos.
6. **Capítulo 6: Escenarios de validación y experimentación.** Presenta el diseño experimental, los escenarios de prueba y las métricas cuantitativas y cualitativas utilizadas para evaluar el sistema.
7. **Capítulo 7: Resultados.** Presenta la comparación de los planes generados por ambas arquitecturas, el análisis de errores y los resultados de latencia, recursos computacionales y coste. Documenta por separado el desempeño observado de Qwen–CLIPS en Justina durante la RoboCup 2026.
8. **Capítulo 8: Discusión.** Interpreta los resultados y sus limitaciones, incluyendo el alcance de la evaluación de planes y de la experiencia con el robot en la competencia.
9. **Capítulo 9: Conclusiones y trabajo futuro.** Resume los hallazgos, las contribuciones y las posibles mejoras de las arquitecturas estudiadas.

Capítulo 2

Antecedentes y Estado del Arte

Este capítulo presenta los trabajos y tecnologías relacionados con la transformación de instrucciones de lenguaje natural a planes para robots de servicio. La revisión se concentra en cuatro componentes relevantes para la tesis: sistemas basados en reglas, CLIPS, modelos de lenguaje para interpretación y planificación, y arquitecturas neuro-simbólicas. Los métodos de percepción, navegación o control se mencionan únicamente cuando contribuyen directamente a la interfaz entre la orden y el plan de tareas.

Justina, el robot de servicio del Laboratorio de Bio-Robótica de la Facultad de Ingeniería de la UNAM, constituye la plataforma robótica de referencia de esta tesis. Sus capacidades de navegación, percepción, manipulación e interacción verbal delimitan el dominio de tareas y el repertorio de acciones considerados en las propuestas de planificación. La comparación experimental se realiza mediante programas independientes que generan planes para este dominio, sin ejecutarlos físicamente en el robot. La integración de la arquitectura Qwen–CLIPS en Justina y su utilización durante la RoboCup 2026 se documentan por separado.

2.1. Sistemas Basados en Reglas

Los sistemas basados en reglas *(Rule-Based Systems, RBS)* constituyen uno de los paradigmas clásicos de la inteligencia artificial (3). Su utilización se consolidó con el desarrollo de los sistemas expertos durante las décadas de 1970 y 1980, en los que el conocimiento de especialistas humanos era representado explícitamente mediante hechos y reglas dentro de dominios bien delimitados (2). En robótica, la adopción de estos sistemas se popularizó en los años 80 y 90, particularmente en entornos industriales

estructurados, donde la previsibilidad y seguridad eran prioritarias.

Este tipo de representación resulta útil cuando las condiciones bajo las cuales deben ejecutarse determinadas acciones pueden expresarse de manera explícita. Las reglas permiten relacionar estados, eventos u objetivos con decisiones específicas, facilitando la construcción de comportamientos reproducibles dentro de dominios previamente definidos. Entre las herramientas desarrolladas para implementar este paradigma se encuentra CLIPS *(C Language Integrated Production System)*, desarrollado originalmente por la NASA. CLIPS proporciona mecanismos para representar conocimiento mediante hechos y reglas de producción, junto con un motor de inferencia encargado de determinar qué reglas deben activarse de acuerdo con el estado de la base de conocimiento (7).

En cuanto a aplicaciones, los sistemas basados en reglas han demostrado su utilidad en diversos dominios robóticos:

1. **Robótica industrial:** cadenas de montaje automatizadas, donde las tareas son repetitivas y el entorno está controlado. Los RBS se utilizan para secuenciar operaciones, gestionar excepciones y garantizar la seguridad de los operarios humanos (1).
2. **Robótica de servicio y doméstica:** en tareas como la entrega de objetos, la navegación en interiores y la asistencia a personas con movilidad reducida, donde las reglas permiten asociar condiciones del entorno con respuestas concretas del robot (**?**).
3. **Robótica espacial y de exploración:** sistemas como Remote Agent de la NASA emplearon arquitecturas basadas en reglas para la planificación y ejecución autónoma de experimentos en misiones no tripuladas (8).

Su principal ventaja para esta tesis es que permiten establecer una correspondencia explícita entre un objetivo simbólico y las acciones emitidas para el robot. Sin embargo, la cobertura del sistema está limitada por las reglas y los tipos de hechos definidos previamente. Para abordar la variabilidad lingüística de las instrucciones, esta investigación incorpora un componente neuronal de interpretación, descrito en la sección 2.3. Este componente transforma las órdenes en objetivos compatibles con el sistema simbólico, pero no amplía por sí mismo la cobertura de las reglas de planificación.

2.2. El Lenguaje CLIPS: Características, Ventajas y Limitaciones

CLIPS es un lenguaje de programación basado en reglas, diseñado específicamente para la construcción de sistemas expertos y aplicaciones basadas en conocimiento (7). Concebido originalmente como una alternativa a los costosos sistemas implementados en *(LISP, LISt Processing)* en estaciones de trabajo especializadas, CLIPS destacó por su portabilidad, eficiencia y su capacidad para ejecutarse en hardware de bajo costo (3). Su adopción se extendió rápidamente en la comunidad de robótica e inteligencia artificial, convirtiéndose en una herramienta de referencia para la implementación de sistemas de producción (2).

2.2.1. Características Fundamentales de CLIPS

CLIPS es un entorno para el desarrollo de sistemas basados en conocimiento que ofrece hechos, plantillas, reglas de producción, funciones y un motor de inferencia que permite razonar sobre una base de conocimiento mediante reglas de producción (3).

En el contexto de esta tesis, las propiedades relevantes de CLIPS son:

1. **Encadenamiento hacia adelante *(forward chaining)*:** CLIPS utiliza predominantemente este encadenamiento. A partir de los hechos presentes en la base de conocimiento, el motor identifica las reglas cuyas condiciones se satisfacen y las incorpora a la agenda de ejecución. La ejecución de estas reglas puede modificar posteriormente la base de hechos y habilitar nuevas reglas (7).
2. **Representación explícita del conocimiento:** además de reglas, CLIPS permite representar conocimiento mediante hechos ordenados y plantillas *(deftemplate)*, así como mediante funciones definidas por el usuario en el lenguaje de programación C (7). Esto facilita la integración con sistemas de percepción y control de bajo nivel (9).
3. **Trazabilidad de las reglas activadas:** debido a que las decisiones dependen de hechos y reglas explícitas, es posible inspeccionar las que son activadas durante una ejecución. Cuando se mantienen constantes los hechos iniciales, las reglas y la estrategia de resolución de conflictos, el comportamiento del motor puede reproducirse de manera determinista.
4. **Portabilidad e integración:** CLIPS fue implementado en C con el propósito de facilitar su utilización en diferentes plataformas. Esta característica permite

incorporar el motor de inferencia dentro de sistemas de software más amplios sin depender de una plataforma especializada (3,7). En el caso de Python, la comunicación puede realizarse mediante bindings externos que exponen las funciones del motor, lo que permite utilizar el sistema basado en reglas como un componente dentro de una arquitectura robótica más amplia.

Estas propiedades contribuyen a restringir y validar el comportamiento del sistema, pero no implican, por sí mismas, una garantía de seguridad ni constituyen un mecanismo de verificación formal.

En esta investigación, CLIPS recibe objetivos estructurados producidos por una etapa externa de interpretación y aplica una base de reglas definida previamente. La comunicación con ROS 2 corresponde a la interfaz desarrollada para integrar el motor con los nodos de Justina. El programa experimental independiente utiliza el motor de reglas dentro de su propia cadena de generación de planes, sin requerir la ejecución física de las acciones en el robot.

2.3. Modelos de lenguaje para interpretación de instrucciones robóticas

El desarrollo del aprendizaje automático, y en particular del aprendizaje profundo, ha ampliado las capacidades de los sistemas robóticos para procesar información compleja y entradas con un alto grado de variabilidad (10). A diferencia de los sistemas basados en reglas, en los que parte del conocimiento del dominio debe representarse explícitamente, los enfoques de aprendizaje automático permiten obtener representaciones y patrones a partir de datos (2).

En el procesamiento de información secuencial, arquitecturas como las redes neuronales recurrentes (RNN), LSTM y GRU permitieron modelar dependencias entre los elementos de una secuencia y fueron utilizadas en diferentes tareas relacionadas con el procesamiento del lenguaje natural (10). Posteriormente, la arquitectura Transformer introdujo un enfoque basado principalmente en mecanismos de atención, permitiendo modelar relaciones entre los elementos de una secuencia sin depender exclusivamente de recurrencia (5). Esta arquitectura constituye la base de gran parte de los modelos de lenguaje de gran escala desarrollados posteriormente.

Los *Large Language Models* (LLMs) basados en Transformers pueden procesar y generar lenguaje natural utilizando el contexto de una instrucción para producir di-

ferentes tipos de representaciones. En robótica, estos modelos se han utilizado para interpretar comandos complejos, traducir lenguaje natural a representaciones simbólicas e incluso generar planes a partir de descripciones de tareas de alto nivel (6). Estas capacidades resultan relevantes para esta investigación porque permiten utilizar un modelo de lenguaje como interfaz entre las expresiones proporcionadas por el usuario y las representaciones estructuradas utilizadas posteriormente por los componentes de planificación.

2.3.1. Familia de modelos Qwen

Qwen es una familia de modelos de lenguaje desarrollada por el equipo Qwen de Alibaba Cloud. La familia incluye modelos de diferentes tamaños y variantes destinadas a tareas generales y al seguimiento de instrucciones (11). Estos modelos están basados en la arquitectura Transformer y pueden utilizarse en tareas como generación de texto, respuesta a preguntas, extracción de información y producción de contenido estructurado.

Qwen2.5 corresponde a una generación de esta familia que incluye modelos con diferentes números de parámetros. En esta investigación se utilizan `Qwen/Qwen2.5-3B` y `Qwen/Qwen2.5-7B` como alternativas para el componente neuronal de interpretación. Las denominaciones 3B y 7B representan tamaños nominales de aproximadamente tres mil millones y siete mil millones de parámetros, respectivamente.

La elección de la familia Qwen2.5 respondió a criterios prácticos: la disponibilidad pública de sus pesos y su compatibilidad con las herramientas de ajuste eficiente empleadas en este trabajo. Durante el desarrollo se comprobó la viabilidad técnica de ajustar las variantes seleccionadas y obtener las representaciones estructuradas requeridas por la arquitectura. Esta comprobación permitió continuar con su evaluación experimental, pero no constituye por sí misma una demostración de la calidad de interpretación o de los planes resultantes. No se realizó un estudio comparativo sistemático frente a otras familias de modelos, por lo que la selección no pretende establecer una superioridad general de Qwen2.5 sobre alternativas como Llama, Mistral o Gemma.

En ambas configuraciones, Qwen funciona como intérprete semántico. Antes de consultar el modelo, la instrucción en lenguaje natural pasa por una etapa de normalización de entrada. A partir de esta instrucción, Qwen produce una secuencia de objetivos canónicos representada en formato JSON, que identifica los tipos de objetivo, las entidades, las ubicaciones y las relaciones interpretadas. Posteriormente, un módulo

determinista valida la salida y convierte los objetivos aceptados en hechos que pueden ser procesados por CLIPS. La normalización corresponde, por tanto, al tratamiento de la instrucción previo a la inferencia, mientras que la validación y la conversión a hechos se realizan sobre la salida del modelo.

Justificación de la selección de Qwen2.5-3B y Qwen2.5-7B

La selección de las variantes Qwen2.5-3B y Qwen2.5-7B responde a las características de la tarea y a las condiciones de cómputo disponibles. La interpretación de instrucciones requiere conservar las subtareas, entidades y relaciones expresadas por el usuario y producir una representación estructurada compatible con el componente simbólico.

Durante las etapas preliminares se realizaron pruebas con Qwen2.5-0.5B, que permitieron comprobar el flujo de entrenamiento y la representación de objetivos. En esas pruebas se observaron respuestas con consistencia insuficiente ante instrucciones complejas o con variabilidad lingüística. Estas observaciones orientaron el desarrollo hacia una variante de mayor tamaño, pero no constituyen una comparación exhaustiva de la familia Qwen2.5.

Qwen2.5-7B se adoptó durante el desarrollo por su viabilidad para realizar el ajuste y la inferencia con los recursos disponibles. Posteriormente se incorporó Qwen2.5-3B para examinar si una variante de menor tamaño puede mantener la calidad de interpretación y de los planes resultantes, y qué diferencias de latencia presenta frente a 7B.

Ambas variantes se ajustaron mediante QLoRA utilizando el mismo conjunto de datos, la misma partición de entrenamiento y validación, los mismos hiperparámetros y el mismo criterio de selección del checkpoint. En el código de entrenamiento se modificó únicamente el identificador del modelo base. Los detalles de estas configuraciones se presentan en el capítulo de implementación.

La comparación mantiene la función asignada al intérprete y la etapa de expansión mediante CLIPS. De este modo, permite estudiar el comportamiento de ambas variantes bajo un procedimiento de ajuste común, sin presuponer que el mayor tamaño producirá mejores resultados en todos los casos.

2.3.2. Modelos GPT

Los modelos GPT constituyen otra familia de modelos generativos basados en la arquitectura Transformer. En esta tesis, el modelo seleccionado se utiliza mediante la

API de OpenAI, en lugar de desplegar sus parámetros dentro de la infraestructura local. La API recibe la instrucción y el contexto proporcionado para la tarea y devuelve una respuesta generada por el modelo.

En el capítulo de implementación se especifican el modelo y la configuración utilizados durante los experimentos, debido a que una denominación general como "ChatGPT" no identifica de manera reproducible un modelo específico accesible mediante API (12).

En la arquitectura experimental GPT directo, el modelo recibe la orden, el catálogo de entidades, el repertorio de acciones, las restricciones del dominio y ejemplos de respuesta. A diferencia de Qwen implementado en la arquitectura neuro-simbólica, este modelo genera tanto la representación de los objetivos como la secuencia propuesta de acciones. Por ello, constituye un planificador generativo independiente y no un módulo interno de la cadena Qwen–CLIPS.

La respuesta del modelo se solicita mediante una salida estructurada definida a partir de JSON Schema. La documentación de la API denomina *Structured Outputs* al mecanismo mediante el cual la respuesta generada debe ajustarse al esquema proporcionado (12). Este mecanismo facilita la integración de la salida con los componentes posteriores del sistema, aunque el cumplimiento del esquema no garantiza por sí mismo la validez semántica del contenido generado. Los niveles de validación empleados se describen en la subsección 2.3.5.

El uso de una API externa introduce variables que deben incluirse en la evaluación, como latencia de comunicación, disponibilidad del servicio, tokens de entrada y salida, coste monetario y posibles cambios entre versiones del modelo. En consecuencia, las pruebas deben fijar y documentar el modelo, los parámetros de generación y el prompt.

2.3.3. Interpretación semántica y generación de planes robóticos

Para esta tesis conviene distinguir dos funciones:

Interpretación semántica: transformación de una orden en una representación intermedia de objetivos, entidades y relaciones.

Planificación generativa: producción directa de una secuencia de acciones a partir de la orden, el contexto y las capacidades declaradas del robot.

En la arquitectura Qwen–CLIPS estas funciones se mantienen separadas: Qwen realiza la interpretación semántica de la orden y CLIPS expande los objetivos obtenidos

en una secuencia de acciones. En la arquitectura GPT directo, ambas funciones se concentran en un único modelo, que produce tanto la representación de los objetivos como el plan propuesto. Esta distinción permite comparar el efecto de mantener una etapa simbólica explícita frente a delegar conjuntamente la interpretación y la composición del plan a un modelo generativo.

La interpretación semántica puede evaluarse comparando la secuencia de objetivos producida por el modelo con una anotación de referencia. La planificación requiere criterios adicionales: pertenencia de las acciones al repertorio del robot, validez de parámetros, conservación de dependencias, cumplimiento de precondiciones y compatibilidad con el ejecutor. Evaluar ambas etapas de forma separada permite localizar el origen de los errores.

2.3.4. Limitaciones de los modelos de lenguaje en robótica

El uso de un modelo de lenguaje introduce riesgos específicos: generación de entidades o acciones inexistentes, incumplimiento del formato, variabilidad entre ejecuciones, sensibilidad al *prompt* y costes de inferencia. Asimismo, una instrucción ambigua puede dar lugar a una interpretación plausible pero diferente de la intención del usuario.

Estas limitaciones son especialmente relevantes cuando la salida se utiliza para controlar un sistema físico. Por ello, una respuesta lingüísticamente coherente no debe considerarse automáticamente un plan ejecutable. En esta investigación se separan la generación, la validación estructural y la validación semántica; los módulos de ejecución del robot conservan la responsabilidad sobre restricciones físicas y condiciones observadas durante la tarea.

2.3.5. Salidas estructuradas y validación

Una estrategia para integrar modelos de lenguaje con software convencional consiste en restringir la salida a una estructura legible por máquina. JSON permite representar objetivos, parámetros y acciones, mientras que JSON Schema permite comprobar tipos, campos obligatorios, enumeraciones y restricciones estructurales.

No obstante, una respuesta conforme al esquema puede seguir siendo semánticamente incorrecta. Por ejemplo, puede incluir una ubicación que no pertenece al mapa o aplicar una acción válida al tipo de entidad equivocado. En consecuencia, la validación debe separar al menos tres niveles:

1. Sintaxis JSON.
2. Conformidad con el esquema.
3. Validez semántica respecto a catálogos, acciones y precondiciones.

Esta separación es aplicable tanto a los objetivos producidos por Qwen como al plan generado directamente por GPT.

2.3.6. Adaptación eficiente de modelos: LoRA y QLoRA

El ajuste completo de un modelo de lenguaje requiere actualizar todos sus parámetros y mantener estados adicionales durante el entrenamiento. Los métodos de adaptación eficiente en parámetros permiten reducir este coste al entrenar únicamente un subconjunto reducido de parámetros, manteniendo congelada gran parte del modelo preentrenado (13,14). LoRA (*Low-Rank Adaptation*) introduce matrices entrenables de bajo rango en determinadas capas del modelo, manteniendo congelados los pesos originales (14), mientras que QLoRA combina estos adaptadores con la cuantización del modelo base para reducir significativamente el uso de memoria durante el ajuste fino (15).

En esta investigación se utiliza QLoRA para especializar Qwen2.5-3B y Qwen2.5-7B en la conversión de órdenes a objetivos canónicos. Los detalles del conjunto de datos, cuantización, hiperparámetros, selección de checkpoints y plataforma se presentan en el capítulo de implementación.

2.4. Arquitecturas neuro-simbólicas

Las arquitecturas neuro-simbólicas buscan combinar las capacidades de aprendizaje y generalización de los modelos neuronales con mecanismos explícitos de representación de conocimiento y razonamiento simbólico. Mientras que los modelos neuronales son especialmente adecuados para procesar información no estructurada y aprender representaciones a partir de datos, los sistemas simbólicos permiten expresar conocimiento mediante estructuras discretas, reglas y relaciones que pueden ser utilizadas durante un proceso de inferencia (16, 17). El objetivo de este enfoque es aprovechar las fortalezas de ambos paradigmas en lugar de utilizar cada uno de manera aislada.

La integración entre ambos componentes puede realizarse de diferentes maneras. Un modelo neuronal puede, por ejemplo, transformar una entrada no estructurada en

una representación simbólica que posteriormente es procesada por un sistema de razonamiento. En la dirección opuesta, el conocimiento simbólico puede utilizarse para imponer restricciones, validar resultados o proporcionar información adicional durante el aprendizaje o la inferencia del modelo neuronal (17). Por tanto, una arquitectura neuro-simbólica no implica necesariamente que el aprendizaje neuronal y el razonamiento simbólico se realicen dentro de un único modelo.

Esta separación resulta particularmente útil cuando la entrada del sistema se encuentra expresada en lenguaje natural, pero la ejecución posterior requiere representaciones estructuradas y operaciones sujetas a reglas explícitas. En este tipo de arquitecturas, el modelo neuronal puede encargarse de interpretar la variabilidad lingüística y transformarla en una representación formal o canónica, mientras que un componente simbólico utiliza dicha representación para realizar razonamiento o planificación.

Un ejemplo de integración entre modelos de lenguaje y planificación simbólica es la arquitectura LLM+P, propuesta en *Empowering Large Language Models with Optimal Planning Proficiency* (18). En este enfoque, el modelo de lenguaje no se utiliza directamente como el mecanismo encargado de buscar la secuencia final de acciones. En su lugar, participa en la transformación de una descripción expresada en lenguaje natural hacia una representación formal que posteriormente puede ser procesada por un planificador clásico.

Para esta representación se utiliza PDDL (*Planning Domain Definition Language*), un lenguaje destinado a describir problemas de planificación automática. De manera general, PDDL permite representar los elementos que caracterizan al dominio de planificación, como las acciones disponibles, sus parámetros, precondiciones y efectos, así como los elementos específicos de un problema, entre ellos los objetos involucrados, el estado inicial y el objetivo que debe alcanzarse.

Una vez obtenida la representación formal del problema, la búsqueda de una secuencia de acciones que satisfaga el objetivo se delega a un planificador clásico. De esta manera, el modelo de lenguaje participa principalmente en la interpretación y formalización de la descripción proporcionada, mientras que el razonamiento necesario para obtener el plan se realiza mediante un mecanismo simbólico.

De manera simplificada, este flujo puede representarse como:

$$\text{Lenguaje natural} \rightarrow \text{Modelo de lenguaje} \rightarrow \text{PDDL} \rightarrow \text{Planificador clásico} \rightarrow \text{Plan}.$$

La arquitectura principal de esta tesis sigue una idea conceptualmente relacionada, aunque no utiliza PDDL ni reproduce directamente la arquitectura LLM+P. En este trabajo, el modelo Qwen2.5 (19), adaptado al dominio de tareas de servicio del robot, recibe una instrucción expresada en lenguaje natural y la transforma en una representación canónica de los objetivos solicitados. Esta representación constituye la interfaz entre el componente neuronal y el componente simbólico de la arquitectura.

El propósito de esta primera etapa es abstraer parte de la variabilidad presente en el lenguaje natural, como diferentes estructuras gramaticales, sinónimos o formas alternativas de expresar una misma intención. De este modo, distintas instrucciones que representan una intención equivalente pueden transformarse en objetivos con una estructura común antes de ser procesadas por el sistema de planificación.

En la segunda etapa, los objetivos obtenidos son enviados a CLIPS. A diferencia del modelo neuronal, CLIPS no interpreta directamente la instrucción original en lenguaje natural. Su función consiste en trabajar sobre los símbolos previamente obtenidos, incorporar los objetivos al estado de conocimiento y aplicar las reglas definidas para descomponerlos y expandirlos en las acciones necesarias para su ejecución.

2.5. Síntesis y conexión con la propuesta

Los antecedentes revisados muestran que los sistemas simbólicos permiten mantener una representación explícita del conocimiento utilizado durante la planificación, mientras que los modelos de lenguaje ofrecen mecanismos para procesar la variabilidad presente en las instrucciones expresadas en lenguaje natural. En una arquitectura híbrida, la combinación de ambos enfoques permite separar el problema de interpretación lingüística del proceso encargado de expandir los objetivos en acciones ejecutables.

Esta tesis estudia dicha separación mediante la comparación de dos arquitecturas: Qwen–CLIPS y GPT directo. La primera se evalúa con dos variantes del intérprete, Qwen2.5-3B y Qwen2.5-7B, mientras que la segunda utiliza el modelo GPT seleccionado. De este modo, se consideran tres configuraciones principales. Adicionalmente, se incorpora una condición auxiliar de control en la que se proporcionan a CLIPS los objetivos de referencia, sin intervención del intérprete neuronal. Esta condición permite examinar la cobertura y la expansión de las reglas sin introducir errores provenientes de la interpretación lingüística; no constituye una tercera arquitectura. La Tabla 2.1 resume las tres configuraciones principales y la condición auxiliar de control.

Tabla 2.1: Configuraciones principales y condición auxiliar de control.

Configuración	**Origen de los objetivos**	**Generación del plan**	**Salida considerada**
Qwen2.5-3B + CLIPS	Qwen2.5-3B ajustado mediante QLoRA	Reglas de CLIPS	Objetivos canónicos y plan
Qwen2.5-7B + CLIPS	Qwen2.5-7B ajustado mediante QLoRA	Reglas de CLIPS	Objetivos canónicos y plan
GPT directo	GPT mediante API	GPT mediante API	Objetivos y plan generados conjuntamente
Control auxiliar	Objetivos de referencia	Reglas de CLIPS	Plan a partir de los objetivos de referencia

En las dos configuraciones de Qwen–CLIPS, la instrucción se normaliza antes de consultar al modelo. Qwen produce una secuencia de objetivos canónicos en formato JSON; posteriormente, un módulo determinista valida la salida y convierte los objetivos aceptados en hechos para CLIPS. Las reglas de producción expanden estos objetivos en acciones del repertorio definido para el robot.

En la arquitectura GPT directo, el modelo genera conjuntamente una representación estructurada de los objetivos interpretados y del plan propuesto. Su salida se restringe mediante JSON Schema y posteriormente se somete a validación determinista.

Ambas arquitecturas se evalúan de forma independiente; por tanto, el trabajo no propone una selección concurrente ni una fusión automática de los planes generados.

Para la comparación experimental, cada arquitectura se ejecuta mediante un programa independiente que reproduce su cadena de generación de planes. Ambos programas reciben órdenes y contexto equivalentes y entregan objetivos interpretados y planes en un formato comparable. Los planes se analizan sin ejecutarlos físicamente en Justina. La arquitectura Qwen–CLIPS también se integró en Justina y se utilizó durante la RoboCup 2026. Los detalles de esa integración se describen en el capítulo de implementación y su desempeño se analiza por separado de la comparación experimental controlada.

La comparación examina la exactitud de los objetivos interpretados, la validez estructural y simbólica de los planes, su consistencia y las diferencias de latencia y coste entre las tres configuraciones principales. Adicionalmente, se compara el consumo de recursos computacionales de Qwen2.5-3B y Qwen2.5-7B bajo las mismas condiciones

de ejecución local. Para GPT directo mediante API no se dispone de acceso a las mediciones internas de memoria y procesamiento del modelo.

Capítulo 3

Formalización del problema y arquitectura conceptual

El problema abordado en esta tesis consiste en transformar una instrucción expresada en lenguaje natural en una secuencia de acciones que pueda ser procesada por el sistema de ejecución de un robot de servicio. Esta transformación requiere relacionar dos niveles de representación diferentes: por una parte, la intención expresada por el usuario y, por otra, las acciones concretas disponibles en el robot.

Como se revisó en el capítulo anterior, los modelos de lenguaje proporcionan mecanismos para procesar la variabilidad presente en las instrucciones lingüísticas, mientras que los sistemas simbólicos permiten representar conocimiento y relaciones mediante estructuras explícitas. A partir de estos antecedentes, en este capítulo se formaliza el problema particular considerado en la investigación y se establecen las representaciones utilizadas para describir órdenes, objetivos y planes.

El dominio experimental toma como referencia las tareas de General Purpose Service Robot (GPSR) de RoboCup@Home. Sin embargo, GPSR se utiliza como una fuente controlada de tareas, entidades y escenarios, y no como una restricción que obligue al usuario a reproducir literalmente las formulaciones lingüísticas utilizadas en la competencia.

Finalmente, se presentan las dos arquitecturas que serán evaluadas. La primera utiliza un modelo Qwen como intérprete semántico y CLIPS como componente encargado de expandir los objetivos obtenidos en acciones compatibles con las capacidades del robot. Esta arquitectura se evalúa con dos variantes del intérprete: Qwen2.5-3B y Qwen2.5-7B, cuyas denominaciones indican aproximadamente tres y siete mil millones

de parámetros, respectivamente. La segunda arquitectura delega tanto la interpretación como la generación del plan a un modelo GPT.

De este modo, se consideran dos arquitecturas y tres configuraciones experimentales: Qwen2.5-3B–CLIPS, Qwen2.5-7B–CLIPS y GPT directo. Las tres utilizan el mismo dominio conceptual, aunque las dos arquitecturas distribuyen de manera diferente la responsabilidad de transformar una orden en un plan.

3.1. Contexto arquitectónico y alcance del sistema

3.1.1. Integración dentro de la arquitectura ViRBot

La investigación se desarrolla sobre el robot de servicio Justina y su arquitectura de software ViRBot (véase la Figura 3.1). Esta arquitectura integra diferentes capacidades necesarias para la operación del robot, entre ellas percepción, interacción humano–robot, navegación, manipulación, representación de conocimiento, planificación y ejecución.

El sistema desarrollado en esta tesis se ubica entre la recepción de una instrucción expresada en lenguaje natural y los componentes encargados de ejecutar las acciones correspondientes. Por lo tanto, no pretende sustituir los módulos de navegación, percepción o manipulación del robot, sino proporcionar una representación ordenada de las acciones que dichos módulos deben realizar.

De manera simplificada, el flujo general puede expresarse como:

$$\boxed{\text{orden en lenguaje natural} \rightarrow \text{interpretación y planificación} \rightarrow \text{plan} \rightarrow \text{ejecución}} \tag{3.1}$$

La contribución estudiada se concentra en las etapas de interpretación y planificación anteriores a la ejecución física. Una instrucción puede contener varias subtareas relacionadas que deben ser identificadas y transformadas en acciones compatibles con las capacidades del robot. Los módulos de percepción, navegación, manipulación e interacción encargados de realizar dichas acciones se consideran externos al problema de planificación estudiado.

En consecuencia, la tesis evalúa la transformación de instrucciones en representaciones semánticas y planes de acción. La comparación experimental no incluye la ejecución física de los planes; el uso de Qwen–CLIPS integrado en Justina durante la RoboCup

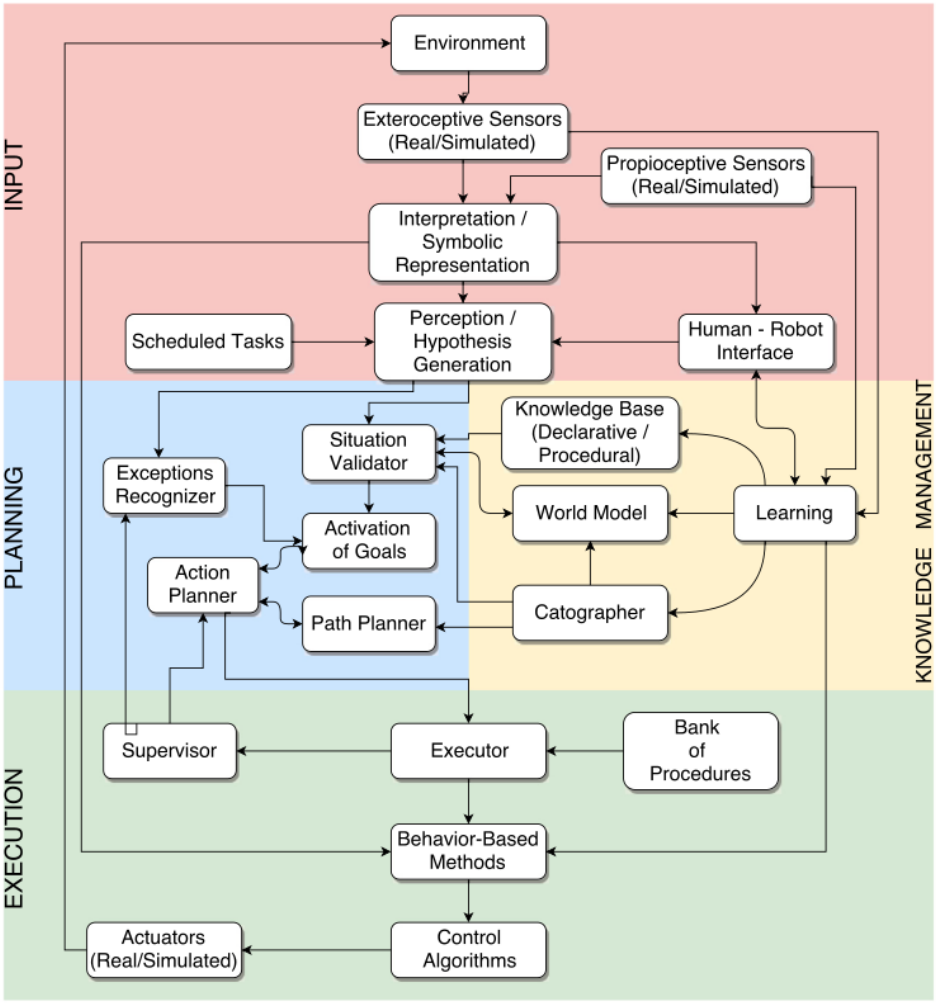


Figura 3.1: Arquitectura general ViRBot del robot Justina.

2026 se analiza por separado.

3.2. GPSR como dominio de referencia

3.2.1. La prueba General Purpose Service Robot

General Purpose Service Robot (GPSR) es una prueba de RoboCup@Home orientada a evaluar la capacidad de un robot de servicio para comprender instrucciones expresadas en lenguaje natural y ejecutar tareas que pueden requerir la combinación de distintas habilidades. Entre ellas se encuentran la navegación, la búsqueda y reconocimiento de personas u objetos, la manipulación y la interacción humano–robot(20).

Una característica importante de GPSR es que una instrucción puede representar una tarea compuesta. Por ejemplo, una orden puede requerir que el robot se desplace hacia una habitación, localice un objeto, lo tome y posteriormente lo entregue a una persona. En consecuencia, el problema no consiste únicamente en reconocer palabras o entidades aisladas, sino en identificar las diferentes intenciones contenidas en la instrucción y conservar las relaciones existentes entre ellas.

En la prueba considerada en esta investigación se proporcionan tres comandos al robot. Al comenzar, el robot se desplaza hacia un *Instruction Point*, que funciona como el punto de interacción con un operador. Desde esta posición recibe una instrucción y, una vez que ha realizado la tarea solicitada, regresa al mismo punto para recibir el siguiente comando. La determinación de que una tarea ha sido completada de manera suficiente para continuar con la prueba corresponde a un árbitro.

La entrega de las órdenes se realiza de manera secuencial. Por tanto, el robot no recibe un nuevo comando mientras la ejecución de la tarea anterior continúe en curso. Una vez concluida la tarea y después de regresar al *Instruction Point*, puede solicitar la siguiente instrucción hasta completar los tres comandos contemplados por la prueba.

Cuando el robot no logra comprender adecuadamente la instrucción recibida mediante su sistema de entrada de audio, puede solicitar explícitamente que la orden sea repetida. En este caso, el operador expresará la misma tarea mediante una construcción lingüística más sencilla, pero manteniendo los objetivos contenidos en la instrucción original. La reformulación no constituye una modificación de la tarea ni permite que el operador indique directamente al robot cómo resolverla.

Este mecanismo es particularmente relevante para la presente investigación, pues permite distinguir entre la tarea que debe realizarse y la forma lingüística utilizada para comunicarla. Dos instrucciones con estructuras sintácticas diferentes pueden representar los mismos objetivos y, por lo tanto, deberían conducir a una interpretación semántica equivalente.

Durante la ejecución no se contempla que una persona intervenga para descomponer la orden, seleccionar las acciones del robot o corregir manualmente el plan generado. La interpretación de la instrucción y la determinación de las acciones necesarias corresponden al propio sistema robótico. De esta manera, GPSR proporciona un escenario apropiado para evaluar la transformación de instrucciones en lenguaje natural en representaciones semánticas y planes de acción.

3.2.2. Generación de comandos y conocimiento del dominio

Una característica relevante de GPSR es que los comandos utilizados durante la prueba no se seleccionan únicamente a partir de una lista fija de oraciones. El reglamento establece que las tareas se construyen mediante el *CommandGenerator* oficial de RoboCup@Home (20, 21). Sobre esta herramienta se realizaron posteriormente adaptaciones específicas para construir el conjunto de datos empleado para el entrenamiento de ambas variantes de Qwen y la evaluación de las configuraciones consideradas. Dichas modificaciones se describen en el capítulo de metodología.

El generador define diferentes familias de instrucciones relacionadas con personas y objetos. Entre ellas se encuentran tareas para desplazarse hacia una ubicación, encontrar personas u objetos, contar entidades, obtener información, seguir o guiar personas, tomar objetos y entregarlos o colocarlos en una ubicación determinada. Algunas de estas estructuras permiten incorporar continuaciones que producen comandos compuestos. De esta manera, una instrucción inicial puede extenderse con subtareas posteriores que dependen del resultado de las anteriores (21).

De forma simplificada, el proceso puede representarse como:

$$\boxed{\text{familia de tarea} + \text{estructura lingüística} + \text{entidades del dominio} \rightarrow \text{comando GPSR}} \tag{3.2}$$

Las estructuras del generador utilizan elementos variables para representar entidades y relaciones del dominio. Por ejemplo, una plantilla puede contener una habitación, una persona, un objeto o una ubicación que posteriormente son sustituidos por valores concretos. De forma similar, diferentes verbos pueden representar una intención relacionada; por ejemplo, una tarea de navegación puede expresarse mediante verbos como *go* o *navigate*, mientras que una búsqueda puede utilizar expresiones como *find*, *locate* o *look for* (21).

El conocimiento utilizado para realizar estas sustituciones se mantiene separado de las estructuras de los comandos. El generador distingue, entre otros elementos, nombres de personas, habitaciones, ubicaciones, superficies de colocación, objetos y categorías de objetos (21). La herramienta permite recibir este conocimiento desde un directorio externo y señala el repositorio *CompetitionTemplate* como referencia para su organización (21, 22).

En *CompetitionTemplate*, la información específica de una competencia se organiza

en archivos relacionados con el mapa y las ubicaciones, los nombres de personas y los objetos disponibles (22). Esta separación resulta importante porque permite mantener las estructuras generales de las tareas mientras se modifican las entidades concretas presentes en una edición o escenario determinado.

Conceptualmente, el dominio de referencia combina las familias de tareas contempladas por el generador con el conocimiento específico utilizado para instanciarlas. Este último se representa mediante K y se formaliza en la Sección 3.3.3.

3.2.3. Dominio de referencia y alcance lingüístico

La utilización de GPSR como referencia no implica que el sistema desarrollado en esta tesis deba reconocer únicamente las frases producidas literalmente por el generador oficial. Esta distinción resulta especialmente importante debido a que una misma intención puede expresarse mediante diferentes construcciones lingüísticas.

De manera adicional, el reglamento de GPSR 2026 incorpora un mecanismo de variación sobre las órdenes generadas. Una vez producido un comando mediante el generador oficial, el reglamento establece que éste se proporciona a un modelo de lenguaje para obtener una formulación similar. Además, una instrucción puede ser reformulada hasta tres veces, utilizando expresiones progresivamente más simples cuando sea necesario (20).

Por ejemplo, el reglamento presenta una transformación equivalente para pasar de una instrucción como:

"Get me a coke from the kitchen."

a una formulación más explícita, como:

"Go to the kitchen, find a coke, and bring it to me."

Aunque la estructura superficial de ambas expresiones es diferente, la tarea subyacente permanece esencialmente sin cambios. Esto permite distinguir dos niveles: forma lingüística e intención semántica.

Esta distinción es central para la presente investigación. GPSR proporciona un conjunto controlado de tareas y entidades, pero la entrada lingüística puede presentar diferentes formas de expresar una misma intención. El objetivo del intérprete no consiste, por tanto, en memorizar las plantillas del generador, sino en establecer una correspondencia entre distintas realizaciones lingüísticas y una representación semántica común.

Ambas formulaciones deben corresponder a la misma secuencia de objetivos. Esta propiedad se formaliza posteriormente mediante la noción de equivalencia semántica presentada en la Sección 3.3.2.

Por tanto, en esta tesis GPSR se utiliza como una fuente controlada para definir las tareas, entidades y relaciones que forman parte del dominio experimental, pero no como una gramática que limite las únicas formulaciones aceptables por el sistema.

Puede resumirse esta relación mediante:

$$\boxed{\begin{array}{c}\text{múltiples formulaciones lingüísticas}\\ \downarrow \\ \text{representación semántica canónica}\\ \downarrow \\ \text{capacidades implementadas del robot}\end{array}} \tag{3.3}$$

En consecuencia, la entrada lingüística puede variar en su formulación, pero la interpretación permanece acotada por el dominio semántico y las capacidades operativas disponibles. Una instrucción puede ser perfectamente comprensible para una persona y, aun así, quedar fuera del alcance del sistema si requiere una intención que no puede representarse mediante los objetivos contemplados o una capacidad que el robot no posee.

El dominio experimental de esta investigación considera principalmente instrucciones en inglés, en correspondencia con las estructuras utilizadas por el generador y con el escenario de evaluación. Por ello, los resultados relacionados con variabilidad lingüística deben interpretarse como capacidad para procesar diferentes formulaciones de las tareas soportadas dentro de dicho idioma, y no como evidencia de independencia respecto al idioma o de comprensión irrestricta del lenguaje natural.

3.3. Formalización de órdenes, objetivos y planes

La transformación de una instrucción en lenguaje natural en un plan para un robot de servicio implica relacionar representaciones con diferentes niveles de abstracción. Por una parte, el usuario expresa una tarea mediante lenguaje natural; por otra, el sistema de ejecución requiere una secuencia de acciones compatibles con las capacidades disponibles del robot.

En esta investigación se introduce una representación semántica intermedia que

permite separar ambos niveles. De manera general, el problema se representa mediante:

$$\boxed{x \rightarrow G \rightarrow P} \tag{3.4}$$

donde x representa la instrucción en lenguaje natural, G una secuencia ordenada de objetivos semánticos y P el plan compuesto por acciones ejecutables.

La primera transformación, $x \rightarrow G$, corresponde a la interpretación de la orden y busca determinar qué solicita el usuario. La segunda, $G \rightarrow P$, corresponde a la planificación y determina qué acciones son necesarias para atender los objetivos identificados.

Esta separación constituye la base conceptual utilizada posteriormente para comparar las arquitecturas Qwen–CLIPS y GPT directo.

3.3.1. Orden en lenguaje natural

Se denota mediante x una instrucción expresada en lenguaje natural dentro del dominio considerado en esta investigación.

Una instrucción puede representar una tarea simple o una composición de varias subtareas relacionadas. Por ejemplo:

"Go to the kitchen, find an apple and bring it to me."

incluye navegación, búsqueda, manipulación y entrega de un objeto. Por tanto, una instrucción lingüística no corresponde necesariamente con una única acción del robot.

Incluso una expresión breve puede implicar varias operaciones. Considérese:

"Bring me an apple from the kitchen."

Dentro del dominio utilizado en esta investigación, esta orden puede representarse mediante la siguiente secuencia semántica:

```
go(kitchen)
find(apple, kind=object)
take(apple)
deliver(apple, to=me)
```

La interpretación no consiste únicamente en identificar palabras clave. También debe reconocer las subtareas explícitas o implícitas, las entidades involucradas y las relaciones necesarias entre ellas.

El resultado de este proceso es una representación semántica intermedia, denotada mediante G, cuya estructura se describe en la Sección 3.3.4.

3.3.2. Equivalencia semántica entre formulaciones

Como se describió en la Sección 3.2, una misma tarea puede expresarse mediante diferentes formulaciones lingüísticas. Esta propiedad resulta especialmente relevante en GPSR, donde una orden puede ser repetida o reformulada sin modificar los objetivos que debe cumplir el robot.

En esta investigación se consideran semánticamente equivalentes dos instrucciones cuando, dentro del dominio utilizado, representan la misma secuencia de objetivos.

Esta relación se denota mediante $x_1 \sim x_2$. Idealmente:

$$\boxed{x_1 \sim x_2 \quad \Rightarrow \quad G_1 = G_2} \tag{3.5}$$

donde G_1 y G_2 son las representaciones semánticas obtenidas para cada instrucción.

Por ejemplo:

"Fetch a bottle from the living room and throw it in the trash."

y:

"Go to the living room, grab a bottle, and toss it in the trash."

presentan estructuras lingüísticas diferentes, pero pueden corresponder a la misma representación:

```
go(living room)
find(bottle, kind=object)
take(bottle)
place(bottle, in=trash)
```

Esta propiedad se utiliza en la presente investigación para representar de forma común órdenes lingüísticamente diferentes que conservan la misma intención o representación semántica canónica.

3.3.3. Entidades y conocimiento del dominio

La interpretación de una instrucción requiere información acerca de las entidades que pueden participar en las tareas consideradas.

En esta investigación, el conocimiento relevante del dominio se representa mediante:

$$K = \{K_P, K_O, K_C, K_L, K_R\}, \tag{3.6}$$

donde:

- K_P representa las personas o nombres conocidos;
- K_O representa los objetos;
- K_C representa las categorías de objetos;
- K_L representa las ubicaciones específicas;
- K_R representa las habitaciones.

Esta información permite determinar el tipo de las entidades utilizadas en una instrucción. Por ejemplo, si `Robin` pertenece al conjunto de personas y `apple` al conjunto de objetos, pueden utilizarse representaciones como:

```
find(Robin, kind=person)
find(apple, kind=object)
```

Ambos ejemplos utilizan el mismo tipo general de objetivo, pero operan sobre entidades semánticamente diferentes.

El conocimiento K cumple dos funciones principales. En primer lugar, permite verificar que las entidades nominales utilizadas pertenezcan al dominio. Una salida que introduzca arbitrariamente una persona, objeto o ubicación no debe considerarse correcta únicamente porque su estructura sea válida.

En segundo lugar, permite verificar la compatibilidad entre las entidades y los objetivos en los que participan.

Las referencias presentes en una instrucción pueden adoptar diferentes formas. En esta investigación se distinguen tres casos principales:

1. **Entidades nominales:** referencias explícitas a una entidad conocida, como `Robin`, `apple` o `kitchen`.
2. **Entidades genéricas:** referencias a una clase de entidad, como `person`, `drink` o `object`.
3. **Entidades descritas mediante propiedades:** referencias que requieren identificar una entidad a partir de alguna característica, como una persona saludando, una persona sentada o el objeto de mayor tamaño.

Por ejemplo:

```
find(person, kind=person, gesture='waving person')
```

no identifica inicialmente a una persona mediante un nombre, sino mediante una característica observable.

De manera similar:

```
find(drink, kind=object, property=largest)
```

representa la búsqueda de un objeto perteneciente a una categoría y sujeto a una propiedad adicional.

Estas distinciones permiten conservar la información necesaria para que las etapas posteriores puedan determinar cómo realizar la tarea.

3.3.4. Representación canónica de objetivos

El resultado de la interpretación se expresa mediante una secuencia ordenada de objetivos:

$$G = (g_1, g_2, \ldots, g_n).$$

Cada objetivo g_i contiene un tipo de intención y los argumentos necesarios para describirla. Algunos ejemplos de objetivos individuales son:

```
go(kitchen)
find(Robin, kind=person)
deliver(apple, to=Robin)
```

Se denota mediante $\mathcal{G}$ el vocabulario de tipos de objetivos semánticos utilizados en esta investigación. La Tabla 3.1 resume estos objetivos y sus argumentos principales.

Los objetivos `find` y `count` requieren distinguir explícitamente si la entidad considerada corresponde a una persona o a un objeto. Para ello se utiliza el argumento `kind`. Por ejemplo:

```
find(Robin, kind=person)
find(apple, kind=object)
count(person, kind=person)
count(drinks, kind=object)
```

Tabla 3.1: Vocabulario de objetivos semánticos utilizado en la representación canónica.

Objetivo	**Propósito**	**Argumentos principales**
`go`	Desplazarse hacia una ubicación	ubicación
`find`	Localizar una persona u objeto	objetivo, `kind`, gesto, pose, vestimenta o propiedad
`take`	Tomar un objeto	objeto
`deliver`	Entregar un objeto	objeto, destinatario
`place`	Colocar un objeto en un destino	objeto, destino y relación espacial
`guide`	Guiar a una persona	persona, destino
`follow`	Seguir a una persona	persona, destino opcional
`count`	Contar personas u objetos	objetivo, `kind` y atributos opcionales
`tell`	Comunicar información	información o propiedad
`save`	Conservar información obtenida durante una interacción	nombre, pose o gesto
`answer_question`	Atender una pregunta	—
`greet`	Saludar a una persona	persona

Otros objetivos tienen un tipo de entidad implícito. Por ejemplo, `take`, `deliver` y `place` operan sobre objetos, mientras que `guide`, `follow` y `greet` operan sobre personas.

Los objetivos pueden incluir además calificadores relevantes presentes en la instrucción. Entre ellos se encuentran gestos, poses, vestimenta, características de objetos y relaciones espaciales. Por ejemplo:

```
find(person, kind=person, gesture='waving person')
find(person, kind=person, pose='sitting person')
find(person, kind=person, wearing='red shirt')
find(drink, kind=object, property=largest)
guide(Robin, to=kitchen)
place(apple, on=table)
```

La representación canónica busca conservar la información necesaria para describir la intención de la orden, eliminando al mismo tiempo las diferencias lingüísticas que no modifican su significado dentro del dominio.

La forma concreta utilizada para serializar y transmitir esta secuencia entre componentes pertenece a la implementación y se describe posteriormente.

Fidelidad de la representación semántica

Una secuencia formada únicamente por objetivos admitidos no constituye necesariamente una interpretación correcta.

Considérese nuevamente:

"Go to the kitchen and find Robin."

La representación esperada es:

```
go(kitchen)
find(Robin, kind=person)
```

Una salida como:

```
find(Robin, kind=person)
```

utiliza un objetivo válido, pero omite la navegación solicitada.

Por otra parte:

```
go(kitchen)
find(Robin, kind=person)
greet(Robin)
```

incorpora una intención que no forma parte de la orden original.

En consecuencia, la fidelidad de la representación depende de que ésta conserve las subtareas, entidades, relaciones y dependencias expresadas o implicadas por la orden, sin introducir intenciones ajenas a ella.

Estos requisitos se formalizan posteriormente como criterios de validez en la Sección 3.5.2.

3.3.5. Objetivos semánticos y acciones ejecutables

Los objetivos canónicos describen **qué** debe conseguir el robot, pero no necesariamente especifican todas las operaciones concretas requeridas para hacerlo.

Se denota mediante $\mathcal{A}$ el repertorio de acciones disponibles para el sistema de ejecución. Entre ellas se encuentran acciones de navegación, interacción, búsqueda, percepción y manipulación, como `go_to`, `say`, `listen`, `find_person`, `find_object`, `approach`, `take`, `find_space`, `drop` y `give`.

El vocabulario de objetivos y el repertorio de acciones representan niveles diferentes:

$$\boxed{\mathcal{G} \neq \mathcal{A}} \tag{3.7}$$

Por ejemplo, el objetivo:

```
deliver(apple, to=Robin)
```

representa una intención de alto nivel. Para realizarla pueden ser necesarias acciones concretas destinadas a localizar al destinatario, aproximarse a él y entregar el objeto.

Conceptualmente:

$$\texttt{deliver} \rightarrow \texttt{find_person} \rightarrow \texttt{approach} \rightarrow \texttt{give}.$$

De manera similar:

```
place(apple, on=table)
```

puede requerir identificar primero un espacio adecuado y posteriormente liberar el objeto mediante las acciones correspondientes.

El plan resultante se representa mediante una secuencia:

$$P = (a_1, a_2, \ldots, a_m),$$

donde cada a_i corresponde a una acción perteneciente al repertorio $\mathcal{A}$.

Un objetivo puede producir una o varias acciones. Por ello, la planificación puede entenderse simplemente como la transformación de la secuencia de objetivos G en un plan P compatible con las capacidades disponibles:

$$G \rightarrow P.$$

Esta distinción permite evaluar por separado si una orden fue interpretada correctamente y si los objetivos obtenidos fueron transformados posteriormente en un plan adecuado.

3.3.6. Dependencias y contexto entre objetivos

La secuencia G no puede tratarse como una colección de objetivos independientes. Algunas subtareas dependen de otras, reutilizan entidades identificadas previamente o establecen condiciones necesarias para objetivos posteriores.

Por esta razón, G se considera una **secuencia ordenada**.

Dependencias de precedencia

Algunos objetivos requieren que una subtarea anterior haya sido realizada.

Por ejemplo:

```
take(apple)
deliver(apple, to=Robin)
```

requiere adquirir el objeto antes de intentar entregarlo. Esta relación se representa mediante:

$$\texttt{take}(o) \prec \texttt{deliver}(o, p), \tag{3.8}$$

donde $\prec$ indica que el objetivo situado a la izquierda debe preceder al situado a la derecha.

Una secuencia que invirtiera esta relación podría contener objetivos individualmente válidos, pero no representaría una tarea coherente.

Dependencias referenciales

Algunas instrucciones introducen entidades que posteriormente deben ser reutilizadas.

Considérese:

```
find(person, kind=person, gesture='waving person')
guide(person, to=kitchen)
```

La persona utilizada en el segundo objetivo no representa una persona arbitraria. Se refiere a la entidad identificada por la búsqueda anterior.

Este tipo de relación permite representar instrucciones que contienen pronombres, referencias genéricas o descripciones cuyo referente se obtiene durante el desarrollo de la tarea.

Dependencias espaciales y de estado

Algunos objetivos establecen un contexto que debe ser considerado por las subtareas posteriores.

Por ejemplo:

```
go(kitchen)
find(apple, kind=object)
```

indica que la búsqueda del objeto debe realizarse después de desplazarse hacia la cocina. El objetivo de navegación establece así el contexto espacial esperado para la búsqueda posterior.

De manera similar:

```
take(apple)
place(apple, on=table)
```

establece una condición intermedia: después de tomar la manzana, el plan puede considerar que el robot dispone del objeto para realizar su colocación.

Estas condiciones representan el **estado esperado durante la planificación**. No constituyen una confirmación de que las acciones hayan sido ejecutadas correctamente en el mundo físico. Esta diferencia se retoma posteriormente al distinguir entre planificación y ejecución.

Orden lingüístico y orden semántico

Finalmente, debe distinguirse el orden en el que las palabras aparecen dentro de una oración del orden lógico requerido para realizar la tarea.

Por ejemplo:

> *"Bring me the apple from the kitchen."*

expresa directamente una intención de entrega, pero para realizarla puede ser necesario construir la siguiente secuencia:

```
go(kitchen)
find(apple, kind=object)
take(apple)
deliver(apple, to=me)
```

El orden lógico puede resumirse como:

$$\texttt{go} \prec \texttt{find} \prec \texttt{take} \prec \texttt{deliver}.$$

La interpretación no consiste, por tanto, en reproducir el orden superficial de las palabras, sino en recuperar las relaciones necesarias entre las subtareas.

Esta propiedad resulta especialmente relevante en instrucciones compactas, en las que algunas operaciones requeridas para cumplir la intención pueden no aparecer expresadas como verbos independientes.

En conjunto, la representación G debe conservar tanto la intención de la orden como las entidades, relaciones y dependencias necesarias para transformarla posteriormente en un plan coherente.

3.4. Arquitecturas conceptuales evaluadas

A partir de la formalización presentada en la Sección 3.3, las dos arquitecturas consideradas en esta investigación resuelven el mismo problema, pero distribuyen de manera diferente las responsabilidades de interpretación y planificación.

En la arquitectura Qwen–CLIPS, ambas funciones se encuentran separadas: Qwen realiza la interpretación semántica y CLIPS transforma posteriormente los objetivos obtenidos en acciones ejecutables.

En la arquitectura GPT directo, un mismo modelo realiza tanto la interpretación de la orden como la generación del plan.

Esta diferencia constituye el eje principal de la comparación entre arquitecturas. Se busca contrastar dos formas de organizar el proceso que relaciona lenguaje natural, objetivos semánticos y acciones ejecutables. Adicionalmente, dentro de la arquitectura Qwen–CLIPS, se comparan las variantes Qwen2.5-3B y Qwen2.5-7B para examinar las diferencias asociadas con la elección del intérprete, manteniendo común el procesamiento simbólico posterior.

3.4.1. Base común de las arquitecturas evaluadas

Para que la comparación sea significativa, ambas arquitecturas operan sobre una misma definición del problema y comparten los elementos conceptuales establecidos en la sección anterior.

En particular, ambas utilizan:

- el mismo conocimiento del dominio K;
- el mismo vocabulario de objetivos semánticos $\mathcal{G}$;
- el mismo repertorio de acciones ejecutables $\mathcal{A}$;
- las mismas familias de tareas consideradas para la evaluación;
- una representación común de referencia para determinar qué objetivos corresponden a cada instrucción.

En consecuencia, ambas arquitecturas deben interpretar órdenes pertenecientes al mismo dominio y producir planes compatibles con las mismas capacidades del robot.

Sin embargo, utilizar el mismo dominio no significa que el conocimiento necesario para resolver la tarea se encuentre distribuido de la misma manera.

En Qwen–CLIPS, Qwen debe determinar únicamente qué solicita el usuario. El conocimiento necesario para convertir posteriormente esos objetivos en acciones se encuentra representado mediante reglas dentro de CLIPS.

En cambio, GPT directo debe disponer durante una misma inferencia tanto del conocimiento necesario para interpretar la orden como de las restricciones necesarias para generar el plan.

Por tanto, la comparación se realiza sobre un mismo problema y un mismo resultado esperado, aunque cada arquitectura distribuya de manera diferente el conocimiento utilizado para alcanzarlo.

En la comparación experimental, las arquitecturas se implementan mediante dos programas independientes del sistema de ejecución física del robot: uno correspondiente a Qwen–CLIPS y otro a GPT directo. El primero se utiliza con Qwen2.5-3B y Qwen2.5-7B, dando lugar a dos configuraciones de una misma arquitectura.

Las tres configuraciones reciben las mismas instrucciones y disponen de información equivalente sobre el dominio y las condiciones iniciales pertinentes. Los programas emplean convenciones comunes para registrar los objetivos interpretados, los planes generados y los fallos del procesamiento, de modo que sus resultados puedan evaluarse mediante los mismos criterios. Esta comparación no incluye la ejecución física de los planes.

3.4.2. Arquitectura Qwen–CLIPS

La primera arquitectura adopta una separación explícita entre interpretación semántica y planificación.

Esta arquitectura se instancia con Qwen2.5-3B y Qwen2.5-7B. Ambos modelos fueron ajustados mediante QLoRA utilizando el mismo conjunto de datos, la misma partición, los mismos hiperparámetros y el mismo criterio de selección del checkpoint. La diferencia introducida en el procedimiento de entrenamiento fue el identificador del modelo base.

En la comparación experimental se mantienen comunes los procedimientos de normalización y adaptación de los objetivos, las reglas de CLIPS y el estado inicial utilizado para generar cada plan. Por tanto, las dos configuraciones conservan la misma arquitectura y difieren en el modelo empleado para la interpretación semántica. Los detalles del entrenamiento y de la implementación se presentan en los capítulos correspondientes.

Su funcionamiento general puede resumirse mediante:

$$\boxed{x \xrightarrow{\text{Qwen}} G \xrightarrow{\text{CLIPS}} P} \tag{3.9}$$

Qwen actúa como intérprete semántico. Su responsabilidad consiste en recibir la instrucción en lenguaje natural y producir una secuencia ordenada de objetivos pertenecientes al vocabulario definido en $\mathcal{G}$. No determina directamente las acciones ejecutables; su responsabilidad termina en la construcción de la representación semántica definida en la Sección 3.3.4.

Antes de ser procesados por CLIPS, los objetivos obtenidos son sometidos a comprobaciones deterministas destinadas a verificar que utilizan la estructura, los tipos de objetivos y las entidades admitidas por el dominio. Los detalles de estas comprobaciones y de la representación utilizada para intercambiar la información entre componentes se presentan posteriormente en el capítulo de implementación.

La Figura 3.2 resume el flujo conceptual de esta arquitectura.

Una vez aceptada la representación semántica, CLIPS se encarga de transformar los objetivos en acciones pertenecientes al repertorio $\mathcal{A}$.

En esta arquitectura, CLIPS funciona como un expansor de objetivos basado en reglas de producción. Su función no consiste en interpretar nuevamente la orden original ni en realizar una búsqueda libre entre planes posibles. En cambio, cada objetivo activa conocimiento explícito que determina las acciones necesarias para atenderlo, siguiendo la distinción entre objetivos semánticos y acciones ejecutables presentada en la Sección 3.3.5.

La expansión conserva el orden y las relaciones presentes en G, de manera que las dependencias descritas en la Sección 3.3.6 puedan ser consideradas durante la generación

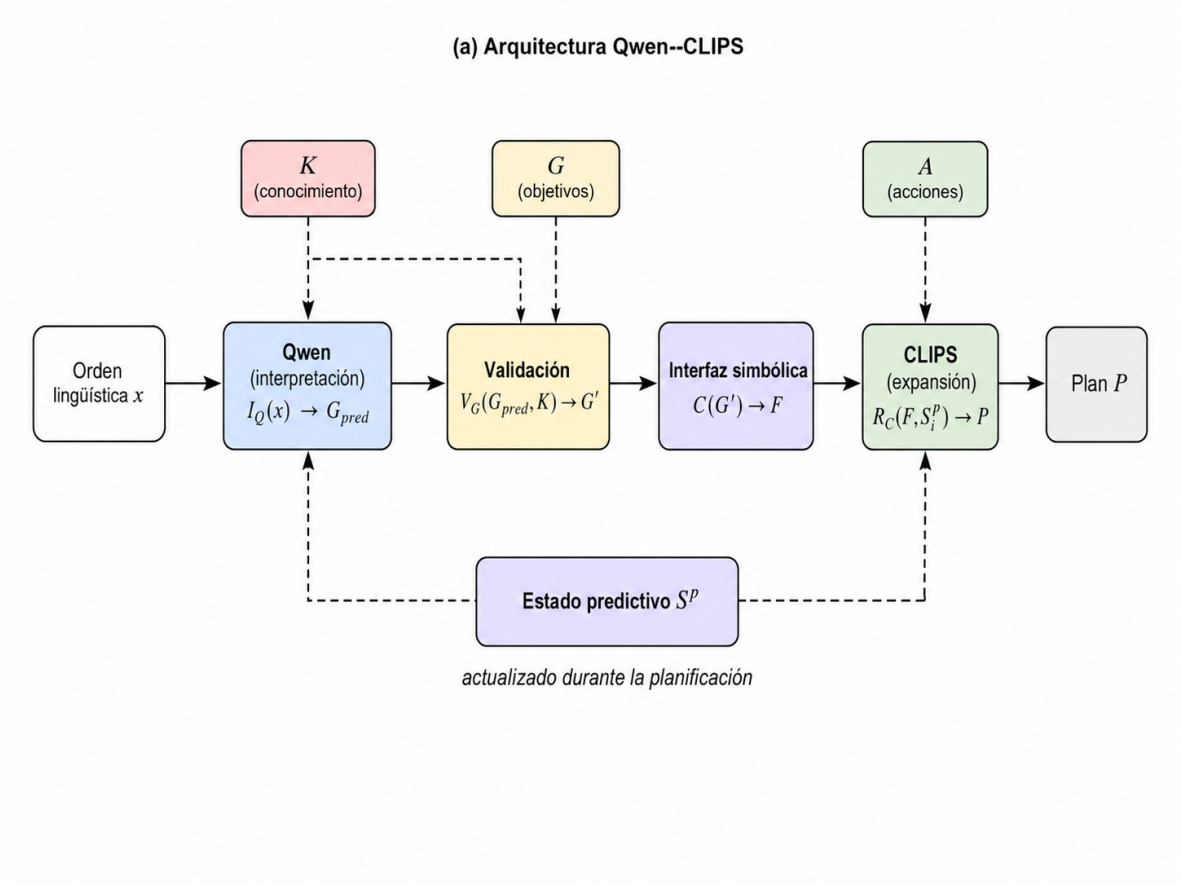


Figura 3.2: Arquitectura conceptual Qwen–CLIPS.

del plan.

Estado esperado durante la planificación

Durante la expansión de los objetivos, CLIPS utiliza el estado esperado descrito en la Sección 3.3.6 para conservar la coherencia entre subtareas consecutivas. De esta manera, las decisiones asociadas con un objetivo pueden considerar los resultados esperados de objetivos procesados previamente.

En esta arquitectura, dicha información forma parte del proceso simbólico de expansión. Su función se limita a mantener consistencia durante la construcción del plan.

Separación de errores

La separación entre Qwen y CLIPS permite identificar dos fuentes principales de error. La primera corresponde a la interpretación semántica, cuando los objetivos producidos no representan correctamente la orden. La segunda corresponde a la planificación simbólica, cuando una representación correcta no puede transformarse adecuadamente en acciones ejecutables.

Además, una interpretación incorrecta puede producir posteriormente un plan internamente coherente con objetivos equivocados. Por ello, la evaluación de esta arquitectura debe considerar de forma independiente la corrección de G y la corrección de P.

Esta separación se utiliza posteriormente para definir la condición experimental de control presentada en la Sección 3.5.5.

3.4.3. Arquitectura de planificación directa con GPT

La segunda arquitectura concentra en un mismo modelo las funciones de interpretación semántica y generación del plan.

Su comportamiento principal puede resumirse mediante:

$$\boxed{x \xrightarrow{\text{GPT}} (G, P)} \tag{3.10}$$

Para una instrucción ejecutable, GPT produce en una misma respuesta tanto la secuencia de objetivos semánticos G como el plan de acciones P.

La conservación explícita de ambos elementos resulta importante. Aunque hayan sido generados por el mismo modelo, G representa la interpretación realizada sobre la

orden, mientras que P representa la secuencia de acciones propuesta para atenderla.

De esta manera, ambos resultados pueden ser analizados posteriormente de forma independiente.

La Figura 3.3 presenta el flujo conceptual de esta arquitectura.

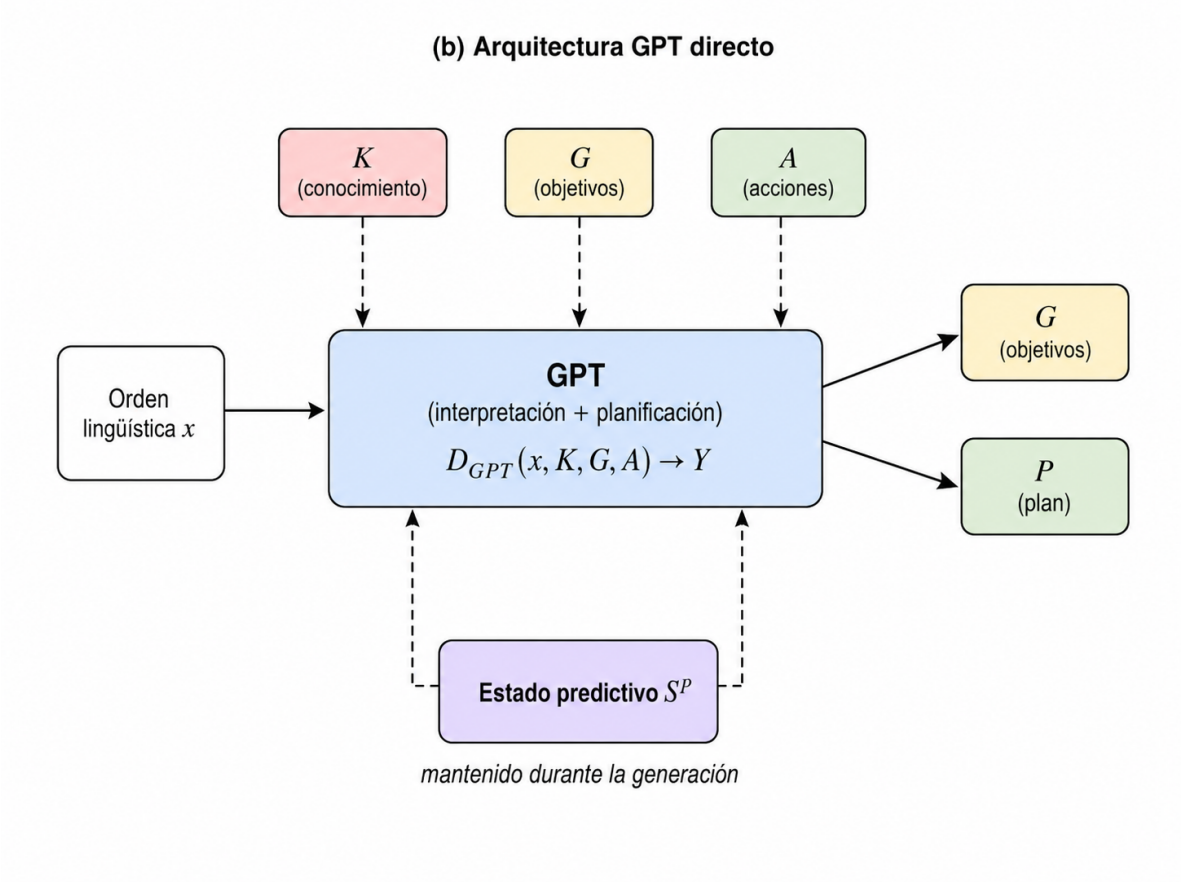


Figura 3.3: Arquitectura conceptual del planificador GPT directo.

Una diferencia fundamental respecto a Qwen–CLIPS es que los objetivos producidos por GPT no se utilizan como entrada para CLIPS. Tanto G como P son generados durante una misma inferencia y se conservan como resultados diferenciados para su posterior evaluación.

Tipos de respuesta

La arquitectura no obliga al modelo a producir un plan para cualquier instrucción recibida. Dependiendo de la información disponible y de la compatibilidad de la orden con el dominio, se contemplan tres tipos de respuesta:

- `executable:` la instrucción contiene información suficiente y puede representarse mediante objetivos y acciones soportadas. En este caso se generan G y P.
- `clarification:` existe información faltante o una ambigüedad que puede resolverse mediante una pregunta breve al usuario. En este caso no se genera un plan hasta resolver dicha ambigüedad.
- `rejected:` la instrucción se encuentra fuera del dominio, requiere una capacidad no disponible o contradice las restricciones establecidas. En este caso tampoco se genera un plan.

Esta clasificación permite evitar que el modelo genere arbitrariamente una secuencia de acciones cuando no dispone de información suficiente para interpretar la orden.

Planificación directa bajo restricciones

El término *planificación directa* indica que GPT produce las acciones del plan sin delegar posteriormente la expansión de los objetivos a CLIPS.

Esto no significa que el modelo pueda generar cualquier comportamiento.

La generación se encuentra restringida mediante información sobre:

- las entidades admitidas en el dominio;
- los tipos de objetivos pertenecientes a $\mathcal{G}$;
- las acciones disponibles en $\mathcal{A}$;
- las relaciones y dependencias necesarias entre subtareas;
- las restricciones que determinan qué combinaciones pueden constituir un plan válido.

Así, una intención semántica como `deliver` no debe utilizarse directamente como una acción ejecutable si no pertenece al repertorio $\mathcal{A}$. El modelo debe generar las acciones concretas necesarias para realizar dicha intención.

El conocimiento que en Qwen–CLIPS se encuentra expresado mediante reglas de producción debe encontrarse disponible para GPT mediante las instrucciones y restricciones proporcionadas durante la generación.

Esta diferencia no modifica el dominio de la comparación, sino la forma en que cada arquitectura representa y utiliza el conocimiento necesario para construir el plan.

Estado esperado durante la generación

GPT también debe mantener coherencia con el estado esperado descrito en la Sección 3.3.6 al generar las acciones del plan.

La diferencia respecto a Qwen–CLIPS se encuentra en el mecanismo utilizado. En la arquitectura simbólica, las relaciones necesarias para mantener esta coherencia se representan mediante conocimiento y reglas explícitas. En GPT directo, el modelo debe conservarlas durante la generación condicionada por el contexto y las restricciones proporcionadas.

Separación posterior de interpretación y planificación

Aunque GPT realice interpretación y planificación durante una misma inferencia, la conservación de G y P permite analizar posteriormente ambas funciones por separado.

Si los objetivos producidos no corresponden con los objetivos de referencia, el error puede atribuirse principalmente a la interpretación de la orden.

En cambio, si G representa correctamente la instrucción pero P omite acciones necesarias, utiliza acciones no soportadas o viola dependencias de la tarea, puede identificarse un error de planificación.

A diferencia de Qwen–CLIPS, esta separación no corresponde a dos componentes independientes del sistema. Se obtiene mediante el análisis de los dos artefactos generados por el mismo modelo.

3.4.4. Comparación conceptual de las arquitecturas

La diferencia esencial entre las arquitecturas puede resumirse mediante:

$$\boxed{\begin{array}{ll} \text{Qwen–CLIPS:} & x \to G \to P \\ \text{GPT directo:} & x \to (G, P) \end{array}} \tag{3.11}$$

En Qwen–CLIPS, la interpretación y la planificación corresponden a etapas explícitamente separadas: Qwen produce la representación semántica y CLIPS transforma posteriormente dicha representación en acciones. En GPT directo, ambas funciones se realizan durante una misma inferencia, aunque G y P se conservan como resultados diferenciados para permitir su análisis posterior.

La Tabla 3.2 resume las principales características de ambas estrategias.

El carácter basado en reglas de CLIPS no implica que cualquier plan producido sea necesariamente correcto ni garantiza el éxito de su ejecución física. De manera similar, restringir la estructura de la respuesta de GPT no garantiza que la interpretación o el plan generado sean semánticamente correctos.

Tabla 3.2: Comparación conceptual entre las arquitecturas evaluadas.

Característica	**Qwen2.5-3B Qwen2.5-7B**	**GPT directo**
Interpretación semántica	Qwen	GPT
Generación del plan	CLIPS	GPT
Representación explícita de G	Sí	Sí
Generación de P	Reglas de producción	Generación condicionada del modelo
Conocimiento para expandir los objetivos	Representado explícitamente mediante reglas	Proporcionado mediante contexto y restricciones
Separación entre interpretación y planificación	Explícita entre componentes	Integrada en una misma inferencia
Estado durante la planificación	Predictivo y simbólico	Predictivo durante la generación
Dependencia de un servicio externo de terceros	No	Sí
Identificación de errores	Separación directa entre etapas	Separación posterior mediante G y P
Variabilidad del resultado	Depende de los objetivos, reglas y condiciones iniciales	Puede presentar variación entre inferencias

Por ello, en ambas arquitecturas resulta necesario evaluar por separado la representación semántica G y el plan P. Esta separación permite examinar la corrección de los resultados, los tipos de error y la consistencia de cada configuración.

La comparación también considera la latencia observada desde los programas experimentales. Adicionalmente, entre Qwen2.5-3B y Qwen2.5-7B se compara el consumo de recursos computacionales bajo condiciones comunes de ejecución local. Para GPT directo mediante API, la evaluación no incluye mediciones internas de memoria o procesamiento del modelo. Las métricas y los procedimientos de medición se definen en el capítulo de validación y experimentación.

3.5. Criterios conceptuales de validez

La generación de una salida correctamente estructurada no implica, por sí misma, que la interpretación de una orden o el plan correspondiente sean correctos.

Por esta razón, en esta investigación se distinguen diferentes niveles de validez. El propósito de esta separación es identificar si un error se origina durante la interpretación de la instrucción, durante la generación del plan o posteriormente durante su ejecución física.

De manera general, el proceso puede representarse como:

$$\boxed{x \xrightarrow{\text{interpretación}} G \xrightarrow{\text{planificación}} P \xrightarrow{\text{ejecución}} E} \tag{3.12}$$

donde x representa la instrucción en lenguaje natural, G la secuencia de objetivos semánticos, P el plan generado y E el resultado observado durante la ejecución.

La comparación experimental evalúa la interpretación y el plan; el éxito de ejecución se conserva como un nivel conceptual distinto, que no se mide para los planes generados en dicha comparación.

3.5.1. Niveles de validez

Para analizar las salidas producidas por las arquitecturas evaluadas se consideran los siguientes niveles:

1. **Validez sintáctica:** la salida puede ser procesada por el sistema y respeta la sintaxis esperada.

2. **Validez estructural:** los objetivos, acciones, argumentos y tipos utilizados pertenecen a las estructuras admitidas por el sistema.
3. **Validez semántica:** la representación G conserva la intención expresada en la instrucción original.
4. **Validez del plan:** la secuencia P constituye una realización coherente de los objetivos representados en G.
5. **Éxito de ejecución:** el robot consigue realizar físicamente las acciones planificadas y alcanzar el resultado esperado.

Estos niveles no deben considerarse equivalentes. Una salida puede ser sintáctica y estructuralmente válida y, aun así, representar incorrectamente la intención del usuario. De manera similar, un plan puede ser conceptualmente correcto pero fallar durante la ejecución debido a condiciones externas al proceso de planificación.

Esta separación permite evitar que todos los fallos observados sean agrupados bajo una única categoría de *plan incorrecto*.

3.5.2. Validez de la interpretación semántica

La interpretación se considera válida cuando la secuencia de objetivos G representa fielmente la tarea expresada en la instrucción x dentro del dominio considerado.

Los requisitos de fidelidad fueron introducidos en la Sección 3.3.4. Para la evaluación conceptual se consideran principalmente los siguientes criterios:

1. **Cobertura de las subtareas:** deben conservarse todos los objetivos necesarios para representar la tarea, incluyendo aquellas operaciones implícitas que resulten necesarias dentro del dominio.
2. **Ausencia de intenciones no solicitadas:** la representación no debe introducir objetivos adicionales que modifiquen la intención original de la orden.
3. **Exactitud de entidades y tipos:** las personas, objetos, categorías y ubicaciones utilizadas deben corresponder con las referencias presentes en la instrucción y ser compatibles con el conocimiento K.
4. **Conservación de relaciones y dependencias:** deben mantenerse las relaciones de precedencia, referencia y contexto necesarias entre los objetivos.

Cuando se dispone de una representación de referencia G_{ref}, la interpretación producida por el sistema puede compararse con ella. El caso ideal corresponde a:

$$G_{\text{pred}} = G_{\text{ref}}.$$

Esta condición representa una correspondencia estricta entre la interpretación obtenida y la esperada. Durante la evaluación puede complementarse con criterios más específicos para identificar omisiones, adiciones, errores de entidad, relaciones incorrectas u otros tipos de discrepancia.

Consistencia ante reformulaciones

La variabilidad lingüística introduce un criterio adicional de validez. Como se definió en la Sección 3.3.2, formulaciones semánticamente equivalentes deben conducir idealmente a la misma representación canónica.

Esta propiedad resulta especialmente relevante en GPSR, donde una misma instrucción puede ser reformulada sin modificar sus objetivos.

Por tanto, la evaluación debe considerar no sólo la interpretación de una formulación aislada, sino también la consistencia del sistema ante diferentes expresiones de una misma intención.

3.5.3. Validez y consistencia del plan

Una vez obtenida la representación semántica G, debe determinarse si el plan P constituye una realización coherente de dichos objetivos.

Un plan se considera conceptualmente válido cuando cumple los siguientes criterios:

1. **Acciones soportadas:** todas las acciones utilizadas pertenecen al repertorio $\mathcal{A}$ disponible para el sistema de ejecución.
2. **Parámetros válidos:** cada acción utiliza argumentos compatibles con su función y con las entidades del dominio.
3. **Respeto de dependencias:** la secuencia conserva el orden y las relaciones necesarias para realizar la tarea.
4. **Cobertura de los objetivos:** las acciones generadas permiten atender los objetivos contenidos en G sin omitir partes necesarias de la tarea.
5. **Coherencia con el estado esperado:** las acciones posteriores son compatibles con los resultados esperados de las acciones previamente incluidas en el plan.

Por lo tanto, una secuencia puede estar formada únicamente por acciones válidas y, aun así, constituir un plan incorrecto si presenta un orden inadecuado, utiliza parámetros incompatibles o no cubre completamente los objetivos solicitados.

3.5.4. Validez de la respuesta del sistema

Además de evaluar la corrección de G y P, en la arquitectura GPT directo debe considerarse si el modelo selecciona adecuadamente el tipo de respuesta descrito en la Sección 3.4.3.

Una respuesta `executable` es apropiada cuando la instrucción contiene información suficiente y puede resolverse mediante las capacidades disponibles. Una respuesta `clarification` es apropiada cuando existe una ambigüedad que puede resolverse solicitando información adicional, mientras que `rejected` corresponde a instrucciones fuera del dominio o de las capacidades admitidas.

Por ejemplo:

"Bring it to her."

puede requerir una aclaración si no existe suficiente contexto para determinar qué representan *it* y *her*.

Por tanto, generar un plan no constituye siempre la respuesta correcta. Ante una orden ambigua o no soportada, solicitar una aclaración o rechazar la solicitud puede representar el comportamiento esperado del sistema.

3.5.5. Condición de control para la etapa simbólica

La separación explícita entre interpretación y planificación de la arquitectura Qwen–CLIPS permite introducir una condición experimental de control destinada a aislar el comportamiento del componente simbólico.

En esta condición, los objetivos de referencia se proporcionan al mismo adaptador utilizado por las configuraciones Qwen–CLIPS para convertirlos en hechos de CLIPS:

$$\boxed{G_{\text{ref}} \xrightarrow{\text{adaptación}} F_{\text{ref}} \xrightarrow{\text{CLIPS}} P_{\text{CLIPS}}} \tag{3.13}$$

donde F_{ref} representa los hechos obtenidos a partir de los objetivos de referencia. Esta condición constituye un control auxiliar del procesamiento posterior a la interpretación; no añade una arquitectura ni una cuarta configuración principal a la comparación.

El propósito del control es evaluar la transformación $G \rightarrow F \rightarrow P$, excluyendo la generación de objetivos por parte del modelo de lenguaje.

Si el control genera un plan correcto pero una configuración Qwen–CLIPS falla para la misma instrucción, deben examinarse los objetivos predichos y su transformación en hechos para localizar el origen de la discrepancia.

Si el control también produce un plan incorrecto, deben revisarse la adaptación de los objetivos, las reglas de expansión y las condiciones iniciales utilizadas.

El análisis distingue los siguientes tipos de error:

- **Error de interpretación:** la secuencia G no representa correctamente la instrucción.
- **Error de adaptación:** la transformación de los objetivos en hechos no conserva correctamente la información contenida en G.
- **Error de planificación simbólica:** los objetivos son correctos y se encuentran adecuadamente representados mediante hechos, pero el plan P no constituye una expansión coherente con ellos y con las condiciones iniciales.

La condición de control apoya esta clasificación, pero la atribución de un fallo requiere examinar las representaciones intermedias y el plan generado. Esta separación será utilizada posteriormente para analizar los resultados de la evaluación.

- **error de interpretación:** la secuencia G no representa correctamente la instrucción;
- **error de planificación:** la representación G es correcta, pero el plan P no constituye una expansión adecuada.

Esta separación será utilizada posteriormente para clasificar los resultados obtenidos durante la evaluación.

3.5.6. Planificación y ejecución física

El último nivel de análisis corresponde a la diferencia entre generar un plan correcto y completar satisfactoriamente su ejecución sobre el robot.

La relación fundamental puede expresarse de manera sencilla como:

$$\boxed{\text{plan correcto} \not\Rightarrow \text{ejecución exitosa}} \tag{3.14}$$

Un plan puede representar correctamente la intención de la instrucción y contener una secuencia coherente de acciones, pero su ejecución puede fallar por factores externos al proceso de planificación.

Entre ellos se encuentran, por ejemplo:

- fallos durante la percepción de personas u objetos;
- obstáculos o cambios en el entorno durante la navegación;
- dificultades durante la manipulación de objetos;
- cambios en la posición de personas u objetos;
- problemas durante la interacción humano–robot.

Por ejemplo, el plan puede indicar correctamente que el robot debe tomar una manzana, pero la ejecución puede fallar si el objeto no es detectado o si el manipulador no consigue sujetarlo.

De manera similar, una acción de navegación puede estar correctamente planificada y, aun así, no completarse debido a obstáculos o modificaciones del entorno.

Por esta razón, el estado esperado utilizado durante la planificación no debe confundirse con el estado real observado durante la ejecución.

Por tanto, la comparación entre Qwen–CLIPS y GPT directo se limita a la interpretación de la instrucción y a la validez y consistencia del plan generado. Los resultados observados al utilizar Qwen–CLIPS en Justina durante la RoboCup 2026 se presentan por separado y no constituyen una medición equivalente de la ejecución de ambas arquitecturas.

Los criterios de esta sección definen conceptualmente una interpretación correcta y un plan válido. Las métricas y procedimientos empleados para evaluarlos se presentan en el capítulo de validación y experimentación.

Capítulo 4

Metodología y diseño de las arquitecturas evaluadas

En esta investigación se desarrolló una arquitectura para transformar instrucciones en lenguaje natural en planes de acciones para el robot Justina, mediante la especialización de modelos Qwen como intérpretes semánticos y la expansión de objetivos mediante reglas CLIPS. Esta arquitectura se estudia con Qwen2.5-3B y Qwen2.5-7B. Como arquitectura de comparación se utiliza GPT directo, que genera los objetivos y el plan durante una misma inferencia. De este modo, el diseño contempla dos arquitecturas y tres configuraciones experimentales.

El desarrollo requirió adaptar el generador de comandos GPSR, construir un conjunto de datos con etiquetas semánticas, establecer una interfaz entre la interpretación y la planificación, y expresar el conocimiento operativo en las dos arquitecturas. Estas decisiones se apoyan en las representaciones y los criterios definidos en el Capítulo 3.

Este capítulo explica cómo se construyó el dominio de trabajo, qué modificaciones se realizaron para obtener el conjunto de datos y cómo se diseñó la comparación entre las arquitecturas. Los detalles de código, entrenamiento y despliegue se presentan en el capítulo 5. El protocolo de evaluación, las métricas y las condiciones concretas de los experimentos se desarrollan en el capítulo 6.

4.1. Diseño metodológico de la investigación

4.1.1. Problema experimental

La investigación estudia la fidelidad y la consistencia con las que dos arquitecturas transforman instrucciones de un dominio común en planes compatibles con las capacidades de Justina. La comparación considera tanto la recuperación de las subtareas solicitadas como su realización mediante acciones, con especial atención a las entidades y dependencias que intervienen en órdenes compuestas.

La unidad básica de análisis es una instrucción x, asociada con una secuencia de objetivos de referencia G_{ref}. Se conserva la notación G_{pred} del capítulo anterior para representar una secuencia de objetivos predicha. Cuando sea necesario distinguir su procedencia, se utilizarán $G_{\text{pred}}^{\text{Qwen}}$ y $G_{\text{pred}}^{\text{GPT}}$. Los superíndices identifican el modelo que produce la representación.

La notación correspondiente a Qwen se utiliza de manera general para ambas variantes. Cuando sea necesario distinguir sus resultados, se emplearán los superíndices Qwen3B y Qwen7B para los objetivos, y Qwen3B $-$ $-$CLIPS y Qwen7B $-$ $-$CLIPS para los planes. Cada resultado se asociará con la configuración que lo produjo.

En Qwen–CLIPS, la predicción $G_{\text{pred}}^{\text{Qwen}}$ se somete a validación y conversión antes de su expansión mediante CLIPS, que produce el plan $P^{\text{Qwen–CLIPS}}$. En la arquitectura directa, GPT produce $G_{\text{pred}}^{\text{GPT}}$ y P^{GPT} durante una misma inferencia. Estos productos permiten examinar la interpretación frente a una referencia común y analizar después la correspondencia entre los objetivos y las acciones.

La evaluación se orienta a determinar en qué tareas aparecen diferencias de comportamiento y qué tipo de errores explica esas diferencias. Cuando se utilizan varias formulaciones de una misma intención, se considera también la estabilidad de la representación obtenida ante los cambios lingüísticos.

4.1.2. Variables y condiciones de comparación

El factor principal de comparación es la arquitectura completa. En Qwen–CLIPS, la interpretación se obtiene mediante un modelo especializado y la expansión se realiza con reglas explícitas. En GPT directo, ambas funciones se asignan a un modelo condicionado por una especificación del dominio y del comportamiento operativo esperado.

Dentro de Qwen–CLIPS se considera un segundo factor de comparación: la variante

del intérprete semántico, Qwen2.5-3B o Qwen2.5-7B. Para estudiar esta diferencia se mantienen comunes el conjunto de datos, la partición de entrenamiento y validación, los hiperparámetros de ajuste y el criterio de selección del checkpoint. También se mantienen fijos los procedimientos de normalización y adaptación, las reglas de CLIPS y las condiciones iniciales de planificación.

Las variables de respuesta comprenden la corrección de los objetivos, la validez y consistencia del plan y el rendimiento computacional. La familia de tarea, la longitud de la secuencia de objetivos, los tipos de entidades y las dependencias referenciales se utilizan para organizar el análisis. Las variantes que expresan una misma intención se mantienen identificadas como tales para interpretar adecuadamente los resultados por caso y por grupo de reformulaciones.

El rendimiento computacional se examina mediante la latencia observada desde los programas experimentales para las tres configuraciones. Adicionalmente, se compara el consumo de recursos de ambas variantes Qwen sobre la misma plataforma y bajo condiciones comunes de inferencia. Para GPT directo mediante API no se incluyen mediciones internas de memoria o procesamiento del modelo. Las métricas y los procedimientos de medición se especifican en el capítulo 6.

La comparación se delimita a instrucciones en inglés, un vocabulario semántico común y un repertorio compartido de acciones. El diseño exige igualdad de catálogos, configuración de manipulación y convenciones de inicio y finalización. Estas condiciones permiten plantear la misma tarea a ambos sistemas, aunque cada uno utilice mecanismos distintos para representar y aplicar el conocimiento.

Dado que entre las arquitecturas cambian también el modelo de lenguaje y su adaptación, las diferencias observadas no pueden atribuirse exclusivamente a la presencia de reglas. Las conclusiones se referirán al comportamiento de las configuraciones evaluadas dentro del dominio establecido.

4.1.3. Estrategia general de evaluación

La evaluación se organiza a partir de los niveles de validez definidos en la Sección 3.5. La comparación controlada examina primero la interpretación de la orden y después la validez y consistencia del plan generado, sin ejecutar físicamente los planes de cada caso. La experiencia de Qwen–CLIPS integrado en Justina durante la RoboCup 2026 se analiza por separado y no se utiliza como resultado comparativo frente a GPT directo.

Para estudiar el procesamiento posterior a la interpretación se incorpora la condi-

ción de control de la Sección 3.5.5. Los objetivos de referencia se transforman en hechos mediante el mismo adaptador utilizado por las configuraciones Qwen–CLIPS y se expanden bajo las mismas reglas y condiciones iniciales. Este control permite contrastar el plan resultante con los obtenidos a partir de las predicciones de ambos modelos Qwen. La inspección de los objetivos, los hechos y los planes ayuda a distinguir errores de interpretación, adaptación y planificación simbólica. Se trata de una condición auxiliar, no de una cuarta configuración principal.

Se separan asimismo las comprobaciones realizadas durante el desarrollo de la evaluación comparativa. Las primeras permiten verificar el funcionamiento del sistema y corregir problemas de implementación. La segunda requiere instrucciones y referencias cuyo uso no haya intervenido en el ajuste de los componentes que se comparan.

4.2. Construcción del dominio experimental

4.2.1. Fuentes de conocimiento GPSR

Se tomó como punto de partida el CommandGenerator oficial de RoboCup@Home y la organización del conocimiento de CompetitionTemplate, descritos en la Sección 3.2.2. El primero aporta estructuras de comandos y familias de tareas; el segundo proporciona las entidades con las que se construyen las instrucciones.

Sobre esta base se desarrolló una adaptación destinada a producir pares de órdenes y objetivos canónicos. Se incorporaron mecanismos para conservar el contexto entre subtareas, generar las etiquetas semánticas, introducir variantes lingüísticas y controlar la distribución y calidad de las muestras. De esta forma, la herramienta de generación se integró en un procedimiento de construcción de datos para la especialización del intérprete.

La separación entre estructuras de tareas y catálogos se conservó en la adaptación. Esto permite modificar las entidades de una instancia sin alterar necesariamente la composición de sus objetivos. El alcance lingüístico se mantiene conforme a la Sección 3.2.3: las plantillas sirven para construir datos controlados, mientras que el sistema busca interpretar otras formulaciones que expresen intenciones pertenecientes al mismo dominio.

4.2.2. Catálogos de personas, objetos y ubicaciones

El conocimiento K, definido en la Sección 3.3.3, se concretó mediante catálogos de nombres de personas, objetos, categorías, habitaciones y ubicaciones. Se incluyeron formas singulares y plurales de categorías, así como las descripciones admitidas para búsquedas de personas y selección de objetos.

Se implementó una validación de los catálogos para detectar condiciones que comprometen la generación de etiquetas, como un catálogo esencial vacío o una entidad clasificada simultáneamente como persona y objeto. También se incorporó una comparación de nombres con el banco de conocimiento de Justina para identificar diferencias entre la generación de datos y el sistema integrado.

La clasificación de entidades se apoya en el tipo y en el contexto semántico. Se evitó utilizar la capitalización como criterio único, pues un objeto puede tener un nombre escrito con mayúscula inicial. Además, se distinguió conceptualmente una ubicación de navegación de un soporte de colocación: la primera indica dónde debe desarrollarse una subtarea; el segundo especifica una superficie o recipiente relacionado con la manipulación.

Para la evaluación comparativa, la versión de los catálogos forma parte de la configuración que se mantendrá fija. Este control incluye su uso en el generador, en el adaptador simbólico y en las instrucciones proporcionadas a GPT.

4.2.3. Familias de tareas

El generador adaptado contempla 29 familias de comandos, organizadas en dos categorías principales: tareas de personas y tareas de objetos. Estas categorías sirven para distribuir las muestras según la familia generadora. Una tarea de objetos puede incluir un destinatario humano, por lo que la clasificación no exige que todas las entidades de la instrucción pertenezcan a un mismo tipo.

Se incluyeron tareas de navegación, búsqueda, saludo, seguimiento, guía, conteo, adquisición y comunicación de información, y transporte de objetos. Durante la adaptación se añadieron familias simples para buscar una persona, buscar un objeto y saludar a una persona en una habitación. Su incorporación permite representar estas intenciones como órdenes completas, además de su aparición dentro de instrucciones compuestas.

Para distinguir las composiciones alcanzables dentro de una familia se utilizó la firma de objetivos: una secuencia que conserva los nombres de los objetivos y omite

sus argumentos concretos. Por ejemplo, [go, find] identifica una búsqueda precedida de navegación. Las firmas permiten distribuir muestras según la estructura de la tarea y analizar diferencias que la clasificación por familia, por sí sola, puede ocultar.

4.2.4. Representación de los objetivos de referencia

La representación de referencia adoptada para la comparación utiliza el vocabulario canónico de la Tabla 3.1 y conserva el orden de los objetivos. Para construir las etiquetas del conjunto de datos se desarrollaron funciones semánticas asociadas con las familias de comandos. Estas funciones transforman las entidades y relaciones del contexto de generación en una secuencia de objetivos.

La construcción es programática: la etiqueta se obtiene del contexto con el que se produce la instrucción, sin solicitar una interpretación al modelo que será entrenado. En las tareas compuestas, las funciones propagan el objeto o la persona de referencia hacia las subtareas posteriores. Los objetivos `find` y `count` incluyen el tipo mediante `kind`, mientras que los argumentos conservan los destinos, destinatarios y atributos relevantes.

Una misma función semántica puede utilizarse para diferentes formulaciones de una tarea. Por ejemplo, la familia de transporte entre ubicaciones construye la firma [go, find, take, place] a partir del objeto, el origen y el destino seleccionados. Esta organización vincula la orden con su etiqueta y reduce la necesidad de definir una anotación independiente para cada realización superficial.

Por ejemplo, la instrucción:

> *"Navigate to the bed then find a chocolate jello and take it and give it to me."*

se asocia con la siguiente representación:

```
go(bed)
find(chocolate jello, kind=object)
take(chocolate jello)
deliver(chocolate jello, to=me)
```

En este caso, las referencias *it* mantienen la identidad del objeto introducido en la búsqueda y *me* identifica al operador como destinatario. La etiqueta recoge las subtareas de la orden, mientras que su expansión determinará las acciones concretas de navegación, percepción, manipulación e interacción.

Durante el desarrollo se realizó una revisión manual inicial de algunos pares de instrucciones y objetivos. Posteriormente se incorporaron comprobaciones automatizadas sobre las etiquetas y las propiedades de generación. La revisión manual fue parcial y no constituyó una anotación exhaustiva ni una evaluación independiente de todos los ejemplos.

Las etiquetas se utilizan como referencias de origen programático. Su consistencia se examina mediante los controles de la Sección 4.3.7, que incluyen expectativas definidas en las pruebas para determinadas estructuras. Estos controles detectan errores de contrato y de composición, pero no sustituyen una comparación semántica independiente entre cada instrucción y su etiqueta. Un error compartido por la función generadora y sus controles puede permanecer en el conjunto y propagarse a varias muestras.

Las intenciones de comunicación de información se representan mediante `tell` y las de colocación mediante `place`, conforme al vocabulario de la Tabla 3.1. Se mantiene la distinción entre objetivos y acciones ejecutables: la expansión de `place` puede incluir una acción `drop` del ejecutor.

4.3. Construcción del conjunto de datos

4.3.1. Adaptación del CommandGenerator

La adaptación se diseñó para vincular la generación lingüística con la generación semántica mediante un contexto compartido. En ese contexto se conservan las entidades seleccionadas y las relaciones necesarias para resolver las continuaciones de una orden. La decisión evita que la frase y su etiqueta se construyan mediante selecciones independientes que puedan producir referencias incompatibles.

Se separó la definición de las formas de expresar una tarea de la función que determina sus objetivos. Las variantes superficiales controlan la redacción, mientras que las funciones semánticas establecen la secuencia canónica. Con esta organización, añadir una formulación para una intención existente no requiere modificar su representación semántica; incorporar una tarea diferente exige definir también su composición de objetivos.

Se introdujeron además ajustes para conservar información necesaria en tareas compuestas. Las órdenes que solicitan el nombre, la postura o el gesto de una persona incluyen la adquisición de esa información mediante `save` antes de comunicarla con `tell`.

En el transporte entre ubicaciones se utilizó `place` con la relación `at` para indicar el destino sin imponer una relación de apoyo sobre una superficie. También se corrigió la clasificación de una familia de adquisición de objetos para asignarla a la categoría correspondiente.

Estos cambios forman parte del desarrollo del conjunto de datos y buscan preservar la correspondencia entre intención, etiqueta y expansión. La estructura de los archivos y las funciones que los implementan se presenta en el capítulo 5.

La Tabla 4.1 sintetiza las decisiones de adaptación y su función dentro de la metodología. Se distingue la construcción de etiquetas de los mecanismos que controlan su estructura y la distribución de las muestras.

Tabla 4.1: Modificaciones incorporadas en la adaptación del generador.

Aspecto	**Modificación**	**Propósito**
Construcción de etiquetas	Funciones semánticas asociadas con familias y subtareas.	Obtener objetivos a partir del contexto utilizado para construir la orden.
Contexto compartido	Propagación del objeto o la persona entre subtareas relacionadas.	Mantener la identidad de las entidades durante la composición de la tarea.
Variación lingüística	Separación de las plantillas superficiales y la generación semántica.	Variar la formulación conservando la intención y sus argumentos.
Cobertura de tareas	Incorporación de familias simples de búsqueda y saludo.	Representar esas intenciones como órdenes completas, además de subtareas.
Composición semántica	Captura de información con `save` y representación del destino de transporte mediante `at`.	Conservar operaciones y relaciones necesarias antes de informar o colocar.
Distribución y control	Balance por categoría, familia y firma; deduplicación y validación.	Controlar la cobertura, evitar repeticiones y detectar inconsistencias.

4.3.2. Generación de órdenes

La construcción de una muestra comienza con la selección de una categoría, una familia y una firma de objetivos alcanzable. A continuación se determinan las subtareas que componen la orden, se seleccionan las entidades compatibles y se sustituyen los elementos variables de la plantilla. El contexto se propaga entre las subtareas para conservar sus referencias.

Cada muestra contiene la instrucción, los objetivos y metadatos que identifican su procedencia. Se registran la familia, la categoría, la variante superficial, la firma de objetivos, los tipos de entidades y los campos utilizados. Esta información permite analizar la cobertura del conjunto y agrupar los errores por características de la tarea.

Se incorporaron una semilla de generación y un límite de reintentos para controlar el procedimiento. La muestra se acepta después de comprobar su etiqueta y su condición de unicidad. Cuando una combinación no cumple los criterios, se realiza un nuevo intento dentro de los límites establecidos. La reproducción del conjunto depende de conservar, además de la semilla, las mismas versiones del generador y de sus catálogos.

4.3.3. Generación de objetivos canónicos

Las funciones semánticas transforman el contexto de la muestra en una secuencia de objetivos. En las instrucciones compuestas, esta transformación conserva los objetos y personas compartidos entre acciones y establece el orden lógico de las operaciones. Las intenciones de colocación se representan mediante `place` y las de comunicación de información mediante `tell`, de acuerdo con el vocabulario común de la investigación.

Se desarrolló un validador del contrato semántico que comprueba los objetivos permitidos, sus argumentos y las relaciones admitidas. También verifica dependencias como la adquisición de un objeto antes de su entrega o colocación. Estas comprobaciones permiten rechazar etiquetas incompletas o incompatibles antes de incorporarlas al conjunto.

Para comparar representaciones se distingue la escritura de las entidades de su contenido semántico. La normalización de mayúsculas puede utilizarse cuando preserve la identidad establecida por el catálogo. En todos los casos se mantienen el tipo, los valores de los argumentos, las relaciones y el orden de los objetivos, de modo que una normalización superficial no oculte una diferencia de significado.

4.3.4. Variación lingüística

Se incorporaron varias realizaciones superficiales por familia para introducir distintas formas de expresar una misma estructura de tarea. Las variantes utilizan los mismos campos de contenido y comparten la función de generación semántica. Esta decisión permite variar la forma lingüística sin cambiar la etiqueta cuando se conservan la intención y las entidades.

Se implementó una comprobación de los campos de las variantes al iniciar la generación. Su finalidad es detectar omisiones o incorporaciones de información incompatibles con la familia. La conservación de esos campos no equivale a una demostración de equivalencia lingüística: el orden, el alcance de las expresiones y las relaciones entre subtareas también determinan el significado de la instrucción.

La diversidad introducida mediante plantillas se considera variación controlada dentro del dominio. Para estudiar formulaciones que no procedan de esas plantillas, el diseño de evaluación contempla un grupo separado de reformulaciones elaboradas de manera independiente del entrenamiento. Esta distinción permitirá analizar por separado la interpretación de estructuras conocidas y la respuesta ante nuevas formas de expresar las mismas tareas.

4.3.5. Balance y deduplicación

El conjunto de datos se construyó con 40 000 muestras. Para distribuir la generación se asignó la mitad a familias de personas y la mitad a familias de objetos. Dentro de cada categoría se establecieron cuotas por familia y por firma de objetivos, con el propósito de reducir la concentración de ejemplos en las estructuras con mayor probabilidad de ser seleccionadas.

La deduplicación se realizó sobre la instrucción convertida a minúsculas y con espacios normalizados. Cuando una misma entrada aparecía asociada con objetivos diferentes, el procedimiento la identificaba como un conflicto de etiqueta y la rechazaba. Este criterio evita repeticiones textuales normalizadas y etiquetas contradictorias para una misma entrada, aunque no agrupa todas las expresiones semánticamente equivalentes.

Se incorporó una redistribución de cuotas para las familias cuyo número de combinaciones únicas resulta limitado. Al alcanzar el límite de reintentos, las muestras faltantes se reasignan dentro de la misma categoría, dando prioridad a las familias y firmas menos representadas. Así se conserva la distribución entre tareas de personas y objetos sin introducir duplicados para completar una cuota.

El balance resultante debe interpretarse en estos dos niveles: la proporción entre categorías se mantiene, mientras que la distribución entre familias puede variar debido a la saturación. Las cantidades finales por familia y firma se presentan junto con las características del conjunto en el capítulo 5.

4.3.6. División de entrenamiento y evaluación

Las 40 000 muestras se dividieron en 36 000 ejemplos de entrenamiento y 4 000 de validación. Ambos modelos Qwen utilizaron exactamente las mismas muestras en cada partición. La partición de entrenamiento se empleó para ajustar los adaptadores de cada modelo y la de validación para observar su comportamiento durante el ajuste y seleccionar el checkpoint conforme a un criterio común. Esta última forma parte del desarrollo de los intérpretes y se distingue del conjunto destinado a la comparación final de las tres configuraciones.

La eliminación de duplicados exactos no impide que ambas particiones compartan plantillas o contengan formulaciones de una misma intención. Por ello, los resultados sobre esta división describen el comportamiento de los modelos bajo la distribución generada y no bastan para establecer su capacidad de generalización a formulaciones independientes.

Para la evaluación comparativa se reserva como criterio de diseño un conjunto que no intervenga en el entrenamiento, la selección del modelo ni la modificación de las reglas o del prompt. Cuando se estudien grupos de reformulaciones de una intención concreta, se mantendrá identificada su relación y se controlará su separación respecto de los datos de desarrollo. El procedimiento de construcción y las condiciones de independencia de esa prueba se especifican en el capítulo 6.

Durante el desarrollo se utilizaron además 48 comandos para comprobar el funcionamiento del sistema. Estos casos constituyen una prueba funcional y se mantienen separados de la evaluación experimental. Su descripción corresponde a la verificación de la implementación.

4.3.7. Validación previa del conjunto de datos

Después de la revisión manual parcial se desarrollaron comprobaciones automatizadas para examinar el contrato de las etiquetas, las propiedades del generador y su compatibilidad con CLIPS. El procedimiento distingue la validación de los catálogos, los controles sobre las secuencias de objetivos y la comprobación de expansión. Cada nivel aporta evidencia sobre una propiedad específica del conjunto.

El validador de objetivos comprueba la sintaxis, los nombres y argumentos admitidos, la presencia de destinos obligatorios y el tipo explícito en `find` y `count`. También detecta calificadores incompatibles con el tipo declarado y algunos conflictos de tipo

entre subtareas. En la secuencia se verifica que una entrega o colocación esté precedida por la adquisición del objeto y se detectan ciertas navegaciones o destinos redundantes. Estos controles operan sobre los objetivos; no interpretan de nuevo la instrucción para contrastar su significado.

Se incorporaron además pruebas sobre propiedades de generación, como la cardinalidad solicitada, la ausencia de entradas textuales duplicadas, la proporción entre categorías y la presencia de metadatos. Algunas pruebas contienen expectativas específicas: las familias de búsqueda simple deben producir la firma [go, find], mientras que la de saludo simple debe producir [go, find, greet]. Otras comprueban la presencia de `save` y del retorno al punto de instrucción en las tareas de información, o el uso de `at` en el transporte entre ubicaciones. Estas expectativas están codificadas como propiedades y firmas; no constituyen un conjunto completo de pares anotados de manera independiente.

La comprobación con CLIPS se realizó sobre un subconjunto de secuencias. Para cada una, el validador reinicia el entorno, convierte los objetivos en hechos y ejecuta las reglas. Se comprueba que no queden objetivos pendientes, que no se emita la respuesta correspondiente a un objetivo no soportado y que se produzca la señal de finalización `GPSR_DONE`. La aceptación indica que la secuencia satisface estas condiciones de expansión, sin acreditar por sí sola el cumplimiento semántico o físico de la tarea.

Se implementaron dos recorridos de comprobación. En la herramienta de auditoría, las secuencias válidas se procesan en el orden de lectura hasta alcanzar el límite configurado. En las pruebas del generador se selecciona el primer ejemplo disponible de cada familia del lote de prueba. Estas selecciones ofrecen coberturas diferentes y no equivalen por sí mismas a un muestreo aleatorio del conjunto completo. La validación efectuada sobre algunas muestras se distingue de una verificación exhaustiva de las 40 000 entradas.

Los metadatos asociados con cada muestra permiten identificar la familia, la categoría, la variante superficial, la firma, los tipos de entidades y los campos presentes. Se conservan en el conjunto de datos y se omiten del contenido utilizado para entrenar el intérprete. Su función es describir la procedencia y estructura de la muestra; no almacenan el resultado individual de la comprobación con CLIPS ni acreditan que esa muestra haya sido examinada por todos los validadores.

La herramienta de auditoría calcula por separado los recuentos de etiquetas válidas e inválidas, las incidencias, la cobertura y los resultados de las secuencias comprobadas

con CLIPS, y presenta un reporte agregado. Así se distingue la información descriptiva del conjunto de la evidencia de una ejecución concreta de validación.

El alcance de estos controles es la consistencia respecto de restricciones programadas y la compatibilidad de expansión de las secuencias examinadas. La fidelidad de la etiqueta permanece definida por su correspondencia con la orden. Para la evaluación final, las referencias y los criterios de inclusión se establecerán a partir de la intención y de las capacidades declaradas, sin exigir previamente que CLIPS resuelva cada caso. Esta separación permite analizar los errores compartidos por generación y validación y conservar los casos de falta de cobertura simbólica.

4.4. Diseño de la arquitectura Qwen–CLIPS

4.4.1. Función asignada a Qwen

Se asignó a Qwen2.5-3B y Qwen2.5-7B la misma función: transformar la instrucción x en una secuencia $G_{\text{pred}}^{\text{Qwen}}$, expresada mediante la representación canónica definida en la Sección 3.3.4. Esta delimitación permite especializar ambos modelos en la identificación de subtareas, entidades y relaciones, manteniendo la construcción de acciones en el componente simbólico.

La respuesta del intérprete se restringió a una estructura que contiene la secuencia de objetivos. La decisión facilita su procesamiento automático y establece un punto de observación entre el lenguaje natural y la planificación. Gracias a esta interfaz, es posible comprobar la interpretación antes de examinar el plan y localizar qué información recibe efectivamente CLIPS.

4.4.2. Especialización para interpretación semántica

La especialización se realizó mediante aprendizaje supervisado con los pares de instrucciones y objetivos construidos para el dominio. Cada entrada se presentó al modelo como contexto y la secuencia canónica como respuesta esperada. Los metadatos de generación se conservaron para el análisis del conjunto y no se incluyeron en la respuesta que el modelo aprende a producir.

Para concentrar el ajuste en la generación de los objetivos, se calculó la pérdida sobre los tokens de la respuesta, excluyendo las posiciones del prompt y del relleno. Se

utilizó QLoRA con el modelo base cuantizado a cuatro bits y adaptadores entrenables, lo que permitió especializarlo dentro de los recursos de cómputo disponibles.

El procedimiento de entrenamiento fue común para Qwen2.5-3B y Qwen2.5-7B. Se utilizaron el mismo conjunto de datos, la misma partición, los mismos hiperparámetros y el mismo criterio de selección del checkpoint. En el código de entrenamiento únicamente se modificó el identificador del modelo base, de `Qwen/Qwen2.5-7B` a `Qwen/Qwen2.5-3B`. La selección se realizó por separado para cada modelo; compartir el criterio no implica seleccionar el mismo paso de entrenamiento.

Durante el desarrollo se exploró inicialmente Qwen2.5-0.5B. Las pruebas realizadas mostraron limitaciones para las órdenes complejas consideradas, por lo que se continuó con Qwen2.5-7B. Posteriormente se incorporó Qwen2.5-3B para examinar las diferencias de calidad y rendimiento respecto de la variante de mayor tamaño, manteniendo el mismo procedimiento de especialización y la misma etapa simbólica. La exploración con Qwen2.5-0.5B se conserva como antecedente de desarrollo y no forma parte de la comparación experimental principal.

Los hiperparámetros, las configuraciones de inferencia y la identificación de los adaptadores y checkpoints seleccionados para cada modelo se presentan en el capítulo 5, junto con el procedimiento de entrenamiento.

4.4.3. Interfaz entre objetivos y planificación simbólica

Se desarrolló un adaptador para convertir la salida estructurada del intérprete en hechos que puedan procesarse mediante CLIPS. La interfaz se diseñó para conservar el orden, el nombre del objetivo, el tipo de entidad y los argumentos que determinan su expansión. La normalización de escritura establece la correspondencia entre los nombres de la representación semántica y los identificadores utilizados por el ejecutor.

El adaptador incorpora inferencias de tipo y reparaciones para determinadas secuencias. Estas operaciones forman parte del sistema integrado y pueden modificar la predicción original. Por ello, el diseño de evaluación distingue la salida directa de Qwen de la representación que finalmente se entrega al motor simbólico. Un objetivo corregido por el adaptador puede mejorar el resultado del sistema sin constituir un acierto del modelo antes de esa intervención.

La preservación de argumentos constituye un criterio de validez de la interfaz. Una pérdida de destino o de un atributo relevante durante la conversión se clasifica como un error de adaptación. Esta separación permite evitar que un problema de representación

entre componentes se atribuya automáticamente a la interpretación o a las reglas.

4.4.4. Función asignada a CLIPS

Se desarrollaron reglas de producción para expandir los objetivos en acciones compatibles con Justina. La selección de la siguiente meta se restringió por su posición en la secuencia, de modo que las reglas procesen los objetivos en el orden recibido. Esta organización permite expresar explícitamente el conocimiento operativo de cada tipo de tarea.

Las reglas utilizan el estado esperado descrito en la Sección 3.4.2 para considerar los efectos previstos de las acciones anteriores. Se mantiene información de ubicación y posesión de objetos, además de referencias a personas y análisis previos. Con ello se comprueban algunas dependencias y se reutiliza el contexto establecido durante la construcción del plan.

Se incorporaron también convenciones de interacción y cierre, como los anuncios, el control del foco y el regreso al punto de instrucción. Estas operaciones complementan la realización de las tareas dentro del funcionamiento de Justina y forman parte de la especificación común proporcionada a la arquitectura directa.

La expansión es determinista para una misma base de reglas, hechos iniciales y configuración. Su finalización indica que concluyó la generación de acciones, mientras que el cumplimiento de la orden se evalúa mediante los criterios de validez del plan. En particular, una respuesta hablada que informe de la imposibilidad de ejecutar un objetivo se considera falta de cobertura de esa subtarea.

4.5. Diseño de la arquitectura GPT directo

4.5.1. Funciones asignadas al modelo

Se diseñó una arquitectura en la que un modelo GPT recibe la instrucción x y produce los objetivos $G_{\text{pred}}^{\text{GPT}}$ y el plan P^{GPT} durante una misma inferencia. La decisión de conservar ambos productos permite aplicar criterios de interpretación comparables a los de Qwen y examinar la coherencia entre los objetivos declarados y las acciones propuestas.

La generación se condiciona mediante una especificación operativa detallada, elaborada a partir del dominio y de las convenciones de expansión utilizadas en CLIPS.

Por tanto, la condición directa estudia la capacidad del modelo para generar un plan ajustado a esa especificación. Su alcance corresponde a las tareas, restricciones y configuraciones de esta investigación.

Los objetivos de GPT se conservan como producto de análisis y no se envían a CLIPS. La evaluación distingue los dos artefactos de la respuesta, aunque su producción no constituya dos componentes independientes como en Qwen–CLIPS.

4.5.2. Conocimiento y restricciones proporcionadas

Para orientar la generación se elaboró un prompt con los catálogos de entidades, el vocabulario de objetivos, las acciones disponibles y las restricciones de composición. Se incluyeron criterios sobre resolución de referencias, dependencias de manipulación, relaciones espaciales e interacción con personas.

La especificación también establece cómo conservar coherencia con el estado esperado durante la generación. El modelo recibe instrucciones para considerar la ubicación, el objeto sostenido y las entidades identificadas al determinar las acciones posteriores. Se incorporaron las convenciones de identificación, inicio y finalización del plan para facilitar su compatibilidad con el ejecutor.

El conocimiento operativo se presenta así mediante instrucciones en la arquitectura directa y mediante reglas en el componente CLIPS. Para la comparación se fijarán la versión del modelo, el prompt y sus configuraciones de generación, evitando modificar estas condiciones a partir de los resultados de la prueba final.

4.5.3. Contrato de salida

Se definió un contrato de respuesta que contiene el estado de la solicitud, un mensaje, los objetivos y el plan. Para los casos ejecutables se establece una secuencia ordenada de pasos con los campos requeridos por el ejecutor, acompañada por las señales de inicio y finalización. La estructura facilita la extracción y el análisis automático de los productos del modelo.

El diseño utiliza un esquema de salida para restringir los campos, tipos y nombres de acciones admitidos. La validación del contenido se considera una comprobación adicional: además de cumplir la estructura, la respuesta debe conservar las entidades de la orden, respetar las dependencias y cubrir las subtareas solicitadas.

La evaluación conservará la respuesta original y distinguirá los errores de formato,

las inconsistencias de contenido y los fallos de comunicación con el servicio. Esta clasificación permite analizar los resultados del modelo sin confundirlos con problemas de transporte o procesamiento de su respuesta.

4.5.4. Plan ejecutable, aclaración y rechazo

Se adoptaron los estados `executable`, `clarification` y `rejected`, definidos en la Sección 3.4.3. Esta decisión permite representar tanto la producción de un plan como las situaciones en las que falta información o la tarea excede las restricciones del dominio. Las respuestas de aclaración y rechazo contienen un mensaje y no incluyen acciones operativas.

La comparación principal se delimita a instrucciones resolubles dentro del dominio común. En estas condiciones, una aclaración innecesaria o el rechazo de una orden válida se consideran ausencia de una respuesta ejecutable correcta y se mantienen en el total de casos evaluados.

El comportamiento ante órdenes ambiguas o fuera del dominio se considera un análisis adicional. Su inclusión requiere criterios específicos para las respuestas disponibles en cada arquitectura, sin suponer que el rechazo de una salida inválida en Qwen–CLIPS equivale a un mecanismo conversacional de aclaración.

4.6. Diseño de la comparación

Para realizar la comparación se emplean dos programas independientes del sistema de ejecución física del robot. Uno implementa la cadena Qwen–CLIPS y se utiliza con Qwen2.5-3B y Qwen2.5-7B; el otro consulta al planificador GPT directo y valida su respuesta. De este modo, los dos programas permiten evaluar las tres configuraciones sobre los mismos casos de prueba.

Ambos emplean convenciones comunes para registrar las entradas, los objetivos interpretados, los planes generados y los fallos. Esta organización permite aplicar criterios de evaluación comunes sin depender de los módulos de percepción, navegación y manipulación de Justina.

4.6.1. Igualdad del dominio de evaluación

La comparación se plantea sobre el mismo conjunto de instrucciones y referencias, con las condiciones comunes establecidas en la Sección 4.1. Cada caso se identificará por su familia, estructura de objetivos y procedencia lingüística. La conservación de esta información permitirá examinar diferencias por tipo de tarea y por formulación.

Las referencias se definirán a partir de la intención de la orden y del dominio de capacidades. Los ejemplos utilizados para modificar el generador, el modelo, las reglas o el prompt permanecerán identificados como parte del desarrollo. El procedimiento experimental establecerá el conjunto de prueba y lo mantendrá fijo durante la comparación.

Se registrará también la entrada efectiva de cada arquitectura. La normalización previa puede modificar aspectos superficiales o resolver ciertas variaciones antes de la inferencia, por lo que forma parte de la configuración evaluada. La medición de latencia considerará el recorrido completo de la solicitud y distinguirá, cuando sea posible, el procesamiento del modelo de la comunicación y la expansión.

La ubicación del cómputo se tratará como una condición de despliegue. El uso de Qwen con pesos bajo control del proyecto permite alojar la inferencia en infraestructura propia, incluida una computadora remota. Por ello, su comparación temporal con GPT debe considerar las condiciones de comunicación utilizadas en cada caso.

4.6.2. Igualdad del repertorio de acciones

Los planes se evaluarán respecto de un contrato común de acciones, parámetros y efectos esperados. Se mantendrán las mismas capacidades de manipulación, convenciones de identificación y políticas de cierre. Este control permite comparar cómo cada arquitectura representa la tarea mediante un plan compatible con las capacidades definidas para Justina.

Se distinguen dos aspectos del resultado: la fidelidad a las convenciones operativas y el cumplimiento de la instrucción. La correspondencia con una expansión de referencia permite estudiar la reproducción de esas convenciones. En GPT directo, este aspecto refleja también el seguimiento de la especificación incluida en el prompt.

El cumplimiento se examinará mediante la cobertura de subtareas, el orden de las acciones, sus parámetros y su coherencia con el estado esperado. Una diferencia en el texto de un anuncio puede conservar el significado de la tarea, mientras que una

secuencia idéntica a una expansión de referencia puede resultar insuficiente si omite una operación necesaria. Por ello, la coincidencia con CLIPS no se utiliza como criterio único de corrección.

La correspondencia entre objetivos y capacidades se establece a partir de los efectos que las acciones permiten obtener. Las expresiones habladas sobre una tarea no sustituyen la operación necesaria para realizarla. Este criterio se aplica también a los atributos y propiedades solicitados, cuya conservación debe examinarse en la traducción hacia las acciones del ejecutor.

4.6.3. Separación entre interpretación y planificación

La interpretación se evaluará mediante la correspondencia de $G_{\text{pred}}^{\text{Qwen}}$ y $G_{\text{pred}}^{\text{GPT}}$ con la misma secuencia de referencia G_{ref}. Para la planificación se examinarán los planes $P^{\text{Qwen–CLIPS}}$ y P^{GPT}, considerando tanto su coherencia con los objetivos producidos por cada arquitectura como su cumplimiento de la tarea de referencia. Esta doble relación permite identificar un plan internamente coherente que, sin embargo, resuelve una interpretación equivocada de la instrucción.

En ambas configuraciones Qwen–CLIPS, el análisis distinguirá la predicción del modelo, su normalización y conversión a hechos, y el plan producido por CLIPS. La condición de control utilizará los objetivos de referencia a través del mismo adaptador y bajo las mismas reglas y condiciones iniciales de planificación.

Las diferencias entre el programa experimental y la integración con ROS 2 se documentarán para delimitar qué resultados de la comparación pueden trasladarse al sistema utilizado en Justina.

En GPT directo se analizarán los objetivos y las acciones de una misma respuesta. La clasificación permite describir qué producto incumple los criterios de validez, aunque no identifica con certeza una etapa causal interna del modelo. En ambas arquitecturas se conservarán los casos inválidos y los fallos de interfaz como parte del registro experimental.

Con esta organización, el diseño relaciona la construcción de datos, la especialización del intérprete y la generación de planes con criterios de evaluación diferenciados. El capítulo siguiente presenta la implementación de los componentes desarrollados y su integración con Justina.

Capítulo 5

Implementación

Este capítulo describe los componentes desarrollados para transformar instrucciones del lenguaje natural en planes de acciones para el robot de servicio Justina. La implementación concreta las decisiones metodológicas del Capítulo 4 mediante la generación de datos supervisados, la especialización del intérprete Qwen, la conversión de objetivos a hechos simbólicos y la expansión de acciones con CLIPS.

El sistema se organiza en componentes con responsabilidades diferenciadas. La construcción del conjunto de datos y el entrenamiento se realizan fuera del ciclo de operación del robot. Durante la planificación, el intérprete recibe una instrucción y devuelve objetivos que se validan y convierten antes de ingresar al motor simbólico. El plan resultante se publica para su consumo por el sistema de ejecución. La arquitectura GPT directo se trata como una implementación separada, con su propio contrato de respuesta e interfaz de generación.

Como ejemplo conductor se utiliza la orden de búsqueda, adquisición y entrega de un objeto introducida en el capítulo anterior. Sus diferentes representaciones permiten relacionar la entrada lingüística, los objetivos, los hechos CLIPS y las acciones del ejecutor. Se incluyen los registros necesarios para identificar el modelo y las comprobaciones preliminares realizadas. El análisis cuantitativo del desempeño y la comparación experimental se reservan para el capítulo de resultados.

5.1. Plataforma experimental

5.1.1. Robot Justina

La implementación se integra en la arquitectura ViRBot de Justina, descrita en la Sección 3.1.1. El sistema de planificación representa solicitudes de navegación, percepción, manipulación e interacción mediante el repertorio de acciones del robot. Su salida especifica las acciones que deberían solicitarse a esos componentes. Las comprobaciones descritas se limitan a la generación del plan y no incluyen la ejecución física de estas capacidades.

La arquitectura Qwen–CLIPS se utilizó integrada en Justina durante la RoboCup 2026. Esa experiencia corresponde a la operación del sistema en el robot y se documenta separadamente de la comparación experimental de planes, en la que no se ejecutan físicamente las salidas de ambas arquitecturas.

La comunicación con la plataforma se realiza mediante ROS 2. El planificador produce identificadores de acciones y parámetros simbólicos, como una ubicación de navegación, un objeto o una referencia a una persona. La resolución física de esas instrucciones corresponde al ejecutor y a los módulos encargados de cada tarea.

En el ciclo GPSR se utiliza `instruction_point` para representar el punto de interacción con el operador. Este símbolo se distingue de `inspection point`, que puede aparecer como una entidad del catálogo. La diferencia se conserva al normalizar los parámetros del plan.

5.1.2. Plataforma de cómputo

La plataforma de cómputo utilizada para el trabajo con Qwen2.5-7B cuenta con una GPU NVIDIA GeForce RTX 4070 SUPER, un procesador Intel Core i7 de undécima generación y 32 GB de memoria RAM. La GPU dispone de 12 GB nominales de memoria; la consulta del dispositivo reporta 12 282 MiB. Estos recursos condicionaron la selección del modelo y la configuración de adaptación, descritas en la Sección 5.3.

Durante la operación, la inferencia puede ejecutarse en el equipo del robot o en una computadora remota. Las modalidades de despliegue y sus interfaces se describen en la Sección 5.3.6.

5.1.3. Entorno de software y ROS 2

Los componentes de generación, validación e interpretación se desarrollaron en Python. El entorno registrado utiliza Ubuntu 24.04.5 LTS, Python 3.12.3 y ROS 2 Jazzy. El entrenamiento se implementó mediante `Trainer`, de Transformers, con PyTorch para el cálculo de tensores, PEFT para los adaptadores LoRA, bitsandbytes para la cuantización y Datasets para la preparación de las particiones. La interacción con CLIPS utiliza `clipspy`, importado en Python como `clips`.

ROS 2 proporciona los servicios y tópicos que conectan los componentes de Qwen–CLIPS en Justina. Los archivos de lanzamiento agrupan los nodos y configuran nombres, parámetros y espacios de nombres. Para la comparación de planes se emplean programas experimentales independientes del sistema de ejecución del robot; su funcionamiento se describe en las secciones correspondientes. El programa GPT utiliza el cliente de OpenAI y la biblioteca `jsonschema`. Las versiones registradas se reúnen en la Sección 5.8.

5.2. Implementación del conjunto de datos

5.2.1. Generador de órdenes

La generación se organiza alrededor de `generate_dataset.py`, que coordina la selección de muestras, el balance, la deduplicación y la validación previa a la escritura. El componente `gpsr_commands.py` construye las órdenes utilizando las familias y variantes definidas en `command_constants.py`.

La generación semántica se implementó en `command_goals.py` mediante la clase `CommandGoalsMixin`. El método `_generate_goals` selecciona la función registrada para una familia y le entrega el contexto de construcción. Cada función devuelve una lista ordenada de objetivos. Las continuaciones de una orden pueden invocar otras funciones y recibir el objeto o la persona establecidos por las subtareas anteriores.

Por ejemplo, la función de transporte entre dos ubicaciones toma el objeto, el origen y el destino del contexto y construye los objetivos de navegación, búsqueda, adquisición y colocación. La función de entrega al operador utiliza el objeto conservado en el contexto y genera `deliver(object, to=me)`. Esta propagación permite mantener el referente de un pronombre en las distintas partes de una orden.

La representación de entrada y etiqueta del ejemplo conductor es:

```
{
  "input": "Navigate to the bed then find a chocolate jello and take it and give it to me",
  "goals": [
    "go(bed)",
    "find(chocolate jello, kind=object)",
    "take(chocolate jello)",
    "deliver(chocolate jello, to=me)"
  ]
}
```

El ejemplo omite los metadatos para destacar la correspondencia entre entrada y objetivos. En el archivo JSONL, cada muestra completa se serializa en una sola línea.

5.2.2. Catálogos y CompetitionTemplate

El módulo `knowledge.py` carga los catálogos mediante `parse_data`. La estructura de entrada separa los nombres de personas, las ubicaciones, las habitaciones y los objetos en archivos de CompetitionTemplate. Las rutas relativas utilizadas son:

```
names/names.md
maps/location_names.md
maps/room_names.md
objects/objects.md
```

El conocimiento cargado se entrega al generador para sustituir los elementos variables de las plantillas. La validación de catálogos se encapsula en `catalog_validation.py`; incluye la detección de colisiones y diferencias con el catálogo de nombres del banco de conocimiento de Justina.

La localización del espacio de trabajo del robot puede configurarse con `JUSTINA_WS`. Esto permite vincular las comprobaciones con el repositorio utilizado en el despliegue sin incorporar rutas personales al contrato de generación.

5.2.3. Validadores

La validación de etiquetas se implementó en `goal_schema.py`. La función `parse_goal` separa el nombre del objetivo, sus argumentos posicionales y los campos nombrados, y almacena el resultado en una estructura `ParsedGoal`. La función `validate_goals` comprueba sintaxis, argumentos obligatorios, campos admitidos, compatibilidad de tipos y

calificadores, y dependencias programadas entre objetivos. Las incidencias se representan mediante `ValidationIssue`, con un código, un mensaje y, cuando corresponde, la posición del objetivo.

La auditoría del archivo final utiliza `evaluate_dataset.py` y las funciones de `dataset_evaluation.p` El procedimiento recorre las filas, valida las etiquetas y acumula estadísticas por familia, firma de objetivos, tipos de entidades y campos. La revisión manual realizada durante el desarrollo abarcó partes del conjunto; la comprobación sistemática posterior se ejecutó mediante estos validadores programados.

La comprobación simbólica se implementó en `ClipsPlanValidator`. El componente carga las reglas, reinicia el entorno para cada secuencia, convierte sus objetivos a hechos e inicia la expansión. Para aceptar una secuencia exige que no queden hechos `gpsr-goal`, que se produzca `GPSR_DONE` y que no aparezca el mensaje de objetivo no soportado. La conversión utilizada en esta auditoría se distingue de las heurísticas del adaptador ROS 2 descritas en la Sección 5.4.1.

La auditoría recorrió las 40 000 filas del archivo final. Todas sus etiquetas fueron aceptadas por las comprobaciones del esquema y no se registraron incidencias de validación. Para CLIPS se estableció un límite de 200 secuencias: se examinaron las primeras 200 etiquetas válidas en el orden del archivo y todas superaron los criterios de expansión. Estas secuencias pertenecen a 25 de las 29 familias presentes en el conjunto. La selección corresponde a un prefijo del archivo y no a una muestra aleatoria ni a una cobertura completa de familias.

Los resultados acreditan el cumplimiento de las restricciones programadas y la expansión de las secuencias examinadas. No constituyen una revisión independiente de la correspondencia semántica de cada orden con su etiqueta ni una comprobación de ejecución física.

Las pruebas de `test_dataset_quality.py` complementan esta auditoría mediante un lote generado de 500 muestras. Comprueban cardinalidad, deduplicación, distribución entre categorías, metadatos y firmas esperadas. La prueba simbólica selecciona el primer ejemplo de cada familia presente en ese lote. Este procedimiento se distingue de las 200 secuencias del archivo final comprobadas en la auditoría.

5.2.4. Dataset final

El conjunto final se conserva en `dataset_gpsr.jsonl`, con 40 000 registros y un objeto JSON por línea. La auditoría identifica 29 familias de órdenes, 18 firmas distintas

de objetivos y 87 identificadores de plantilla superficial. Cada registro contiene `input`, `goals` y `meta`; el vocabulario de objetivos se define en la Tabla 3.1.

Los metadatos identifican la familia, la categoría, la variante superficial, la firma de objetivos, los tipos de entidades y los campos presentes. Permiten caracterizar el conjunto y analizar su cobertura, pero se excluyen de la entrada y de la respuesta supervisadas. Tampoco constituyen un registro individual de los resultados de CLIPS.

El entrenamiento utiliza la instrucción como entrada y un objeto JSON con la propiedad `goals` como respuesta esperada. El cargador conserva temporalmente la familia y la categoría como columnas auxiliares y las elimina al construir las entradas tokenizadas.

El archivo auditado corresponde al conjunto utilizado en el entrenamiento. La partición utiliza `train_test_split`, una proporción de validación de 0.1 y semilla 42, con 36 000 muestras de entrenamiento y 4 000 de validación. La división se realiza aleatoriamente por filas, sin agrupar ni estratificar por familia o plantilla. Por ello, la pérdida de validación caracteriza ejemplos de la distribución del generador y no acredita por sí sola generalización a familias excluidas del ajuste.

La identidad del archivo auditado se conserva mediante su huella SHA-256, incluida en la Sección 5.8.

5.3. Entrenamiento e inferencia de Qwen2.5-7B

5.3.1. Preparación del modelo

El entrenamiento se organiza en `Nl-Cl.py` y utiliza `Qwen/Qwen2.5-7B` como modelo base. Cada fila se valida y se transforma en una conversación mediante `apply_chat_template`. El código proporciona un mensaje de usuario con la orden y un mensaje de asistente con los objetivos serializados como JSON.

La plantilla conservada en `tokenizer_config.json` incorpora automáticamente un mensaje de sistema cuando la conversación no contiene uno explícito. Su contenido es *You are a helpful assistant.* Por tanto, aunque el código construye dos mensajes, el texto tokenizado incluye también este prefijo predeterminado. El tokenizador utiliza `<|endoftext|>` como token de fin de secuencia y de relleno.

El cálculo de la pérdida se concentra en la respuesta del asistente. Las posiciones del prefijo reciben la etiqueta de exclusión `-100`, al igual que el relleno del lote. El

componente `CompletionOnlyCollator` realiza relleno dinámico a la derecha hasta un múltiplo de ocho posiciones y ajusta máscaras de atención y etiquetas.

La conversación completa se tokeniza con truncamiento y una longitud máxima de 512 tokens, que comprende el prefijo, la orden y la respuesta. El prefijo se tokeniza por separado para determinar las posiciones excluidas de la pérdida. El límite configurado para el entrenamiento es independiente de la longitud máxima declarada por el tokenizador; una secuencia extensa puede perder objetivos situados al final de la respuesta.

5.3.2. Configuración QLoRA

El modelo base se carga con cuantización de cuatro bits en formato NF4 y doble cuantización. Sus pesos permanecen congelados y el ajuste se realiza sobre adaptadores LoRA insertados en las proyecciones de atención y del bloque de transformación del modelo.

Los módulos seleccionados son `q_proj`, `k_proj`, `v_proj`, `o_proj`, `gate_proj`, `up_proj` y `down_proj`. Se utiliza rango 32, factor de escala 64 y una probabilidad de descarte de 0.05. Esta configuración delimita la capacidad entrenable sin actualizar todos los parámetros del modelo base.

El cálculo utiliza BF16 cuando la GPU declara soporte para ese formato y FP16 en caso contrario. La carga selecciona la implementación de atención `eager`, desactiva la ventana deslizante y distribuye el modelo con `device_map='auto'`. Durante el ajuste se desactiva la caché y se habilita *gradient checkpointing* con `use_reentrant=False`, reduciendo las activaciones conservadas a cambio de recalcularlas durante la retropropagación. El script requiere CUDA y limita a 0.9 la fracción de memoria disponible para el asignador de PyTorch en el dispositivo configurado.

Al guardar el modelo se conserva el adaptador y su configuración, junto con el tokenizador. La inferencia requiere cargar estos artefactos sobre la misma versión del modelo base.

5.3.3. Hiperparámetros de entrenamiento

La Tabla 5.1 presenta los valores conservados en los argumentos de entrenamiento y la configuración del adaptador. El lote de entrenamiento es de una muestra por dispositivo y se acumulan ocho pasos antes de actualizar los parámetros, lo que corresponde

a un lote efectivo de ocho muestras con una GPU.

Tabla 5.1: Configuración conservada del entrenamiento de Qwen2.5-7B.

Parámetro	**Valor**
Longitud máxima de la conversación	512 tokens
Lote de entrenamiento por dispositivo	1
Lote de evaluación por dispositivo	8
Acumulación de gradientes	8
Épocas configuradas	2
Optimizador	AdamW de PyTorch
Tasa de aprendizaje inicial	10^{-4}
Planificador de tasa de aprendizaje	Coseno
Pasos de calentamiento	150
Decaimiento de pesos	0
Coeficientes de Adam	$\beta_1 = 0.9$, $\beta_2 = 0.999$
Épsilon de Adam	10^{-8}
Norma máxima del gradiente	1.0
Intervalo de evaluación	250 pasos
Intervalo de guardado	250 pasos
Intervalo de registro	50 pasos
Límite de checkpoints conservados	2
Precisión mixta configurada	BF16
Semilla de entrenamiento	42
Rango / escala / descarte LoRA	32 / 64 / 0.05

El optimizador se identifica como `adamw_torch`. Los argumentos registran `bf16=True`, `fp16=False` y `seed=42`; no se especifica una semilla independiente en `data_seed`. La partición de los datos utiliza también el valor 42, en una operación distinta del control de aleatoriedad del entrenamiento.

Con 36 000 muestras, un lote efectivo de ocho y dos épocas, la configuración corresponde a 9 000 actualizaciones. El registro del checkpoint final confirma ese número de pasos y dos épocas completadas. La política de conservación se describe a continuación.

5.3.4. Selección y conservación del checkpoint

La selección utiliza la pérdida de validación calculada sobre las 4 000 muestras de esa partición. La evaluación y el guardado se realizan cada 250 actualizaciones, con un límite de conservación de dos checkpoints. Los argumentos efectivos del entrenamiento establecen:

```
load_best_model_at_end=True
metric_for_best_model="eval_loss"
greater_is_better=False
```

El entrenamiento completó 9 000 actualizaciones y dos épocas. El estado final conserva 36 evaluaciones e identifica `checkpoint-5000` como el checkpoint seleccionado, con una pérdida de validación de aproximadamente 0.2321400046. El último checkpoint, correspondiente al paso 9 000, registra una pérdida de aproximadamente 0.2321625948. Por tanto, el paso de terminación del entrenamiento se distingue del paso de los pesos seleccionados. La selección responde exclusivamente al criterio de menor pérdida de validación; no demuestra por sí sola que ese checkpoint produzca los mejores planes.

Al finalizar `trainer.train()`, el entrenador restaura los pesos del checkpoint seleccionado. Después, el script ejecuta:

```
trainer.save_model(OUTPUT_DIR)
tokenizer.save_pretrained(OUTPUT_DIR)
```

El directorio `nl2cd_qwen7b` conserva así el adaptador seleccionado y el tokenizador para la inferencia. Los subdirectorios conservados corresponden a `checkpoint-5000` y `checkpoint-9000`.

La Figura 5.1 se construye con el historial completo, presenta la evolución de las pérdidas de entrenamiento y validación durante las 9 000 actualizaciones. Se observa una reducción inicial pronunciada, seguida de variaciones de menor magnitud. El registro del entrenador identifica el paso 5 000 como el de menor pérdida de validación, aunque esta diferencia no se distingue con claridad en la escala de la figura.

El script realiza posteriormente una comprobación generativa sobre las primeras 50 muestras de la partición de validación, utilizando las órdenes del dataset sin el normalizador del servicio HTTP y un máximo de 256 tokens nuevos. Compara las listas ordenadas de objetivos predichos con sus referencias, aplica el esquema y comprueba la expansión mediante el validador CLIPS fuera de línea. La salida conservada registra 50 coincidencias exactas, 50 secuencias válidas según el esquema y 50 secuencias aceptadas por CLIPS. También incluye cinco órdenes adicionales cuyas respuestas se imprimen como ejemplos de inferencia.

Esta comprobación utiliza una parte de la misma partición empleada para seleccionar el checkpoint. Su alcance es preliminar: no constituye una evaluación sobre un conjunto de prueba independiente, una verificación del recorrido HTTP–ROS 2 ni una

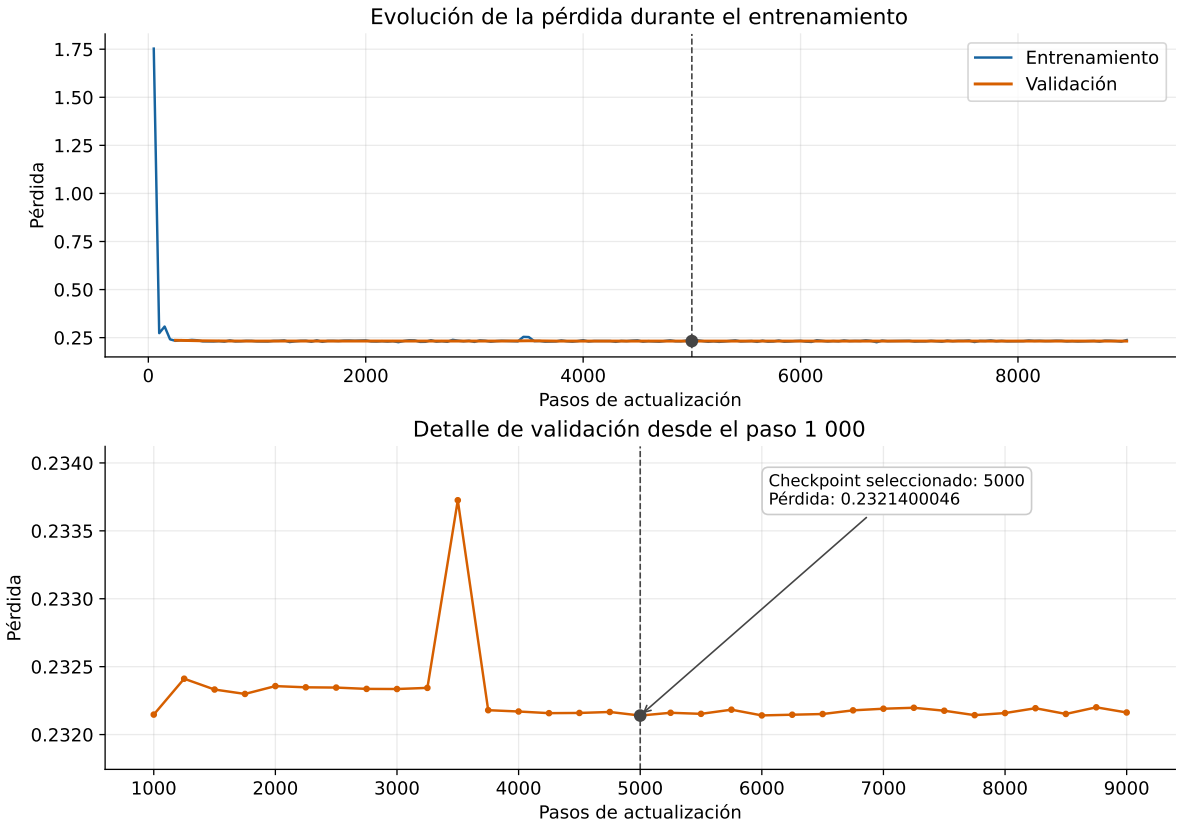


Figura 5.1: Evolución de las pérdidas de entrenamiento y validación durante el ajuste de Qwen2.5-7B mediante QLoRA. La pérdida de entrenamiento se registra cada 50 actualizaciones y la de validación cada 250. Se muestra el historial completo, hasta el paso 9 000. Elaboración propia.

comparación con GPT. La evaluación detallada de ambas arquitecturas queda reservada para el capítulo de resultados.

5.3.5. Inferencia

El módulo `inference.py` carga `Qwen/Qwen2.5-7B` en NF4 con doble cuantización e incorpora los pesos del adaptador mediante `PeftModel.from_pretrained`. Recupera el tokenizador del directorio del adaptador y coloca el modelo en modo de evaluación. El dispositivo predeterminado es `cuda:0`; las opciones de ejecución permiten modificar la distribución y los límites de memoria.

Antes de tokenizar la orden se aplica `CommandNormalizer`. El procedimiento convierte el texto a minúsculas, uniforma espacios, guiones y signos, elimina las expresiones de cortesía y muletillas definidas en el código y reduce repeticiones adyacentes. Después sustituye variantes de expresiones, como *pick up* por *take* y *search for* por *find*, y aplica alias de entidades, como *fridge* por *refrigerator*.

El normalizador construye su repertorio con los catálogos de `CompetitionTemplate` y las listas de atributos e información del generador. Tras las sustituciones exactas, busca coincidencias aproximadas mediante `SequenceMatcher`. Compara candidatos con el mismo número de palabras y utiliza un umbral de 0.86 para una palabra y 0.82 para expresiones de varias palabras. Descarta de esta búsqueda los fragmentos de menos de cuatro caracteres y examina primero las secuencias más largas. Las sustituciones se basan en similitud textual; no constituyen una comprobación semántica de la orden.

El modo de diagnóstico del normalizador conserva la entrada original, la normalizada y la lista de transformaciones. En el recorrido de inferencia se utiliza el texto normalizado, que puede devolverse junto con la predicción mediante `return_normalized=True`. Estas transformaciones pertenecen al preprocesamiento y se distinguen de las capacidades aprendidas por Qwen.

La orden procesada se introduce como mensaje de usuario y la plantilla añade el prefijo de sistema predeterminado y el inicio de la respuesta del asistente. La generación utiliza `do_sample=False`, una penalización de repetición de 1.05 y un máximo de 128 tokens nuevos. Las condiciones de parada incluyen el token de fin de secuencia y `<|im_end|>`, cuando está disponible. Las comprobaciones generativas del script de entrenamiento utilizan 256 tokens nuevos; se trata de una configuración distinta de la inferencia del servicio.

Se decodifica únicamente la continuación generada. La extracción localiza el primer

fragmento con llaves balanceadas e intenta convertirlo mediante `json.loads`; posteriormente se valida el contrato de objetivos. La generación no impone una gramática JSON. Si la extracción o validación falla, se devuelve un error con hasta 200 caracteres de la respuesta para diagnóstico. La aceptación comprueba estructura y restricciones programadas, sin acreditar por sí sola fidelidad a la instrucción.

La salida del intérprete se representa mediante $G_{\text{pred}}^{\text{Qwen}}$. Las reparaciones posteriores del adaptador ROS 2 se registran como operaciones adicionales y no se atribuyen a la predicción original al evaluar el intérprete.

5.3.6. Despliegue del modelo

El intérprete Qwen admite dos modalidades de despliegue: ejecución local y ejecución como servicio remoto. Ambas emplean el modelo base, el adaptador LoRA y el tokenizador, y conservan el mismo procedimiento de inferencia y contrato de objetivos semánticos. La diferencia reside en la ubicación del proceso y en el mecanismo utilizado para solicitar la interpretación.

En la modalidad local, el modelo se carga en el mismo equipo que ejecuta los componentes de control del robot. El módulo `inference.py` proporciona las funciones de carga y traducción, permitiendo procesar las instrucciones desde un proceso Python. Una vez disponibles los archivos necesarios, la interpretación de cada orden no requiere consultar un servidor externo.

Cuando la carga conjunta de los componentes requiere liberar recursos del equipo del robot, la inferencia puede trasladarse a otra computadora accesible mediante una red local o Internet. Para esta modalidad se desarrolló `server.py`, que mantiene cargados el modelo, el tokenizador y el normalizador entre solicitudes. Justina envía la instrucción por HTTP y recibe los objetivos, mientras que la conversión a hechos y la planificación con CLIPS permanecen en su entorno ROS 2.

El servidor atiende solicitudes mediante hilos y utiliza un mecanismo de exclusión mutua para permitir una sola traducción a la vez. La interfaz ofrece `GET /health` para consultar la disponibilidad del servicio y `POST /translate` para solicitar la interpretación. El cuerpo de la petición contiene la propiedad `command`:

```
{"command": "go to the kitchen"}
```

Una respuesta aceptada incluye el indicador `ok`, el tiempo `elapsed_ms` y un objeto `result`. Dentro de este último, `normalized_input` conserva la entrada procesada y

`prediction` contiene los objetivos. La envoltura del servicio se retira antes de entregar al adaptador el contrato semántico:

```
{"goals": ["go(kitchen)"]}
```

Las peticiones incorrectas, las predicciones inválidas y los fallos internos se comunican mediante respuestas de error. Cuando se configura una clave de acceso, el servidor comprueba el encabezado `Authorization` mediante el esquema `Bearer`.

El despliegue configurado utiliza el servicio HTTP, iniciado desde el directorio de trabajo `Qwen2.5-7B` del usuario, con la dirección IP `100.102.21.62` y puerto 8008. El comando proporciona la clave mediante la variable `QWEN_API_KEY` y no especifica `--adapter-path`. Por ello, el servidor utiliza el valor `ADAPTER_PATH` del módulo de inferencia, establecido como `nl2cd_qwen7b`. La ruta resultante es:

```
~/Qwen2.5-7B/nl2cd_qwen7b
```

El tokenizador y el adaptador se recuperan de ese directorio; el modelo base se carga mediante el identificador `Qwen/Qwen2.5-7B`. El adaptador exportado corresponde al checkpoint seleccionado durante el entrenamiento. La selección y restauración se realizan en el entrenador; el servidor carga los archivos de la ruta configurada sin comparar los checkpoints. El comando conserva también el dispositivo predeterminado `cuda:0`, mantiene TF32 habilitado y no activa la opción `--cpu-offload`.

La modalidad utilizada forma parte de la configuración experimental. El tiempo informado por el servidor corresponde al procesamiento de la traducción y no incluye el recorrido completo de comunicación observado por el cliente. El registro de estas condiciones y el alcance de las mediciones se describen en la Sección 5.8.

5.4. Implementación del planificador CLIPS

5.4.1. Conversión de objetivos a hechos gpsr-goal

El adaptador `goals_to_clips_node.py` recibe la lista de objetivos y construye hechos `gpsr-goal`. El componente `GoalParser` acepta un objeto con la propiedad `goals` o directamente una lista JSON. Analiza cada expresión y separa los argumentos posicionales de los pares `clave=valor`, respetando las comas delimitadoras y los valores entre comillas. Cada hecho conserva la posición, el nombre del objetivo, el objetivo principal y los campos utilizados para su expansión. La estructura compartida es:

```
(deftemplate gpsr-goal
  (slot step (type INTEGER))
  (slot name)
  (slot target (default ""))
  (slot kind (default unknown))
  (slot destination (default ""))
  (slot relation (default ""))
  (slot value (default ""))
  (slot qualifier (default "")))
```

Para el segundo objetivo del ejemplo conductor, la representación es:

```
(gpsr-goal
  (step 2)
  (name find)
  (target "chocolate_jello")
  (kind object)
  (destination "")
  (relation "")
  (value "")
  (qualifier ""))
```

Interpretación del campo `destination`. El campo `destination` representa el destino o destinatario asociado a un objetivo. Su significado depende de la operación indicada en `name`: en una entrega identifica al receptor; en una colocación, el lugar o soporte donde se depositará el objeto; y en una tarea de guía, la ubicación de destino. Por tanto, su presencia no implica por sí sola una acción de navegación. Las reglas determinan las acciones necesarias a partir del objetivo, sus argumentos y el estado simbólico.

Este campo se distingue de `target`, que conserva el argumento principal del objetivo. Por ejemplo, en `deliver(fork, to=me)`, `target` identifica el objeto y `destination` al destinatario. En cambio, para `go(kitchen)`, la ubicación se almacena directamente en `target` y `destination` permanece vacío.

Los espacios de los argumentos se convierten en guiones bajos para construir los identificadores de ejecución. Los valores de cadena se escapan antes de insertarse en CLIPS, mientras que el nombre y el tipo se representan como símbolos.

La clasificación de entidades utiliza la secuencia completa. Los destinatarios de entrega, los participantes de guía, seguimiento y saludo, los atributos visuales y la capitalización de los nombres aportan indicios para interpretar una búsqueda como búsqueda de persona. El campo `kind` del hecho se calcula mediante estas reglas de contexto; no se copia directamente del argumento homónimo de la predicción. Por ejemplo, `go` y `place` se clasifican como ubicación, `take` y `deliver` como objeto, y las operaciones de información como `info`. Esta clasificación cumple una función operativa en el adaptador y no redefine el vocabulario semántico.

`ClipsGoalFactBuilder` conserva los destinos indicados por `to`, `at`, `on` o `in`, en ese orden de prioridad. También admite el segundo argumento posicional para entrega, guía, seguimiento y colocación. La relación principal selecciona el primer campo disponible entre destino, gesto, postura, vestimenta y propiedad. El campo `qualifier` conserva el primer atributo visual, con prioridad de gesto, postura, vestimenta y propiedad, en la forma `clave:valor`. La representación no codifica una conjunción de varios calificadores visuales.

Además de la normalización léxica, el adaptador aplica una reparación específica cuando recibe una navegación cuyo argumento corresponde a una persona. Ese objetivo se transforma en búsqueda. Si inmediatamente después aparece un seguimiento de la misma persona con destino, se adelanta el destino como navegación, se añade la búsqueda y se conserva el seguimiento sin destino. Esta operación modifica la interpretación de la secuencia y debe registrarse como posprocesamiento, sin atribuir la corrección al modelo.

Para mantener la notación del capítulo anterior, la salida del intérprete conserva el símbolo $G_{\text{pred}}^{\text{Qwen}}$. Se denomina Adaptar a la transformación del adaptador, que comprende el análisis, las reparaciones, la clasificación y la serialización. El recorrido implementado queda expresado como:

$$P^{\text{Qwen–CLIPS}} = \text{CLIPS}\Big(\text{Adaptar}\Big(G_{\text{pred}}^{\text{Qwen}}\Big)\Big). \tag{5.1}$$

La calidad de la interpretación se calcula sobre la predicción anterior a esta transformación. La evaluación del sistema completo considera también los hechos y el plan que resultan de ella.

La auditoría fuera de línea utiliza `goal_to_clips_fact` para construir el mismo tipo de hecho, pero emplea el análisis del esquema semántico y no reproduce todas las heurísticas y reparaciones del nodo ROS 2. En consecuencia, compartir la plantilla

`gpsr-goal` no garantiza equivalencia entre ambos recorridos. La comprobación integrada debe conservar la transformación que efectivamente recibió CLIPS.

5.4.2. Representación del estado simbólico

La planificación comienza con un estado que representa ubicación desconocida y ausencia de un objeto sostenido:

```
(current-state (location "unknown") (holding "nothing"))
```

Las reglas actualizan este estado al añadir acciones al plan. Los valores representan efectos previstos, no confirmaciones físicas. Por ejemplo, la incorporación de una adquisición modifica el objeto esperado en posesión del robot. La colocación exige que coincida con el objeto solicitado; las reglas de entrega no imponen esa misma igualdad como precondición general. La validación de la secuencia de objetivos y las condiciones de expansión son, por tanto, controles complementarios.

Otros hechos conservan referencias a personas, el último contexto de interacción, análisis visuales y el estado del foco. Las referencias `guest_N` y `customer_N` permiten reutilizar una entidad durante distintas subtareas. Los objetos se identifican mediante nombres normalizados e índices, como `chocolate_jello_1`.

La referencia `guest_last` se utiliza para una persona capturada por `find_person`, mientras que `customer_N` identifica una búsqueda por atributo realizada con `find_pose`. Una persona nombrada puede requerir búsqueda anónima, solicitud del nombre y escucha mediante `guest_name_last`. La asociación efectiva de estos identificadores con detecciones y respuestas corresponde al ejecutor.

5.4.3. Reglas de expansión

Las reglas se agrupan en `goals_planning.clp`. Una regla de procesamiento consume un objetivo únicamente cuando no existe otro pendiente con un paso menor. Después de expandirlo, retira el hecho correspondiente y añade las acciones de la subtarea.

La navegación a una ubicación distinta de la registrada produce una solicitud `go_to` y una confirmación hablada. Si ambas ubicaciones coinciden, el objetivo se consume sin repetir el desplazamiento. La regla siguiente muestra esta decisión y la actualización del estado:

```
(defrule process-go
  (declare (salience 200))
  (planning-ready)
  ?g <- (gpsr-goal (step ?step) (name go) (target ?dest))
  (not (gpsr-goal (step ?prev&:(< ?prev ?step))))
  ?st <- (current-state (location ?loc) (holding ?held))
  =>
  (retract ?g)
  (if (neq ?loc ?dest) then
    (append-say (str-cat "I will navigate to " ?dest))
    (append-action go_to ?dest)
    (append-action say (str-cat "I have arrived at " ?dest))
    (retract ?st)
    (assert (current-state (location ?dest) (holding ?held)))))
```

La búsqueda de objetos genera `find_object` o una operación de análisis cuando se utiliza una propiedad admitida. Las búsquedas de personas recurren a `find_person` o `find_pose` y conservan el contexto necesario para la interacción posterior.

La manipulación utiliza `take`; la entrega incorpora aproximación y `give`; la colocación puede incluir navegación al destino, `find_space` y `drop`. La selección concreta depende de los argumentos y del estado simbólico. Las acciones de diálogo y de control del foco complementan estas operaciones.

Para la colocación se distinguen puntos de navegación y superficies genéricas. Un destino como `storage_rack` puede añadir navegación, mientras que `table`, `shelf` o `cabinet` se tratan como soportes en el lugar actual. La función siguiente incorpora la búsqueda de espacio y la liberación del objeto; el suelo constituye una excepción que utiliza directamente el argumento `floor`:

```
(deffunction append-drop-sequence (?obj ?destination)
  (bind ?surface (drop-surface ?destination))
  (if (eq ?surface "floor") then
    (append-action drop "floor")
    else
    (append-action find_space ?surface)
    (append-action drop (object-id ?obj))))
```

En este fragmento, `drop` identifica la acción del ejecutor que materializa el objetivo semántico `place`. La búsqueda de espacio selecciona una clase de soporte, como mesa, estante o contenedor, y la liberación utiliza el identificador del objeto.

Tras la colocación, la regla `process-place` asigna a `location` el valor de `destination`, incluso cuando este representa un soporte genérico y no se generó navegación. Por ello,

el estado resultante puede mezclar referencias a ubicaciones y a soportes; esta limitación debe considerarse al interpretar las acciones posteriores, sin asumir que hubo un desplazamiento físico.

La guía genera una instrucción hablada, navegación al destino y cierre de la interacción. El seguimiento recupera o busca a la persona, incorpora aproximación cuando corresponde y emite mediante `say` las claves `instruct_follow_only` o `instruct_follow_to_<destino>`. Esta regla no emite una acción explícita de seguimiento ni actualiza la ubicación al destino. Su expansión representa el protocolo de interacción; la realización física del seguimiento depende del tratamiento de esas claves en el ejecutor y debe verificarse por separado.

Las propiedades de tamaño se traducen a consultas de análisis de objetos grandes o pequeños. La implementación también utiliza el tamaño como aproximación para las solicitudes de mayor o menor peso y menor grosor. Esta correspondencia permite producir una consulta visual, pero no constituye una medición de peso o grosor ni demuestra la satisfacción de esas propiedades. Estos casos requieren una interpretación diferenciada en la evaluación.

El conteo genera `analyze_objects`, incorpora retorno al punto de instrucción cuando corresponde y emite la respuesta `count_category`. Para guardar un nombre se solicita y escucha `guest_name_last`; para atributos de persona se utilizan operaciones de análisis o búsqueda por postura. La información adquirida se comunica mediante las claves de respuesta de `tell`. La respuesta a preguntas combina `listen` y `say` con la clave `answer_question`, y el saludo utiliza la persona referenciada y la clave `greet`.

5.4.4. Generación de acciones

Las reglas construyen hechos intermedios `action-step`, que posteriormente se convierten en `ros-message`. Los anuncios generales utilizan numeración desde 1 y las acciones operativas desde 1001. Al ordenar la salida, se presenta primero el anuncio de las tareas y después las operaciones del plan.

Para la orden utilizada como ejemplo, las reglas generan la siguiente secuencia de acciones de navegación, búsqueda, adquisición y entrega. Se omiten los anuncios y las frases de confirmación para destacar cómo se expanden los cuatro objetivos semánticos.

```
go_to       bed
find_object chocolate_jello_1
```

```
take        chocolate_jello_1
go_to       instruction_point
approach    host_1
give        chocolate_jello_1
```

El objetivo de entrega al operador se expande en retorno al punto de instrucción, aproximación y entrega física. Esta secuencia es una ilustración de la expansión; el plan completo incluye también las acciones de comunicación y finalización establecidas por las reglas.

5.4.5. Condición de finalización

Cuando no quedan objetivos pendientes, las reglas añaden las acciones de cierre: desactivación del foco si continúa activo, retorno a `instruction_point` cuando corresponde y mensaje de tarea completada. La construcción concluye con la nota `GPSR_DONE`.

La finalización se distribuye en tres reglas. `add-final-return` incorpora el cierre y el hecho `final-steps-added`; `serialize-action-step` convierte las acciones en mensajes una vez presente ese hecho; finalmente, `goals-finish` emite la nota de cierre cuando no quedan hechos `action-step` por serializar. Sus prioridades son 20, 5 y 1, respectivamente.

El componente `PlanExtractor` construye la representación textual a partir de los mensajes y las notas del plan. El nodo publica esa representación como una cadena y la añade a `plan_execution.log`. El registro se vacía al iniciar el nodo y acumula los planes publicados durante la sesión.

La comprobación que realiza el nodo después de ejecutar el motor consiste en buscar hechos `ros-message`; cuando existen, solicita la extracción y publica si el resultado no está vacío. Este criterio del nodo se distingue de las condiciones de cierre implementadas en las reglas. El nodo no comprueba explícitamente `GPSR_DONE`, ausencia de objetivos pendientes o ausencia de mensajes de objetivo no soportado antes de solicitar la extracción.

La regla de objetivo no soportado consume el hecho y añade una explicación hablada. Por ello, la ausencia de objetivos pendientes y la marca de cierre pueden coexistir con una incapacidad de expansión. La comprobación funcional debe examinar también el contenido del plan, incluidas las respuestas de manipulación no disponible. Las marcas de inicio y cierre describen la generación del plan y no confirman su ejecución física.

5.5. Integración Qwen–CLIPS con Justina

5.5.1. Componentes ROS 2

La integración se organiza mediante el nodo semántico, el adaptador de objetivos y el nodo que administra CLIPS. El primero solicita la inferencia Qwen y proporciona el objeto con `goals`; el adaptador construye los hechos y coordina las solicitudes de inserción y planificación. El nodo `clips_node.py` expone los servicios del motor y publica el plan resultante.

El nodo CLIPS delega la interacción con el entorno simbólico en `ClipsEngine`, la carga de reglas en `RuleLoader` y la extracción de la salida en `PlanExtractor`. El componente `PlanLogger` registra los planes publicados. Esta separación mantiene las reglas de expansión en la base CLIPS y concentra en el nodo la comunicación y coordinación de tareas.

Las solicitudes recibidas por los servicios se convierten en tareas que se procesan secuencialmente mediante una cola y un trabajador dedicado. Los callbacks añaden el trabajo a la cola y responden sin esperar a que termine. La ejecución posterior del motor y la publicación ocurren en el planeador.

El nodo declara el parámetro `plan_name`, cuyo valor predeterminado es `default`, y lo entrega al cargador de reglas. El archivo `planning.launch.xml`, del paquete `planning_tools`, agrupa los lanzamientos de CLIPS y del parser semántico bajo el espacio de nombres `/planning`. La invocación utilizada selecciona `plan_name:=goals` y propaga ese argumento al lanzamiento de CLIPS.

El lanzamiento mantiene `qwen_response_format=goals`, `use_goals_to_clips=true` y `use_entity_extractor=true`. También propaga al parser un tiempo de espera Qwen de 5.0 segundos. Este valor sustituye los 15 segundos predeterminados del lanzamiento semántico incluido.

El archivo `semantic_parser.launch.xml` añade el espacio de nombres `semantic_parser` y configura el servicio del nodo Qwen con el nombre relativo `semantic_parser`. Su composición con el lanzamiento superior produce la ruta utilizada por el programa de pruebas:

```
/planning/semantic_parser/semantic_parser
```

Los argumentos `qwen_url` y `qwen_api_key` llegan vacíos en la invocación utilizada. El nodo resuelve cada valor consultando primero el parámetro ROS; si está vacío, utiliza

QWEN_SERVER_URL o QWEN_API_KEY, respectivamente. Si la variable de URL no existe, utiliza http://127.0.0.1:8008; si no existe la variable de clave, emplea una cadena vacía. En el despliegue se utilizan las variables de entorno configuradas en la sesión de terminal del usuario.

El nodo semántico proporciona el servicio mediante el tipo Parser y delega la solicitud HTTP en QwenHTTPClient. Utiliza un grupo de callbacks reentrante y un ejecutor de dos hilos. También ofrece un servicio de recepción de órdenes desde el componente de escucha; las pruebas por consola utilizan directamente la entrada textual.

Los servicios y el tópico del nodo CLIPS tienen nombres relativos. El espacio de nombres del lanzamiento los hace corresponder con las rutas bajo /planning utilizadas por el adaptador y por el programa de pruebas. El archivo superior no introduce remapeos explícitos y delega la creación de los nodos en los lanzamientos incluidos.

El adaptador mantiene sus clientes de servicios en un nodo auxiliar. El ejecutor SingleThreadedExecutor funciona en un hilo dedicado y procesa las respuestas mientras el callback de conversión espera la aceptación de sus solicitudes dependientes.

5.5.2. Flujo de comunicación

El programa qwen_to_plan.py recibe la orden como argumento posicional y la envía al servicio /planning/semantic_parser/semantic_parser, de tipo Parser del paquete semantic_parser_interfaces. La instrucción se coloca en request.command y el resultado se recupera de response.response. Con el formato goals, el nodo semántico devuelve un objeto JSON con input y goals; el primero conserva la orden normalizada recibida del cliente o, si no está disponible, la orden enviada. Si la solicitud HTTP falla, el nodo devuelve un objeto con status=aborted, la entrada y la causa del fallo. El programa comprueba que el texto recibido sea un objeto JSON con una lista goals en el nivel superior y lo envía al servicio de conversión. El adaptador construye los hechos y solicita primero su inserción por lote y después el inicio de planificación. Para una configuración del nodo CLIPS bajo /planning, sin remapeos adicionales de estas interfaces, las rutas son las siguientes:

```
/planning/semantic_parser/semantic_parser
/planning/goals_to_clips/convert
/planning/clips/assert_facts_batch
/planning/clips/start_planning
```

`/planning/clips/results`

La conversión utiliza `clips_interfaces/srv/String`: recibe el JSON en `request.data` y devuelve un mensaje de estado en `response.data`. La inserción por lote utiliza el tipo `StringArray`, del mismo paquete. El inicio de planificación utiliza `std_srvs/srv/Trigger`.

Al recibir un lote, el nodo encola una tarea que borra su copia del plan actual, reinicia el entorno mediante `reset`, inserta los hechos recibidos y ejecuta el motor. La solicitud de inicio encola otra tarea que borra la copia del plan, inserta el disparador `(start (name action-planning))` y vuelve a ejecutar el motor. El reinicio del entorno se realiza en la tarea de inserción por lote; la tarea de inicio no reinicia el entorno y conserva los hechos cargados para la planificación.

El servicio por lote devuelve `successful=True` después de encolar la tarea. El servicio de inicio devuelve `success=True` y el mensaje `Planning started.` en ese mismo punto del procedimiento. Estas respuestas confirman la aceptación de las solicitudes, no la terminación de la inserción ni de la planificación. Si una tarea falla posteriormente, el trabajador escribe el error en el registro ROS; el callback que ya respondió no comunica ese fallo mediante su respuesta original.

El adaptador espera hasta tres segundos por la disponibilidad de cada servicio dependiente y hasta cinco segundos por su respuesta. Rechaza una lista vacía y devuelve `OK` cuando ambas solicitudes reciben respuestas favorables. Estos límites corresponden a la comunicación de cada operación y no establecen un tiempo máximo global de generación del plan.

Después de ejecutar el motor, el nodo busca hechos `ros-message`. Si existen, solicita su extracción y publica la salida no vacía mediante `std_msgs/msg/String`, con una instrucción por línea. La obtención del plan se verifica sobre esa publicación y su contenido, por separado del estado devuelto por los servicios. El tópico publicado tiene el nombre relativo `clips/results`.

5.5.3. Recepción del plan y alcance de las pruebas

En el recorrido de pruebas utilizado, el consumidor del plan es `qwen_to_plan.py`, ejecutado mediante `ros2 run`. El programa se suscribe a `/planning/clips/results` e imprime la respuesta de Qwen, el estado de conversión y el plan recibido. Este recorrido termina en la recepción y presentación del plan; no envía sus pasos a los controladores del robot.

El programa utiliza por defecto diez segundos para cada espera de servicio o respuesta y cinco segundos para la espera del plan. Estos límites se distinguen de los del adaptador y del cliente HTTP. El callback de suscripción conserva el último texto recibido y la espera termina cuando ese texto no está vacío. La recepción confirma que existe una salida disponible; la validación del contenido del plan constituye una comprobación adicional.

El programa no incorpora un identificador que vincule cada mensaje del tópico con la orden enviada. La prueba funcional se interpreta por ello como un recorrido de una orden a la vez, sin atribuirle correlación entre solicitudes concurrentes. Tampoco verifica por sí misma la ejecución física o todas las condiciones semánticas del plan.

La publicación mantiene una interfaz disponible para un consumidor de ejecución, pero las comprobaciones aquí descritas se circunscriben a interpretación, conversión y generación simbólica de acciones.

Durante el desarrollo se utilizaron 48 comandos como prueba de funcionamiento del recorrido de integración. Esa comprobación es anterior al entrenamiento cuyo adaptador se identifica en la Sección 5.3.4, por lo que no se atribuye a ese artefacto. Al cierre de esta implementación, las pruebas detalladas del nuevo adaptador mediante HTTP y ROS 2 y la comparación con GPT aún no se han realizado. La comprobación automática de 50 muestras posterior al ajuste corresponde al recorrido fuera de línea.

5.6. Ejemplo integrado de funcionamiento de Qwen–CLIPS

Esta sección reúne los componentes anteriores mediante una orden de búsqueda, adquisición y entrega. Las figuras muestran las transformaciones de la instrucción y la expansión por reglas, mientras que la lista final permite consultar las acciones y los mensajes exactos del plan. Se considera una predicción que interpreta correctamente la orden, con la manipulación habilitada. El ejemplo es ilustrativo y no constituye una medición de precisión de Qwen ni una prueba de ejecución física.

5.6.1. De la instrucción a los hechos simbólicos

La Figura 5.2 parte de una instrucción que solicita desplazarse a `bed`, buscar un objeto, adquirirlo y entregarlo al operador. La recepción puede utilizar la inferencia

local o el servicio remoto descritos anteriormente. El recorrido contempla normalización, tokenización, generación, extracción del JSON y validación de los objetivos. La figura presenta la predicción considerada para el ejemplo, $G_{\text{pred}}^{\text{Qwen}}$.

De la instrucción a los hechos

Instrucción recibida
Navigate to the bed then find a chocolate jello
and take it and give it to me.

Objetivos: predicción ilustrativa de Qwen
go(bed)
find(chocolate jello, kind=object)
take(chocolate jello)
deliver(chocolate jello, to=me)

Adaptador de objetivos
Conservar el orden, normalizar nombres y asignar tipos

step	name	target	kind	destination
1	go	bed	location	""
2	find	chocolate_jello	object	""
3	take	chocolate_jello	object	""
4	deliver	chocolate_jello	object	me

En deliver: relation="to" y value="me". Los demás campos no mostrados están vacíos.

Figura 5.2: Transformaciones de la instrucción del ejemplo: orden, objetivos predichos y campos de los hechos `gpsr-goal`. La predicción de Qwen es ilustrativa. Cada fila de la tabla representa un hecho construido por el adaptador. Elaboración propia.

La referencia *it* se vincula con `chocolate jello`, y *me* se conserva como destinatario de la entrega. El adaptador normaliza los espacios como guiones bajos, asigna tipos operativos y conserva el orden mediante `step`. En este caso no se activa la reparación de navegaciones dirigidas a personas. El destino y la relación de entrega permiten seleccionar posteriormente la regla dirigida al operador.

Los hechos se envían mediante el servicio de inserción por lote. Tras su aceptación, el adaptador solicita el inicio de planificación mediante el disparador `(start (name`

`action-planning))`. La respuesta favorable de los servicios indica que la coordinación fue aceptada; el plan se obtiene mediante la expansión posterior.

5.6.2. Aplicación de reglas y evolución del estado

La regla `goals-initialize` establece una ubicación desconocida, ausencia de un objeto sostenido, los contadores y la nota `GPSR_START`. Desde este punto, cada regla de procesamiento exige que no exista otro objetivo pendiente con un paso menor. La Figura 5.3 se lee de arriba hacia abajo; cada fila relaciona una regla, sus acciones operativas principales y el estado resultante.

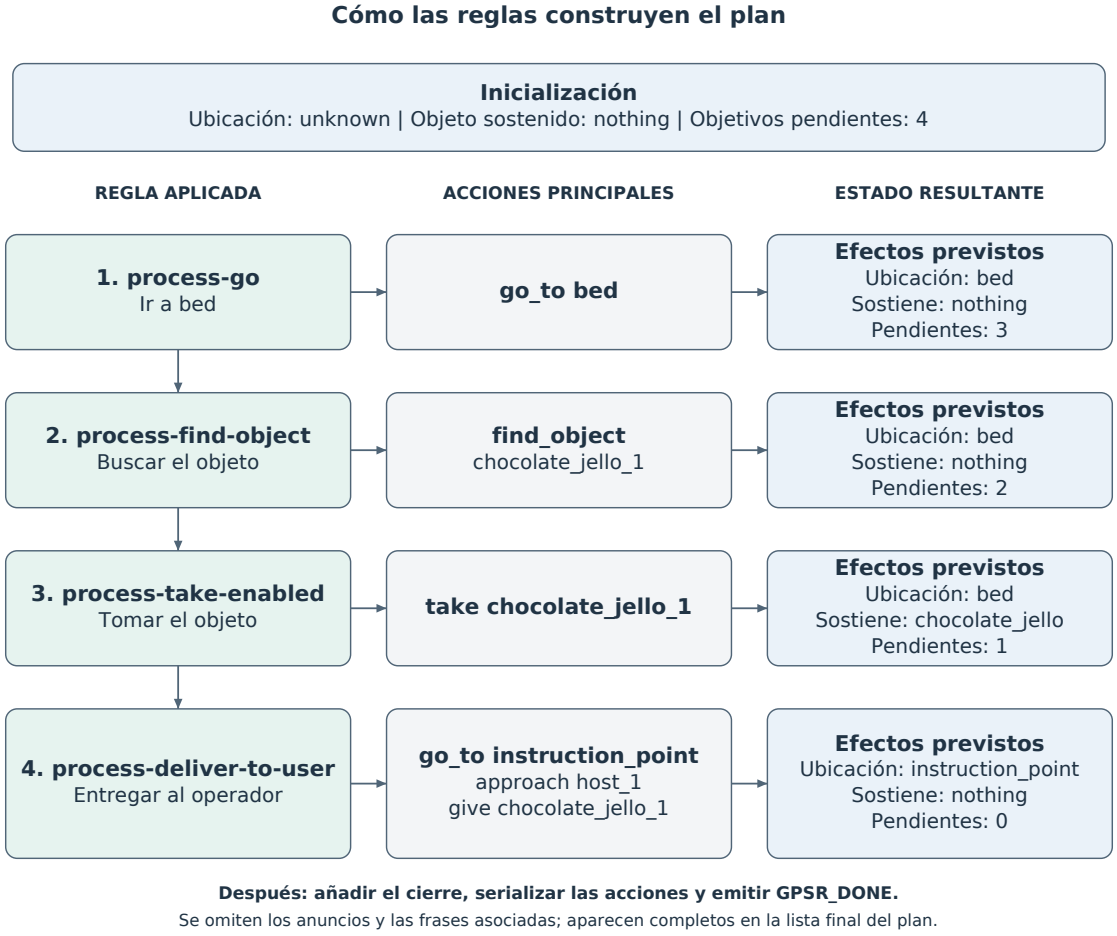


Figura 5.3: Expansión de los cuatro objetivos y evolución del estado simbólico. Se muestran las acciones operativas principales; los anuncios y las frases asociadas se conservan en el plan completo. Los cambios de estado representan efectos previstos durante la planificación. Elaboración propia.

La prioridad no permite adelantar una regla cuyas condiciones aún no se cumplen. Por ejemplo, la entrega tiene prioridad 190 y la adquisición habilitada, 181; sin embargo, la entrega debe esperar a que desaparezca el objetivo de adquisición con paso 3. El orden de los objetivos y la prioridad de las reglas cumplen funciones distintas.

La búsqueda mantiene la ubicación y el objeto sostenido; la adquisición modifica la posesión prevista. El estado utiliza `chocolate_jello`, mientras que las acciones utilizan el identificador operativo `chocolate_jello_1`. En la entrega, `me` se concreta mediante el retorno a `instruction_point` y la aproximación a `host_1`. Estos detalles operativos se incorporan mediante las reglas y no estaban expresados como objetivos independientes en la predicción.

5.6.3. Finalización y construcción de la salida

Cuando no quedan objetivos pendientes, `add-final-return` incorpora el cierre. La ubicación prevista ya es `instruction_point`, por lo que no se añade otro retorno. Tampoco existe foco activo que requiera desactivación. En este caso se agrega únicamente la frase `task completed`.

La regla `serialize-action-step` convierte cada acción `action-step` en un mensaje `ros-message`. Por ejemplo:

```
(action-step (step 1001) (action go_to) (arg "bed"))
```

se convierte en:

```
(ros-message
  (step 1001) (robot Justina) (action go_to) (params "bed"))
```

Cuando tampoco quedan acciones por serializar, `goals-finish` añade `GPSR_DONE`. El componente de extracción ordena los mensajes por `step`. Los anuncios se numeran desde uno y aparecen primero; el bloque numerado desde 1001 reúne las operaciones y las frases asociadas a ellas. Esta separación explica el orden de presentación del plan, aunque los anuncios y las acciones se hayan construido durante la expansión de los mismos objetivos.

5.6.4. Plan resultante

La expansión produce los siguientes pasos:

```
==Justina GPSR_START==
Paso 1: Justina say I will navigate to bed
Paso 2: Justina say I will search for chocolate_jello
Paso 3: Justina say I will pick up chocolate_jello
Paso 4: Justina say I will return to the instruction point
Paso 5: Justina say I will deliver chocolate_jello to you
Paso 1001: Justina go_to bed
Paso 1002: Justina say I have arrived at bed
Paso 1003: Justina find_object chocolate_jello_1
Paso 1004: Justina take chocolate_jello_1
Paso 1005: Justina say I have picked up chocolate_jello
Paso 1006: Justina go_to instruction_point
Paso 1007: Justina approach host_1
Paso 1008: Justina give chocolate_jello_1
Paso 1009: Justina say I have delivered chocolate_jello
Paso 1010: Justina say task completed
==Justina GPSR_DONE==
```

Los cuatro objetivos generan quince pasos: cinco anuncios iniciales y diez instrucciones posteriores. Estas últimas comprenden seis acciones de navegación, percepción y manipulación, y cuatro instrucciones habladas. Las marcas de inicio y cierre no se cuentan como pasos.

El plan queda disponible para su publicación y consumo por el ejecutor. Las frases de llegada, adquisición y entrega forman parte de las instrucciones generadas; no representan observaciones recibidas durante la planificación. La verificación de que el robot realizó físicamente cada operación corresponde a la etapa de ejecución.

5.6.5. Programa experimental Qwen–CLIPS

Para la comparación de planes se implementó un programa independiente del recorrido ROS 2 de Justina. El programa recibe la instrucción y el contexto del caso, obtiene los objetivos mediante Qwen, aplica la normalización y conversión correspondientes, y utiliza CLIPS para generar el plan. Registra la entrada, los objetivos, el plan y las incidencias de cada etapa. Las acciones producidas no se envían al ejecutor del robot.

Este programa se distingue de `qwen_to_plan.py`, que prueba la cadena integrada mediante servicios ROS 2. Las diferencias entre ambos recorridos, particularmente en la normalización, la conversión a hechos y la configuración de reglas, se documentan para delimitar el alcance de la comparación experimental.

5.7. Implementación del planificador GPT

El planificador directo se implementó como un programa independiente para pruebas por consola. Recibe una instrucción en lenguaje natural y solicita en una misma respuesta los objetivos semánticos y el plan de acciones. Esta implementación permite estudiar la interpretación y generación de planes bajo las convenciones operativas de Justina. El programa no utiliza CLIPS para construir el plan y no publica acciones al robot.

El componente se organiza en `planner_test_node.py`, un prompt estático, un esquema JSON y ejemplos del contrato. Su salida consiste en una representación estructurada del plan y un registro de la solicitud, que permiten analizar el comportamiento del modelo sin ejecutar físicamente las acciones.

El programa se mantuvo independiente porque el modelo genera los objetivos semánticos y el plan de acciones en una misma respuesta. Para obtener y analizar estas salidas no se requiere la transferencia entre un intérprete, un adaptador y un motor simbólico, como ocurre en Qwen–CLIPS. El cliente concentra la solicitud, la validación y el registro de resultados.

Esta organización responde al alcance de la comparación, centrado en la interpretación y generación de planes. Una integración destinada a ejecutar las acciones en Justina requeriría incorporar las interfaces de comunicación y los controles correspondientes.

5.7.1. Interfaz con la API

El programa utiliza el cliente de OpenAI y el método `responses.create`. La solicitud contiene un mensaje de sistema con las instrucciones del planificador y un mensaje de usuario con la orden. Cada invocación realiza una solicitud independiente y no conserva una conversación de varios turnos.

El modelo seleccionado para este componente es `gpt-5.6-terra`. El programa recibe su identificador mediante `--model` o la variable `OPENAI_MODEL`, sin fijar un modelo predeterminado en el código. El registro distingue el identificador solicitado del modelo informado en la respuesta, cuando el servicio proporciona este último.

La solicitud produjo una respuesta con estado `completed`, aceptada por el validador local. Esta comprobación acredita el recorrido de solicitud y procesamiento del ejemplo y se distingue del conjunto experimental de comparación.

El cliente configura un tiempo de espera de 60 segundos y desactiva los reintentos

automáticos. El límite predeterminado de salida es 8 192 tokens y puede modificarse mediante `--max-output-tokens`. La petición utiliza `store=False`; el programa conserva su propio registro local. No especifica temperatura, `top_p` ni un parámetro de esfuerzo de razonamiento en la solicitud.

5.7.2. System prompt

El prompt estático, identificado como versión 1.1, se elaboró en inglés. Define los catálogos de entidades, las formas canónicas de los doce objetivos semánticos, la resolución de referencias y las convenciones de normalización. También establece las dependencias entre búsqueda, adquisición, transporte e interacción.

Las instrucciones describen la expansión de cada objetivo en acciones y el mantenimiento de un estado previsto: ubicación, objeto sostenido, foco de interacción y referencias a personas. Especifican la separación entre anuncios y operaciones, la numeración de pasos y las condiciones de cierre. Los cambios de estado pertenecen a la construcción del plan y no representan observaciones del robot.

El modelo recibe, por tanto, las convenciones de expansión empleadas por el sistema simbólico. La comparación estudia la generación directa bajo estas instrucciones y se circunscribe a esa configuración. El prompt se conserva separado de la orden y su huella SHA-256 se registra en cada solicitud para identificar el contenido utilizado.

5.7.3. JSON Schema y Structured Outputs

La solicitud incorpora el esquema mediante `text.format`, con tipo `json_schema`, nombre `gpsr_direct_plan` y `strict=True`. Los seis campos del objeto raíz son obligatorios y no se admiten propiedades adicionales.

Tabla 5.2: Contrato estructurado del planificador GPT directo.

Campo	**Contenido**
`status`	`executable`, `clarification` o `rejected`.
`message`	Pregunta o explicación; vacío para planes ejecutables.
`goals`	Lista de expresiones de objetivos semánticos.
`plan`	Lista de pasos con `step`, `robot`, `action` y `params`.
`start_note`	`GPSR_START` o cadena vacía.
`end_note`	`GPSR_DONE` o cadena vacía.

Cada paso exige un número entero, el identificador `Justina`, una acción del repertorio de catorce acciones y una lista de cadenas como parámetros. El repertorio comprende navegación, comunicación, búsqueda, análisis, aproximación, control del foco y manipulación. El esquema controla estos tipos y valores admitidos; la coherencia entre `status` y los demás campos, la numeración y el número de parámetros se comprueban adicionalmente en el programa.

5.7.4. Generación de objetivos y planes

Para una instrucción ejecutable, la respuesta contiene $G_{\text{pred}}^{\text{GPT}}$ y P^{GPT}. Los objetivos permiten evaluar la interpretación frente a G_{ref}, mientras que los pasos permiten analizar la secuencia de acciones y su correspondencia con las convenciones del ejecutor. Ambos elementos proceden de la misma respuesta del modelo.

El programa convierte `response.output_text` mediante `json.loads`, conserva el resultado y lo somete a las comprobaciones locales. La respuesta se imprime como JSON y se añade al registro de ejecuciones. En este programa de pruebas no se realiza una conversión posterior a mensajes ROS ni una entrega al ejecutor.

Los ejemplos incluidos ilustran una instrucción de navegación, búsqueda, adquisición y colocación, una solicitud de aclaración y un rechazo de una tarea fuera del repertorio. Se utilizan para verificar el contrato local y se distinguen de las respuestas observadas en solicitudes reales al modelo.

Como ejemplo de una respuesta real, la instrucción *go to the kitchen and find an apple* produjo los objetivos:

```
{"goals": ["go(kitchen)", "find(apple, kind=object)"]}
```

El plan devuelto contiene siete pasos. La Tabla 5.3 presenta sus acciones y el único parámetro de cada paso; todos utilizan el identificador de robot `Justina`. La respuesta incluye también `status=executable`, mensaje vacío y las marcas `GPSR_START` y `GPSR_DONE`.

Los dos anuncios preceden al bloque operativo. El retorno al punto de instrucción y la frase final corresponden a las convenciones de cierre del prompt. El plan se conservó como salida de la generación; esta comprobación no incluyó su ejecución en el robot.

Tabla 5.3: Plan observado en la comprobación funcional del cliente GPT.

Paso	Acción	Parámetro
1	say	I will navigate to kitchen
2	say	I will search for apple
1001	go_to	kitchen
1002	say	I have arrived at kitchen
1003	find_object	apple_1
1004	go_to	instruction_point
1005	say	task completed

5.7.5. Tratamiento de aclaraciones y rechazos

El estado `clarification` representa una pregunta para resolver información faltante o ambigua. El estado `rejected` representa una tarea fuera del dominio o de las capacidades admitidas. En ambos casos se exige un mensaje no vacío y objetivos, plan y marcas de inicio y cierre vacíos. El programa conserva e imprime esa respuesta; no implementa automáticamente un diálogo posterior.

Estos estados del contrato se distinguen de una respuesta incompleta, un rechazo comunicado mediante el campo `refusal` de la API, un error de comunicación o una salida que no supera la validación. Las incidencias se registran como fallos de la solicitud o del procesamiento, conservando la respuesta recibida cuando está disponible.

5.7.6. Validación determinista

La función `validate_result` aplica primero el esquema mediante `Draft202012Validator`. Para una respuesta ejecutable exige objetivos y plan no vacíos, mensaje vacío y las marcas `GPSR_START` y `GPSR_DONE`. Las expresiones de objetivos se revisan con el validador semántico compartido y se restringen al vocabulario de la investigación.

El programa comprueba que los pasos estén ordenados, sin duplicados ni huecos dentro de cada bloque. Los anuncios se numeran desde 1 y deben utilizar `say`; las operaciones comienzan en 1001. Cada paso debe contener exactamente un parámetro no vacío y el bloque de operaciones debe terminar con `say task completed`. Para aclaraciones y rechazos verifica que el mensaje esté presente y los campos operativos permanezcan vacíos.

El indicador `valid` señala que la respuesta superó estas comprobaciones. Puede ser verdadero tanto para un plan como para una aclaración o rechazo bien formado.

No demuestra correspondencia semántica completa entre orden, objetivos y acciones ni satisfacción de todas las precondiciones del plan. Las verificaciones solicitadas al modelo dentro del prompt se distinguen de estos controles externos.

El programa incorpora un modo de validación local de los tres ejemplos del contrato. Las pruebas unitarias comprueban respuestas inconsistentes, numeración, parámetros, cierre, restricciones de objetivos y registro de errores. Para la interfaz con la API utilizan respuestas simuladas, por lo que esas pruebas no requieren solicitudes al servicio.

5.8. Reproducibilidad

5.8.1. Versiones de modelos

El modelo base se identifica como `Qwen/Qwen2.5-7B`. La configuración conservada del adaptador declara PEFT 0.18.1, `task_type=CAUSAL_LM`, rango 32, escala 64 y descarte 0.05, sobre las proyecciones de atención y del bloque de transformación descritas anteriormente. La caché del entorno de entrenamiento conserva una única revisión de este repositorio, identificada también por su referencia local `main`:

```
d149729398750b98c0af14eb82c78cfe92750796
```

Este identificador documenta la revisión local conservada del modelo base. Se recuperó de la caché al registrar la configuración: el script de carga no especifica `revision` y ese campo es nulo en la configuración del adaptador. Por tanto, la identificación recuperada se distingue de una selección de revisión fijada explícitamente en el código durante el entrenamiento.

El tokenizador guardado declara la clase `Qwen2Tokenizer` y conserva la plantilla conversacional utilizada para construir las entradas. Los archivos del adaptador, tokenizador y argumentos de entrenamiento constituyen registros complementarios. Los estados conservados documentan los checkpoints de 5 000 y 9 000 actualizaciones. El estado final registra:

```
global_step = 9000
epoch = 2.0
best_global_step = 5000
best_model_checkpoint = "nl2cd_qwen7b/checkpoint-5000"
best_metric = 0.23214000463485718
```

Los argumentos conservados confirman la selección por `eval_loss`, con evaluación y guardado cada 250 pasos. El guardado posterior al entrenamiento exporta el adaptador restaurado, según el procedimiento de la Sección 5.3.4. El comando de despliegue inicia el servidor desde `Qwen2.5-7B`, dentro del directorio personal del usuario, y utiliza el adaptador y el tokenizador de `nl2cd_qwen7b`, según la ruta descrita en la Sección 5.3.6.

Para el componente GPT se utilizó `gpt-5.6-terra`. El registro de la solicitud completada conserva ese identificador tanto en la petición como en la respuesta del servicio. El prompt versión 1.1 y el esquema coinciden con las huellas registradas en ambas solicitudes conservadas. La versión nominal del prompt se complementa con su huella para distinguir modificaciones de contenido.

5.8.2. Configuraciones experimentales

La Tabla 5.4 reúne las versiones obtenidas del entorno utilizado para entrenar Qwen. La versión CUDA indicada corresponde a la compilación de PyTorch. Este inventario se conserva junto con los argumentos, la configuración del adaptador y los estados de entrenamiento.

Tabla 5.4: Versiones del entorno de software utilizado para el entrenamiento.

Componente	**Versión**
Ubuntu	24.04.5 LTS
Python	3.12.3
ROS 2	Jazzy
Controlador NVIDIA	580.178.04
PyTorch	2.9.0+cu130
CUDA de PyTorch	13.0
Transformers	4.51.3
PEFT	0.18.1
bitsandbytes	0.49.2
Accelerate	1.13.0
Datasets	4.8.4
Tokenizers	0.21.4
clipspy	1.0.5
Cliente OpenAI	2.8.1
jsonschema	4.25.1
NumPy	1.26.4

El archivo `dataset_gpsr.jsonl` auditado es el utilizado en el entrenamiento, contiene 40 000 registros y se identifica mediante la siguiente huella SHA-256:

0695f400370b93abbe97fc403a2acaa9f2d6bc8241667788bd466c6ad7c1c406

La partición implementada utiliza una fracción de validación de 0.1 y semilla 42. La identidad del archivo, el orden de sus registros y la versión de la biblioteca de partición forman parte de la configuración necesaria para reconstruir la división.

La inferencia Qwen del módulo compartido configura hasta 128 tokens nuevos, generación sin muestreo y penalización de repetición de 1.05. El normalizador utiliza los catálogos de `CompetitionTemplate` y los umbrales descritos en la sección de inferencia. Su entrada normalizada se conserva separada de la orden original. La ubicación del proceso, local o remota, y las transformaciones posteriores del adaptador forman parte de la condición del sistema completo.

El servicio se inicia desde el directorio de trabajo del modelo mediante el siguiente comando. La clave se proporciona como variable de entorno y su valor no forma parte del registro de configuración:

```
cd ~/Qwen2.5-7B/
python3 server.py --host 100.102.21.62 --port 8008 \
  --api-key "$QWEN_API_KEY"
```

El lanzamiento del sistema de planificación y el programa de pruebas se ejecutan en terminales independientes:

```
ros2 launch planning_tools planning.launch.xml plan_name:=goals
ros2 run planning_tools qwen_to_plan.py \
  "go to the kitchen and find an apple"
```

La orden mostrada ilustra el argumento posicional requerido por el programa de pruebas. El archivo de lanzamiento fija el espacio de nombres `/planning` y conserva el formato de respuesta `goals`. Las rutas de servicios y del tópico coinciden con los valores predeterminados del programa.

La URL del servidor y la clave se configuran mediante las variables `QWEN_SERVER_URL` y `QWEN_API_KEY`, definidas en la configuración de la terminal e incorporadas al entorno del proceso ROS. El nodo utiliza esas variables cuando los parámetros ROS de conexión están vacíos. Un parámetro ROS no vacío tiene precedencia sobre su variable de entorno. Cuando no existe una URL en los parámetros ni en el entorno, el nodo utiliza `http://127.0.0.1:8008`, que se refiere al equipo donde se ejecuta el cliente.

El valor de la URL debe identificar la dirección de escucha del servidor en el despliegue utilizado. La clave se conserva fuera del texto de la tesis. La resolución de parámetros, variables y valores predeterminados queda definida por el código del nodo semántico. El tiempo de espera HTTP efectivo de la invocación descrita es de cinco segundos, propagado desde el lanzamiento superior; se distingue de los diez segundos de espera de servicios del programa de pruebas y de sus cinco segundos de espera del plan.

En GPT, el programa permite configurar el modelo, el límite de tokens y el archivo de resultados. El límite predeterminado es 8 192 tokens, el tiempo de espera es de 60 segundos y los reintentos automáticos están desactivados. Los valores efectivamente enviados de modelo y límite de tokens quedan asociados a cada solicitud. La comprobación funcional utilizó el límite de 8 192 tokens. Aunque el cliente no envía temperatura, `top_p` ni esfuerzo de razonamiento, la respuesta conservada informa los valores 1.0, 0.98 y `medium`, respectivamente. Se registran como metadatos de esa respuesta, sin atribuirles una configuración explícita del cliente. Las condiciones de la comparación se establecen en el capítulo siguiente a partir de estas configuraciones.

5.8.3. Registro de ejecuciones

El entrenamiento conserva los argumentos, las configuraciones del adaptador y tokenizador, los estados de los checkpoints y la salida de consola. Esta última registra 27 772.8461 segundos para `train_runtime`, la finalización de las dos épocas y el paso al guardado del adaptador. Ese tiempo corresponde a la ejecución del entrenador; no incluye las comprobaciones generativas que el script realiza después de `trainer.train()`.

La consola conserva también la comprobación de 50 muestras descrita en la Sección 5.3.4 y las respuestas de cinco órdenes adicionales. Durante el cierre del validador CLIPS aparece el aviso `ENVRNMNT8` de memoria no liberada por completo. Se conserva como incidencia del entorno junto con los resultados reportados, sin describir la ejecución como libre de advertencias. Estos registros documentan el entrenamiento y su comprobación preliminar; las pruebas comparativas detalladas permanecen pendientes.

El programa GPT añade un objeto JSON por solicitud a `runs/results.jsonl`, o a la ruta indicada mediante `--output`. El registro contiene la fecha UTC, la orden, el modelo solicitado, las huellas del prompt y esquema, los parámetros explícitos, el indicador de validación y la latencia. Cuando se obtiene una respuesta, conserva su representación completa y, si el texto pudo convertirse a JSON, el resultado estructurado. Los datos

de uso de tokens y el modelo informado quedan dentro de la respuesta completa cuando el servicio los proporciona.

El registro de comprobación funcional contiene solicitudes para la orden de navegación a la cocina y búsqueda de una manzana. Se conserva una respuesta completada y validada, con un tiempo de 4.4844 segundos. El servicio informa 4 251 tokens de entrada y 283 de salida, para un total de 4 534; el desglose de salida incluye 76 tokens de razonamiento. Estos datos documentan solicitudes individuales y no se interpretan como una estimación de precisión, tasa de fallos o latencia media del sistema.

La huella del prompt se calcula sobre el texto leído. La del esquema se calcula sobre su serialización JSON con claves ordenadas; por ello, no debe confundirse con una huella de los bytes originales del archivo. Las solicitudes fallidas también producen un registro con el error y el tiempo observado. La latencia se mide desde antes de la llamada hasta después del procesamiento y la validación, y excluye la escritura final del registro. Incluye comunicación y espera del servicio, por lo que no equivale únicamente al tiempo interno de generación del modelo.

El nodo CLIPS añade cada plan publicado a `plan_execution.log`. Al iniciar el nodo, el componente `PlanLogger` vacía el archivo; por tanto, su contenido corresponde a la sesión en curso y debe conservarse antes de reiniciar cuando se utiliza como evidencia. Este archivo contiene el texto de los planes, sin asociar por sí mismo un identificador de solicitud, la orden original o los tiempos de cada etapa. Los errores del planeador se envían al registro ROS y no se escriben mediante `PlanLogger`. El servidor Qwen devuelve `elapsed_ms` para el procesamiento de la traducción; esta medición no comprende el recorrido completo observado por el cliente ni las etapas posteriores de adaptación y expansión simbólica.

El módulo de inferencia incorpora `ResourceTimer` para diagnosticar tiempo transcurrido, tiempo de CPU, máximo de RAM del proceso y memoria de GPU asignada y reservada. Las mediciones temporales sincronizan CUDA en los límites del bloque observado. El porcentaje de CPU puede superar el 100 % cuando participan varios hilos y el máximo de RAM corresponde a la vida del proceso. Estos diagnósticos no se incorporan automáticamente a cada respuesta HTTP.

Los registros disponibles describen etapas y límites temporales distintos. El archivo de planes, el tiempo de traducción y los diagnósticos del proceso no constituyen por separado una medición completa de Qwen–CLIPS. La evaluación debe asociar cada orden con sus objetivos, transformaciones, plan e incidencias, y delimitar de forma

común las mediciones utilizadas en la comparación.

Capítulo 6

Validación y experimentación

Este capítulo establece el procedimiento para comparar las arquitecturas Qwen–CLIPS y GPT directo descritas en los capítulos anteriores. La evaluación se centra en la transformación de instrucciones a objetivos semánticos y planes de acciones del dominio GPSR. Se examinarán la precisión de la interpretación, el cumplimiento de las tareas mediante el plan generado y el rendimiento computacional de cada configuración.

El protocolo distingue el desarrollo del instrumento de evaluación de su aplicación final. Las pruebas preliminares permitirán corregir los registros, precisar las referencias y fijar la configuración experimental. Posteriormente, la comparación se realizará sobre un conjunto reservado que no se utilizará para ajustar los sistemas. Los resultados de esa comparación se presentarán en el capítulo siguiente.

6.1. Diseño experimental

El diseño será pareado: cada orden del conjunto final se presentará a ambas arquitecturas bajo condiciones iniciales y criterios de evaluación comunes. La arquitectura completa será el factor principal de comparación. Cambian simultáneamente el modelo, su especialización y el mecanismo de generación del plan; por ello, las diferencias no se atribuirán exclusivamente al empleo de reglas simbólicas.

La comparación responderá a cuatro preguntas: qué tan fielmente se recupera la intención de la orden; qué proporción de planes satisface sus subtareas y restricciones; cómo cambia ese comportamiento ante reformulaciones y mayor composición; y cuál es el tiempo y consumo observable requerido para producir la respuesta. La evaluación por capacidades se apoya en el enfoque de pruebas de comportamiento de CheckList, que

complementa las medidas agregadas con casos dirigidos a comportamientos lingüísticos específicos (**?**). La distinción entre una respuesta plausible y un plan verificable se relaciona con la evaluación de acciones y cambios de estado abordada por PlanBench (**?**). En esta tesis se desarrollará una rúbrica propia para el contrato operativo de Justina.

6.1.1. Conjuntos de desarrollo y prueba

Se distinguirán tres usos de los datos. El corpus de 40 000 órdenes empleado en la especialización de Qwen, dividido en entrenamiento y validación, conserva la función descrita en el capítulo de implementación. El conjunto de desarrollo permitirá preparar el experimento. El conjunto de prueba final se reservará para la comparación con las configuraciones ya fijadas. Las comprobaciones funcionales previas y las predicciones efectuadas sobre la partición de validación no sustituirán este último conjunto.

El conjunto de desarrollo preparado contiene 60 órdenes, distribuidas como se muestra en la Tabla 6.1. Las tres formulaciones de una misma intención se mantendrán agrupadas mediante un identificador común.

Tabla 6.1: Organización del conjunto de desarrollo propuesto.

Bloque	Órdenes	Intenciones
Funcional y lingüístico: formulación canónica, paráfrasis y ruido leve	48	16
Límites de representación y composición	6	6
Ambigüedad, contradicción y tareas fuera del repertorio	6	6
Total	60	28

Estos 60 casos no constituyen 60 intenciones independientes ni acreditan cobertura exhaustiva de las familias del generador. Cincuenta y cuatro fichas contienen objetivos propuestos y seis describen conductas de aclaración o rechazo. La existencia de objetivos no implica que todos los casos pertenezcan al bloque común resoluble: las capacidades cuyo contrato sea incompleto, como determinadas formas de seguimiento o identificación, requieren revisión.

Cada ficha incluirá la orden, los objetivos de referencia G_{ref}, las subtareas requeridas, las restricciones, los comportamientos inadmisibles y las equivalencias aceptables. Las

referencias se revisarán a partir de la intención expresada y del contrato de acciones, antes de observar las salidas de los modelos.

Se auditarán las coincidencias con el corpus completo mediante igualdad literal, limpieza básica y aplicación del normalizador utilizado por Qwen. Una coincidencia con las 40 000 filas no identifica por sí misma la partición a la que perteneció la fila. Asimismo, ausencia de coincidencias bajo estas transformaciones no demuestra novedad semántica, independencia de plantilla ni ausencia en el preentrenamiento de los modelos.

En desarrollo podrán conservarse coincidencias identificadas como casos conocidos, porque su función es diagnosticar el procedimiento. No se modificarán indefinidamente las órdenes sólo para obtener un contador de coincidencias igual a cero. Para la prueba final se excluirán los duplicados detectados y las instancias de intención utilizadas durante el desarrollo, junto con sus reformulaciones. Se podrán conservar las mismas clases de tarea para evaluar generalización dentro del dominio; esto no se describirá como evaluación de familias nunca vistas.

El tamaño definitivo del conjunto final se establecerá después del piloto y antes de ejecutar la comparación. Como propuesta de cobertura, se consideran diez intenciones por familia admisible y tres formulaciones por intención. Si F es el número de familias incluidas, se obtendrían $10F$ intenciones y $30F$ órdenes. Incluir las 29 familias del generador produciría 290 intenciones y 870 órdenes, pero esta cifra queda condicionada a la revisión de su correspondencia con las capacidades y al presupuesto de ejecución. No representa un tamaño calculado mediante potencia estadística.

La admisión al bloque principal dependerá de que la tarea esté suficientemente especificada y pueda evaluarse mediante el contrato declarado, no de que ambos sistemas consigan resolverla. Una arquitectura que falle en una tarea admisible conservará ese fallo. Las tareas cuyo significado o verificación no pueda establecerse se identificarán antes de la comparación y se estudiarán como diagnósticos. Los casos ambiguos y fuera de dominio formarán un bloque separado, con cantidad y distribución fijadas también antes de las ejecuciones.

6.1.2. Procedimiento experimental

El procedimiento se organizará en preparación, piloto, fijación de la configuración y evaluación final. Durante la preparación se revisarán las fichas y se construirá un registro común para ambas arquitecturas. Los validadores de formato existentes se reutilizarán donde corresponda, pero se añadirán verificaciones de contenido basadas

en las subtareas y restricciones de cada caso. Los registros actuales de consola y planes no constituyen por sí solos este instrumento completo.

El evaluador se comprobará con ejemplos correctos y con errores introducidos deliberadamente: omitir una adquisición, cambiar el destinatario, eliminar un atributo o truncar la secuencia. Esta comprobación permitirá detectar un evaluador que acepte cualquier plan bien formado o que rechace alternativas válidas por no reproducir una única secuencia textual. Las reglas de puntuación se fijarán antes de observar el conjunto final.

El piloto comenzará con diez casos de desarrollo que cubran operaciones básicas y referencias; después se completarán los 60 casos. Los primeros diez forman parte de esos 60 y no constituyen datos adicionales. El objetivo será confirmar trazabilidad, reinicio de estado, límites de salida, tiempos de espera y aplicación de la rúbrica. Las repeticiones del piloto servirán para estimar variabilidad y coste, sin presentarse como resultados finales.

En cada ejecución se seguirá esta secuencia:

1. Recuperar la orden y su identificador del manifiesto experimental. La arquitectura recibirá únicamente el texto de la orden, además de su configuración fija; las referencias permanecerán en el evaluador.
2. Restablecer el contexto del ensayo. Se utilizará una solicitud sin historial conversacional y, para CLIPS, un entorno sin hechos de la orden anterior. La configuración inicial prevista será ubicación desconocida, ningún objeto sostenido, foco desactivado y manipulación habilitada.
3. Iniciar la medición del recorrido completo y procesar la orden mediante la arquitectura correspondiente. Se conservarán las representaciones intermedias disponibles, sin sustituir la salida original por una corrección manual.
4. Esperar una respuesta terminada o registrar un fallo o agotamiento del tiempo establecido.
5. Conservar la respuesta cruda, comprobar el contrato de salida y detener la medición de latencia de respuesta. La evaluación semántica posterior y la revisión humana se realizarán fuera de ese intervalo.
6. Evaluar objetivos, plan y dependencias con la misma rúbrica, registrar las incidencias y asociar el resultado con la versión de la orden y de la configuración.

Se evitará mezclar respuestas de órdenes diferentes. Como la publicación de planes actual no proporciona por sí sola un identificador de solicitud, el instrumento deberá

incorporar correlación explícita o aislamiento verificable por ensayo. Después de un tiempo agotado se cancelará o aislará el trabajo pendiente antes de enviar otra orden. Vaciar una variable local sin controlar publicaciones tardías no basta para garantizar la correspondencia.

El manifiesto conservará las huellas de las entradas, referencias, adaptador, prompt, esquemas, normalizador, catálogos y reglas; las revisiones del código; las versiones del entorno; los parámetros de generación; los tiempos de espera; y la política de reintentos. Tras el piloto se fijarán estos elementos. Si posteriormente se corrige un componente usando errores del conjunto final, la nueva configuración se distinguirá como una evaluación posterior y no sustituirá silenciosamente la comparación original.

6.1.3. Repeticiones y control de variabilidad

Se propone realizar tres repeticiones independientes por orden y arquitectura en la evaluación final. El número se confirmará mediante la estimación de coste y duración del piloto, antes de ejecutar el conjunto reservado. No se escogerá la mejor respuesta de las repeticiones. Qwen conservará generación sin muestreo; ello no elimina la necesidad de registrar variación de latencia y posibles diferencias de la infraestructura.

El orden de las condiciones se aleatorizará por bloques con una semilla registrada, alternando qué arquitectura procesa primero cada orden. Las ejecuciones serán secuenciales para evitar interferencia entre solicitudes y atribución ambigua de planes. Los identificadores de repetición vincularán los resultados pareados, aunque no impliquen compartir una semilla de generación entre proveedores.

Los reintentos no serán correcciones selectivas de respuestas erróneas. Se propone conservar la primera tentativa como observación principal y registrar cualquier reintento de infraestructura por separado. Un error de servicio o tiempo agotado se mantendrá en el denominador de disponibilidad y de éxito de extremo a extremo. El análisis condicionado a respuestas recibidas se identificará como secundario.

Las variantes y repeticiones de una misma intención están relacionadas. Para comparar tasas se promediarán primero los resultados de cada intención y se calculará la diferencia pareada entre arquitecturas. Se propone estimar un intervalo de confianza del 95 % mediante remuestreo de intenciones completas, manteniendo dentro de cada grupo sus variantes, repeticiones y ambos sistemas. El remuestreo se estratificará por familia cuando el diseño final lo requiera; se registrarán su semilla y número de remuestras. No se tratarán todas las variantes como observaciones independientes.

6.2. Condiciones experimentales

Ambas arquitecturas recibirán las mismas órdenes originales en inglés y se evaluarán con el mismo vocabulario semántico, catálogos, repertorio de acciones y convenciones de cierre. El normalizador de Qwen permanecerá como parte de su arquitectura; GPT conservará el procesamiento establecido por su prompt. La comparación principal medirá esas configuraciones completas. Una eventual ablación del normalizador se definirá como análisis adicional, sin mezclar sus resultados con la condición principal.

6.2.1. Arquitectura Qwen–CLIPS

La condición Qwen–CLIPS utilizará el adaptador especializado de Qwen2.5-7B seleccionado mediante pérdida de validación. El entrenamiento registrado seleccionó el checkpoint 5000 y guardó el adaptador restaurado en `nl2cd_qwen7b`. Para cada campaña se verificará la identidad del artefacto cargado por el servidor, de modo que una modificación posterior de esa carpeta no altere inadvertidamente la condición.

La cadena experimental será:

$$x \longrightarrow G_{\text{pred}}^{\text{Qwen}} \longrightarrow \text{Adaptar}(G_{\text{pred}}^{\text{Qwen}}) \longrightarrow P^{\text{Qwen–CLIPS}}. \tag{6.1}$$

Se conservarán el texto normalizado, los objetivos predichos antes de su adaptación, las transformaciones realizadas, los hechos insertados y el plan. Los objetivos originales se utilizarán para evaluar la interpretación del modelo; los hechos permitirán examinar pérdidas o cambios introducidos en la integración.

El límite actual de generación y los tiempos de espera se comprobarán con las órdenes compuestas del piloto. Si se ajustan, se documentarán los nuevos valores antes de la prueba final. No se impondrá igualdad numérica de tokens con GPT, porque Qwen produce objetivos y GPT produce además el plan; se establecerán límites adecuados a cada contrato y se informará el truncamiento.

6.2.2. Arquitectura GPT directo

La condición GPT utilizará el programa independiente `planner_test_node.py`, el prompt del dominio y el esquema de respuesta estructurada. El modelo configurado actualmente es `gpt-5.6-terra`; se registrarán el identificador solicitado, el informado

por el servicio y la fecha de ejecución. El recorrido será:

$$x \longrightarrow \left(G_{\text{pred}}^{\text{GPT}}, P^{\text{GPT}}\right). \tag{6.2}$$

Los objetivos y el plan se producen en una misma respuesta. No se enviarán los objetivos de GPT a CLIPS para construir o corregir su plan.

La independencia del programa es compatible con la comparación, porque el producto evaluado es un plan representado bajo el contrato de Justina. No es necesario conectar GPT al ejecutor físico para medir ese producto. Sí será necesario aplicar las mismas condiciones iniciales declaradas y la misma rúbrica de contenido. La latencia incluirá el acceso al servicio externo y se interpretará como rendimiento del sistema desplegado.

El prompt comunica convenciones operativas próximas a las utilizadas por CLIPS. Por tanto, la comparación corresponde a generación directa guiada por esas instrucciones frente a expansión mediante reglas. No se interpretará como una prueba de planificación sin conocimiento del dominio ni como un experimento que aísla únicamente el efecto de CLIPS.

6.2.3. Control con objetivos de referencia y CLIPS

La condición de control reemplazará la predicción de Qwen por G_{ref} y conservará el mismo adaptador y las mismas reglas de la cadena principal:

$$G_{\text{ref}} \longrightarrow \text{Adaptar}(G_{\text{ref}}) \longrightarrow P^{\text{control}}. \tag{6.3}$$

Se utilizará el adaptador del recorrido de ejecución, no otro conversor que produzca hechos diferentes. Se reiniciará el estado bajo las mismas condiciones. Este control no constituye una tercera arquitectura competidora y su latencia no se comparará como si incluyera interpretación lingüística.

Si el control satisface una orden y Qwen–CLIPS no, la inspección de los objetivos y hechos ayudará a localizar la diferencia. Si el control también falla, se revisarán la referencia, la adaptación, las reglas y el contrato de acciones. La coincidencia de fallos no prueba por sí sola una causa concreta. El plan de control se evaluará con la rúbrica independiente; no se convertirá automáticamente en la respuesta correcta contra la cual juzgar a GPT.

6.3. Evaluación de la interpretación semántica

6.3.1. Exactitud de objetivos

Se distinguirán coincidencia exacta y equivalencia semántica. La primera requerirá que la lista de objetivos predicha coincida con la referencia canónica, conservando orden y parámetros. Para el conjunto resoluble,

$$\mathrm{EM}_G^a = \frac{1}{NR} \sum_{i=1}^{N} \sum_{r=1}^{R} \mathbf{1}\left[G_{\mathrm{pred},i,r}^a = G_{\mathrm{ref},i}\right], \tag{6.4}$$

donde a identifica la arquitectura, N el número de órdenes y R el número fijado de repeticiones. Una salida ausente o no interpretable no constituirá coincidencia. Las órdenes ambiguas o fuera de dominio se evaluarán separadamente y no tendrán una lista vacía como referencia correcta.

La equivalencia semántica permitirá variantes previamente justificadas que conserven subtareas, entidades y relaciones. Por ejemplo, un sinónimo de una propiedad de tamaño puede diferir textualmente sin cambiar la intención. No se permitirá eliminar atributos, intercambiar destinatarios ni reordenar dependencias para aumentar artificialmente la coincidencia. Las decisiones de equivalencia se documentarán con reglas comunes para ambos sistemas.

6.3.2. Validez estructural

Se verificará que la salida pueda analizarse, que los objetivos utilicen el vocabulario final y que sus argumentos cumplan el esquema. Los doce objetivos considerados son `go`, `find`, `take`, `deliver`, `place`, `guide`, `follow`, `count`, `tell`, `save`, `answer_question` y `greet`. Un esquema aceptado no garantiza correspondencia con la orden. Se conservarán separadas la validez de estructura, la corrección semántica y la disponibilidad de respuesta.

6.3.3. Entidades y parámetros

La revisión comprobará objeto, persona, ubicación de origen, destino, destinatario, tipo de entidad y atributos exigidos. Se distinguirán omisiones, sustituciones y entidades inventadas. La medida agregada de orden completamente correcta se acompañará de

tasas por campo cuando haya suficientes casos.

Las secuencias de distinta longitud requieren alineación de subtareas: no se comparará ciegamente el objetivo de una posición con el de la misma posición en una lista desplazada. Las omisiones y adiciones conservarán su penalización y la alineación no podrá ocultar cambios de orden relevantes. Las tasas de acierto sobre campos evaluables informarán su denominador y se acompañarán del resultado sobre todas las órdenes.

6.3.4. Dependencias entre objetivos

Se verificará la conservación de referencias y precedencias: buscar antes de adquirir, adquirir antes de entregar, mantener el mismo objeto durante el transporte y asociar la interacción con la persona solicitada. Las fichas describirán estas relaciones sin exigir que todo orden alternativo sea idéntico al de la referencia si respeta las mismas dependencias.

Un ejemplo de desarrollo es *Bring me a mug from the cabinet*. La referencia propuesta contiene:

```
go(cabinet)
find(mug, kind=object)
take(mug)
deliver(mug, to=me)
```

Un resultado que cambie `cabinet` por otra ubicación tendrá un error de interpretación aunque produzca después un plan bien formado. Si los objetivos son correctos pero el plan omite la adquisición, el fallo aparecerá en la etapa de planificación. Este ejemplo es ilustrativo y no describe una salida experimental observada.

6.4. Evaluación de la planificación

La evaluación del plan utilizará subtareas y restricciones, no igualdad literal con una única lista de acciones. Se admitirán anuncios equivalentes y referencias internas consistentes, preservando los efectos exigidos. La revisión automatizada se complementará con inspección humana cuando la equivalencia no quede resuelta por los verificadores. Se ocultará la etiqueta de arquitectura durante esa revisión en la medida en que el formato lo permita.

6.4.1. Acciones válidas

Se comprobarán nombres de acciones, número y tipo de argumentos, entidades admisibles, numeración y convenciones de inicio y cierre. La separación entre anuncios y operaciones deberá respetar el contrato compartido. Una respuesta con JSON válido o con un marcador de finalización no se considerará un plan correcto únicamente por esas propiedades.

6.4.2. Consistencia del plan

Se recorrerá la secuencia mediante un estado simbólico mínimo que represente ubicación prevista, posesión del objeto, referente de interacción y datos adquiridos cuando corresponda. El contrato del ejecutor determinará qué acciones modifican esas variables. Este recorrido es un verificador de consistencia, no una simulación de percepción o dinámica física.

Los mensajes hablados ordinarios no acreditarán por sí solos adquirir un objeto, seguir a una persona o comprobar una identidad. Si una clave de protocolo activa una operación en el ejecutor, será necesario documentar ese efecto antes de concederle cumplimiento. Cuando no pueda precisarse, el resultado se marcará como no verificable con el contrato disponible.

6.4.3. Cumplimiento de subtareas

Para cada orden se comprobará si el plan cubre todas las subtareas y respeta las restricciones obligatorias. La variable principal será el cumplimiento completo del plan en el bloque principal previamente definido:

$$C_{i,r}^{a} = \begin{cases} 1, & \text{si el plan es válido y satisface todas las exigencias;} \\ 0, & \text{si hay incumplimiento o no se obtiene un plan evaluable.} \end{cases} \tag{6.5}$$

La tasa correspondiente será

$$\text{Cumplimiento}^{a} = \frac{1}{NR} \sum_{i=1}^{N} \sum_{r=1}^{R} C_{i,r}^{a}. \tag{6.6}$$

Se informarán también las fracciones de subtareas cumplidas para localizar fallos parciales, sin sustituir la medida de orden completa. Un indicador secundario exigirá si-

multáneamente interpretación correcta y plan correcto, de modo que un plan acertado acompañado de objetivos incorrectos no oculte una inconsistencia entre productos.

La categoría *no verificable* se conservará separada de *incumple* en el diagnóstico. Si surge en un caso del bloque principal después de fijar el protocolo, no se eliminará silenciosamente del denominador: se informará como éxito no acreditado, se explicará la carencia del evaluador y se presentará un análisis de sensibilidad. La proporción de casos verificables permitirá valorar el alcance efectivo de la medida.

6.4.4. Precondiciones y dependencias

El verificador comprobará que los efectos requeridos estén disponibles antes de cada acción dependiente. Para la orden de la taza, no bastará con enumerar una entrega: el plan deberá localizar y adquirir la taza, conservar su referencia y dirigir la entrega al operador. Cuando el contrato de entrega incluya movimientos de manera implícita, se reconocerán esos efectos documentados sin exigir una acción redundante de navegación.

Las condiciones iniciales serán supuestos declarados del ensayo. La existencia y detectabilidad física de los objetos o personas no se comprobará en este experimento. Tampoco se asumirán efectos simplemente porque CLIPS actualice un hecho interno: el estado previsto del plan se contrastará con el significado del contrato de acciones.

6.4.5. Planificabilidad y terminación

Se registrará si la cadena produce un plan terminado bajo su contrato y si quedan objetivos pendientes o diagnósticos de falta de soporte. En CLIPS se examinará la terminación efectiva del procesamiento, no sólo la recepción de un mensaje no vacío. Una regla alternativa que consuma un objetivo sin satisfacerlo podrá permitir terminar, pero no acreditará cumplimiento.

Se diferenciarán cuatro hechos: respuesta recibida, formato válido, terminación del proceso y cumplimiento de la orden. Esta separación permitirá explicar planes que concluyen formalmente y omiten tareas, así como fallos de infraestructura que impiden observar el resultado del planificador.

6.5. Clasificación de errores

6.5.1. Errores de interpretación

Se clasificarán como errores de interpretación las subtareas omitidas o inventadas, las entidades incorrectas, la pérdida de atributos, la resolución errónea de pronombres y el cambio de dependencias en los objetivos predichos. En Qwen se examinará también si el normalizador altera la orden antes de la inferencia. Una modificación errónea del texto se distinguirá del fallo de generación del modelo dentro de la cadena completa.

6.5.2. Errores de adaptación y planificación

Los errores de adaptación corresponderán a diferencias entre los objetivos producidos y los hechos insertados, por ejemplo pérdida de un atributo o reasignación de un tipo de entidad. Los errores de planificación incluirán acciones inválidas, precondiciones incumplidas, referencias inconsistentes y subtareas no materializadas pese a contar con objetivos suficientes.

También se registrarán separadamente truncamiento, ausencia de respuesta, agotamiento del tiempo, fallos de servicio y problemas de asociación entre solicitud y plan. No se atribuirá automáticamente a razonamiento un fallo cuyo origen sólo sea observable como error técnico.

6.5.3. Errores combinados y alcance de la atribución

Una ejecución podrá recibir más de una etiqueta. Por ejemplo, una entidad incorrecta en los objetivos y una entrega sin adquisición son errores distintos. Se registrará el primer punto de divergencia observable y las incidencias posteriores, sin asumir que todas comparten una causa.

La condición de control permitirá examinar la cadena simbólica con referencias revisadas. En GPT, objetivos y plan se generan conjuntamente; su comparación permite detectar inconsistencias entre productos, pero no demuestra una separación causal interna entre interpretación y planificación. Las tablas de errores conservarán esta diferencia de observabilidad entre arquitecturas.

6.6. Robustez, límites del dominio y alcance físico

6.6.1. Variación lingüística y composición

Las variantes canónica, parafraseada y con ruido leve conservarán una misma intención revisada. Se comparará el cumplimiento entre variantes y se registrará cuántos grupos mantienen éxito en todas ellas. Obtener tres respuestas equivalentes pero incorrectas no se interpretará como robustez satisfactoria; la estabilidad se informará junto con la corrección.

Los casos compuestos permitirán evaluar conservación de referentes y dependencias al aumentar las subtareas. El análisis se organizará por número de objetivos, entidades y cambios de referente. Un resultado peor en órdenes largas no se atribuirá únicamente a su longitud cuando también cambien familia o dificultad. Se mantendrá una descripción de esas diferencias.

6.6.2. Ambigüedad, restricciones y capacidades no verificables

Se examinarán órdenes con referentes ausentes, destinos no especificados, acciones fuera del repertorio y contradicciones. Para GPT se evaluará si la respuesta solicita la información necesaria o explica adecuadamente el rechazo, sin generar un plan basado en supuestos inventados. Se separará un rechazo del dominio de un rechazo de la API o error del servicio.

Qwen no implementa el mismo contrato conversacional de aclaración y rechazo. Se registrará su conducta observable —bloqueo, error, objetivos propuestos o plan generado— sin convertir un error de formato en una aclaración correcta. Estos resultados describirán cobertura funcional y límites; no se mezclarán con la tasa principal de tareas resolubles.

Los diagnósticos incluirán atributos conjuntos de personas, conteos filtrados, seguimiento, identificación por nombre y propiedades que no puedan determinarse con los efectos disponibles. Por ejemplo, tamaño y peso no se considerarán intercambiables. Un plan que use una aproximación no autorizada se distinguirá de uno que satisface la propiedad solicitada. Las referencias no se modificarán para aceptar una limitación común de las dos arquitecturas.

6.6.3. Delimitación de la evaluación física

El protocolo principal no requiere accionar al robot. Se detendrá el recorrido en la obtención y verificación del plan; los nodos ejecutores físicos no participarán en la campaña. Esta delimitación permitirá completar la comparación aun cuando la plataforma presente problemas técnicos.

Las conclusiones se referirán a fidelidad semántica, consistencia simbólica y rendimiento computacional. No permitirán estimar probabilidad de completar una tarea en un entorno real ni certificar ausencia de daños. Si más adelante se realizan demostraciones físicas, se describirán como una etapa adicional con condiciones, casos y métricas propios. No se exigirá simetría física entre una arquitectura integrada con ROS y un programa de evaluación independiente para responder las preguntas de este protocolo.

6.7. Rendimiento computacional

6.7.1. Latencia

La medida principal de latencia comenzará antes de enviar la orden original y terminará cuando el evaluador reciba una respuesta terminal y complete la validación automática del contrato. Para Qwen–CLIPS incluirá comunicación, normalización, inferencia, adaptación y expansión; para GPT incluirá la solicitud remota, generación, recepción y validación. La calificación semántica detallada posterior quedará fuera de este intervalo.

Se mantendrá cargado el modelo local para medir operación habitual. La carga inicial, la puesta en marcha de servicios y las solicitudes de calentamiento se registrarán por separado y utilizarán casos de desarrollo, no entradas de prueba final. Se fijará una única modalidad de despliegue de Qwen para la comparación principal; una modalidad remota adicional se identificará como otra condición de infraestructura.

Se presentarán mediana y percentil 95, junto con el número de observaciones, la tasa de respuestas recibidas y la de planes correctos. Los tiempos hasta fallo se informarán aparte; un fallo rápido no se interpretará como una ventaja de latencia. Los tiempos agotados se conservarán como observaciones censuradas al umbral declarado, sin asignarles un tiempo de éxito inventado. Los tiempos internos de Qwen o CLIPS servirán para diagnóstico, pero no se compararán directamente con la latencia total del servicio GPT.

6.7.2. Uso de recursos

En la plataforma local se registrarán memoria RAM y memoria GPU observables, especificando si la medida corresponde al proceso, a memoria asignada por el framework o al dispositivo completo. Se indicarán la frecuencia de muestreo, la carga residente y el máximo observado durante la campaña. Un máximo acumulado desde el inicio del proceso no se presentará como un máximo exclusivo de una solicitud.

Para GPT no se dispone de acceso a los recursos internos del proveedor. La memoria del cliente local no representa la memoria necesaria para ejecutar el modelo remoto. Por tanto, se presentarán por separado los requisitos locales de Qwen–CLIPS y los recursos observables del acceso al servicio GPT, sin afirmar una comparación directa de consumo interno o energía.

6.7.3. Tokens y coste del modelo GPT

Se conservará el uso de tokens informado por el servicio para cada solicitud, incluyendo el desglose disponible de entrada, salida y caché. Los tokens de razonamiento se interpretarán según su pertenencia a los totales reportados, sin sumarlos dos veces. El coste se obtendrá con la tarifa o facturación aplicable en la fecha de ejecución, cuya fuente y moneda quedarán registradas. Si no existe una tarifa verificable para el acceso utilizado, se informarán tokens y cargo observado, sin inventar un coste teórico.

El informe distinguirá coste total de la campaña y coste por orden con plan correcto. Los reintentos y solicitudes fallidas con consumo facturado se conservarán en el total. Para Qwen, ausencia de un cargo por solicitud no equivale a coste computacional nulo; no se construirá una comparación monetaria integral sin medir y justificar los componentes adicionales.

La fijación de datos, referencias, configuraciones y denominadores permitirá relacionar los resultados del capítulo siguiente con un procedimiento explícito. El número final de casos, repeticiones y parámetros operativos se incorporará cuando concluya el piloto, conservando los registros que justifiquen esas decisiones.

Capítulo 7

Resultados

7.1. Resultados de la especialización de Qwen

7.1.1. Comportamiento durante entrenamiento

7.1.2. Evaluación del modelo seleccionado

7.2. Resultados de interpretación semántica

7.2.1. Qwen

7.2.2. GPT

7.2.3. Comparación

7.3. Resultados de planificación

7.3.1. Qwen–CLIPS

7.3.2. GPT directo

7.3.3. Condición de control CLIPS

7.3.4. Comparación

7.4. Análisis de errores

7.4.1. Errores de interpretación

7.4.2. Errores de planificación

7.4.3. Casos representativos

Capítulo 8

Discusión

8.1. Interpretación de los resultados respecto a la hipótesis

8.2. Interpretación semántica

8.2.1. Especialización de Qwen

8.2.2. Capacidad de GPT

8.2.3. Efecto de la variabilidad lingüística

8.3. Planificación de acciones

8.3.1. Expansión determinista mediante CLIPS

8.3.2. Planificación generativa mediante GPT

8.3.3. Separación entre errores de interpretación y planificación

8.4. Comparación Qwen–CLIPS frente a GPT directo

8.4.1. Exactitud

8.4.2. Consistencia

8.4.3. Latencia

Capítulo 9

Conclusiones y trabajo futuro

9.1. Conclusiones principales

9.2. Cumplimiento de los objetivos

9.3. Evaluación de la hipótesis

9.4. Contribuciones de la investigación

9.4.1. Representación canónica de órdenes

9.4.2. Arquitectura Qwen–CLIPS

9.4.3. Comparación con planificación directa mediante GPT

9.4.4. Metodología para separar errores de interpretación y planificación

9.5. Trabajo futuro

Referencias

[1] R. R. Murphy, *Introduction to AI Robotics*. MIT Press, 2nd ed., 2019.

[2] S. Russell and P. Norvig, *Artificial Intelligence: A Modern Approach*. Pearson, 4th ed., 2020.

[3] J. C. Giarratano and G. D. Riley, *Expert Systems: Principles and Programming*. Thomson Course Technology, 4th ed., 2005.

[4] R. A. Brooks, "A robust layered control system for a mobile robot," *IEEE Journal of Robotics and Automation*, vol. 2, no. 1, pp. 14–23, 1986.

[5] A. Vaswani, N. Shazeer, N. Parmar, J. Uszkoreit, L. Jones, A. N. Gomez, Ł. Kaiser, and I. Polosukhin, "Attention is all you need," in *Advances in Neural Information Processing Systems*, vol. 30, 2017.

[6] L. Shi, X. Chen, Y. Hu, J. Li, H. Zhang, and H. Zhao, "Chatgpt for robotics: Design principles and model abilities," *Microsoft Research*, 2023.

[7] NASA Johnson Space Center, "Clips reference manual," tech. rep., NASA, 1991.

[8] B. Pell *et al.*, "An autonomous spacecraft agent prototype," in *First International Conference on Autonomous Agents*, pp. 253–261, 1997.

[9] J. Savage and otros, "Virbot: A modular architecture for service robots with hybrid planning," in *RoboCup Symposium*, 2024.

[10] L. Cheng, Y. Mao, and T. Ward, "Editorial: Neural network models in autonomous robotics," *Frontiers in Neurorobotics*, 2025.

[11] Alibaba Cloud Qwen Team, "Qwen official github repository," 2026.

[12] OpenAI, "Openai developer documentation and model guides," 2026.

[13] N. Houlsby, A. Giurgiu, S. Jastrzebski, B. Morrone, Q. De Laroussilhe, A. Gesmundo, M. Attariyan, and S. Gelly, "Parameter-efficient transfer learning for

NLP," in *Proceedings of the 36th International Conference on Machine Learning*, vol. 97 of *Proceedings of Machine Learning Research*, pp. 2790–2799, PMLR, 2019.

[14] E. J. Hu, Y. Shen, P. Wallis, Z. Allen-Zhu, Y. Li, S. Wang, L. Wang, and W. Chen, "LoRA: Low-rank adaptation of large language models," in *International Conference on Learning Representations*, 2022.

[15] T. Dettmers, A. Pagnoni, A. Holtzman, and L. Zettlemoyer, "QLoRA: Efficient finetuning of quantized LLMs," in *Advances in Neural Information Processing Systems*, vol. 36, pp. 10088–10115, 2023.

[16] A. d'Avila Garcez, T. R. Besold, L. De Raedt, P. Földiák, P. Hitzler, T. Icard, K.-U. Kühnberger, L. C. Lamb, R. Miikkulainen, and D. L. Silver, "Neural-symbolic learning and reasoning: Contributions and challenges," in *Knowledge Representation and Reasoning: Integrating Symbolic and Neural Approaches: Papers from the 2015 AAAI Spring Symposium*, pp. 18–21, AAAI Press, 2015.

[17] T. R. Besold, A. d'Avila Garcez, S. Bader, H. Bowman, P. Domingos, P. Hitzler, K.-U. Kuehnberger, L. C. Lamb, D. Lowd, P. M. V. Lima, L. de Penning, G. Pinkas, H. Poon, and G. Zaverucha, "Neural-symbolic learning and reasoning: A survey and interpretation," *arXiv preprint arXiv:1711.03902*, 2017.

[18] B. Liu, Y. Jiang, X. Zhang, Q. Liu, S. Zhang, J. Biswas, and P. Stone, "LLM+P: Empowering large language models with optimal planning proficiency," *arXiv preprint arXiv:2304.11477*, 2023.

[19] A. Yang, B. Yang, B. Zhang, B. Hui, B. Zheng, B. Yu, C. Li, D. Liu, F. Huang, H. Wei, *et al.*, "Qwen2.5 Technical Report," *arXiv preprint arXiv:2412.15115*, 2024.

[20] J. Hart, A. Moriarty, K. Pasternak, J. Kummert, M. Leonetti, L. Contreras, L. Ruegemer, A. Mitzutani, T. Ribeiro, A. Golding, T. Kang, and F. Pimentel, "RoboCup@Home 2026: Rules and Regulations." `https://github.com/RoboCupAtHome/RuleBook/releases/tag/2026.2`, 2026. RoboCup@Home Technical Committee.

[21] RoboCup@Home, "CommandGenerator: Command Generator for RoboCup@Home." `https://github.com/RoboCupAtHome/CommandGenerator`, 2026. Repositorio oficial de RoboCup@Home.

[22] RoboCup@Home, "CompetitionTemplate." `https://github.com/RoboCupAtHome/CompetitionTemplate`, 2026. Repositorio oficial de RoboCup@Home.